Universidad de Alcalá

TRABAJO DE FIN DE GRADO

GRADO EN INGENIERÍA DE COMPUTADORES

ESCUELA POLITÉCNICA SUPERIOR

Desarrollo de un simulador interactivo para el movimiento coordinado óptimo de dos robots

Daniel Senespleda Fernández

Tutor: Guillermo Esteban Pascual

Cotutor: David Orden Martín

2025/2026

Resumen

Este trabajo de fin de grado presenta el desarrollo de un simulador web interactivo diseñado para la planificación y visualización de movimientos coordinados entre robots en entornos compartidos. El objetivo principal es la resolución de conflictos de trayectoria y la evasión de colisiones mediante algoritmos geométricos avanzados, modelando a los robots como entidades circulares. De esta forma, el simulador permite al usuario definir las posiciones y los radios de los robots para observar cómo el sistema identifica y resuelve automáticamente los diferentes escenarios posibles.

El motor de simulación clasifica dichos escenarios en tres categorías principales: casos fáciles, anidados o multi obstruyentes. Para los escenarios de alta complejidad, se han implementado lógicas de asignación dinámica de roles (quién actúa como el robot anidado y quién como obstáculo) y técnicas de inversión temporal para calcular maniobras de evasión desde los destinos. Además, el sistema incorpora una función que evalúa simultáneamente trayectorias en sentido horario y antihorario, seleccionando siempre la ruta que minimiza la distancia total recorrida.

El simulador ha sido desarrollado empleando Next.js (un framework de *React* enfocado en la construcción de aplicaciones web escalables y de alto rendimiento) y *TypeScript* (un superconjunto tipado de JavaScript que garantiza una mayor robustez y prevención de errores en el código) para su diseño y comportamiento y la API de Canvas (una interfaz nativa de HTML5 para el renderizado dinámico de gráficos 2D acelerados por hardware) para el renderizado de alta fidelidad en tiempo real. La interfaz incluye herramientas de control como una cámara inteligente con auto-centrado para la correcta visualización de los robots, un sistema de capturas de estado para la repetición de experimentos y una representación visual detallada de los conos de tangencia y zonas de maniobra.

Palabras clave: Robótica móvil, evasión de colisiones, simulador web, geometría analítica, planificación de trayectorias, optimización de rutas, TypeScript, Next.js.

Abstract

This bachelor's thesis presents the development of an interactive web simulator designed for the planning and visualization of coordinated motions between robots in shared environments. The main objective is the resolution of trajectory conflicts and collision avoidance through advanced geometric algorithms, modeling the robots as circular entities. Thus, the simulator allows the user to define the positions and radii of the robots to observe how the system automatically identifies and resolves different possible scenarios.

The simulation engine classifies these scenarios into three main categories: Easy, Nested, or Multi-Obstruction cases. For high-complexity scenarios, dynamic role allocation logic (determining who acts as the nested robot and who as the obstacle) and time reversal techniques have been implemented to calculate evasion manoeuvres from the destinations. Furthermore, the system incorporates an algorithm that simultaneously evaluates clockwise and counter-clockwise trajectories, consistently selecting the route that minimises the total distance travelled.

The simulator has been developed using Next.js (a React framework focused on building scalable, high-performance web applications) and TypeScript for its structural design and behaviour, alongside the Canvas API for high-fidelity, real-time rendering. The interface includes control tools such as a smart camera with auto-centring for proper robot visualization, a state capture system for experiment repetition, and a detailed visual representation of tangency cones and manoeuvring zones.

Keywords: Mobile robotics, collision avoidance, web simulator, analytic geometry, trajectory planning, route optimization, TypeScript, Next.js.

Índice general

Resumen 2
Abstract 3
Índice general 4
Capítulo 1. Introducción 6
1.1. Motivación 6
1.2. Objetivos 7
1.3. Estructura del documento 8
Capítulo 2. Marco Teórico 10
2.1. Estado del arte en evasión de colisiones 10
2.2. Fundamentos geométricos del Velocity Obstacle 11
2.3. Clasificación de escenarios de conflicto 12
2.4. Generación de trayectorias y análisis espacial 13
2.4.1. Cálculo de tangentes 13
2.4.2. Partición por Zonas y Mundos Paralelos 14
2.5. Tecnologías de desarrollo web 16
2.5.1. Next.js y React 17
2.5.2. TypeScript 17
2.5.3. HTML5 Canvas API 17
Capítulo 3 – Desarrollo y Algoritmia 18
3.1. Arquitectura del motor lógico 18
3.2. Clasificación de escenarios 19
3.3. Algoritmia del caso Fácil (Easy) 21
3.4. Algoritmia del caso Anidado (Nested) 22
3.5. Algoritmia del caso Multi Obstruyente (Multi-Obstruction) 26
3.6. Asignación de roles 35
3.7. Evaluación dual y optimización de rutas 36
Capítulo 4. Arquitectura e implementación computacional 39
4.1. Arquitectura general del sistema 39
4.2. Capa de interfaz y controles 40
4.2.1. El lienzo interactivo 40
4.2.2. El panel de control 45
4.3. Capa de lógica y geometría 47
4.3.1. Funciones de geometría analítica y clasificación 47
4.3.2. Orquestación, evaluación dual y competición de mundos 50
4.3.3. Gestión de estado y sincronización 51

4.4. Capa de renderizado 52
4.4.1. El bucle de animación 53
4.4.2. Dibujado por capas y componentes gráficos 54
4.4.3. Compatibilidad y soporte del entorno web 55
Capítulo 5: Conclusiones y Trabajo Futuro 57
5.1. Dificultades y retos superados 57
5.2. Conclusiones 60
5.3. Mejoras a futuro 61
Bibliografía 63
Enlaces de acceso 64
Anexo 65
simulationLogic.tsx 65
page.tsx 67

Capítulo 1. Introducción

En la última década, la robótica móvil ha experimentado un gran crecimiento, integrándose en entornos que van desde grandes almacenes automatizados hasta la circulación de vehículos autónomos en espacios urbanos compartidos. A medida que aumenta el número de robots operando en un mismo espacio, surge un gran desafío: la planificación de trayectorias seguras y la evasión de colisiones en tiempo real.

1.1. Motivación

A pesar de que existen algoritmos matemáticos para la evasión de colisiones, comprenderlos, depurarlos y validar su comportamiento suele requerir el uso de simuladores pesados y complejos, como Gazebo (Open Robotics, s.f, -a) o ROS (Open Robotics, s.f, -b) entre otros. Estos entornos de simulación exigen un tiempo considerable de aprendizaje y no siempre ofrecen una visualización clara y aislada de la geometría que hay detrás de las decisiones de los robots.

En particular, este proyecto se fundamenta en el marco teórico desarrollado inicialmente por Kirkpatrick y Liu (2016) para la caracterización de movimientos de longitud mínima en el caso de dos discos. La motivación surge de la necesidad de trasladar estos conceptos a un entorno práctico, abarcando también los estudios posteriores que adaptan esta base teórica para trayectorias óptimas con robots cuadrados (Esteban et al., 2025) y cualquier otra figura geométrica simétrica respecto a un centro (Kirkpatrick y Liu, 2025).Dichas investigaciones demuestran de forma analítica un método para calcular la trayectoria óptima que permite a dos robots alcanzar sus respectivos destinos minimizando la distancia total, incluso cuando sus caminos originales se obstruyen. Es importante diferenciar este escenario de otros problemas clásicos de la robótica, como las trayectorias de Reeds y Shepp (1990) para vehículos con restricciones de giro (como un coche que no puede desplazarse lateralmente). En nuestro caso, los robots tienen libertad total de traslación, lo que paradójicamente complica la búsqueda de la ruta óptima debido al enorme abanico de maniobras de evasión posibles que el sistema debe evaluar.

Aunque este modelo teórico demuestra matemáticamente la viabilidad de las trayectorias, su estudio, validación y comprensión exigen una herramienta gráfica capaz de representar

gráficamente los movimientos. Como ocurre en cualquier avance teórico en robótica, disponer de un simulador dedicado es fundamental para analizar casos límite, observar la toma de decisiones en tiempo real y aprender del comportamiento del algoritmo.

Desde un punto de vista de la ingeniería de software, las tecnologías web modernas y las APIs de renderizado en navegadores ofrecen la oportunidad perfecta para construir este "puente" entre la teoría y la práctica. Por lo tanto, la motivación principal de este proyecto es crear una herramienta ligera, accesible y visual que dé vida a este modelo matemático, permitiendo simular las interacciones entre los robots y visualizar la geometría que define las rutas óptimas. Con el fin de facilitar la evaluación interactiva de la herramienta y permitir su uso en tiempo real, el simulador desarrollado se encuentra desplegado de forma pública en la siguiente dirección web:

https://simulador-robots-tfg-daniel-senespl.vercel.app/

1.2. Objetivos

El objetivo principal de este Trabajo de Fin de Grado es desarrollar un simulador web interactivo que permita la planificación, visualización y resolución de conflictos de trayectoria entre robots móviles, utilizando para ello algoritmos de geometría analítica y prevención de colisiones.

Para alcanzar este objetivo principal, se establecen los siguientes objetivos específicos:

- **Desarrollar una interfaz web interactiva y de alto rendimiento:** Utilizando Next.js y la API de Canvas para permitir a los usuarios posicionar, dimensionar y arrastrar robots en un entorno bidimensional en tiempo real de forma fluida.
- **Clasificar escenarios de riesgo:** Implementar un motor lógico capaz de analizar las trayectorias y los radios de los robots para identificar automáticamente si se enfrentan a un paso libre (*Easy*), a un caso donde un robot debe rodear al otro (*Nested*) o a un cruce complejo con obstrucciones mutuas (*Multi-Obstruction*).

- **Resolver colisiones mediante geometría avanzada:** Programar el cálculo exacto de tangentes, intersecciones y conos de peligro (*Velocity Obstacles*) para determinar las zonas seguras de maniobra.

- **Implementar dinámicas de prioridad y tiempo invertido:** Desarrollar algoritmos que decidan qué robot debe maniobrar y cuál tiene prioridad de paso, incluyendo técnicas de inversión temporal (*Time Reversal*) para resolver obstrucciones que ocurren cerca del destino final.

- **Optimizar las rutas de evasión:** Diseñar un sistema que calcule simultáneamente la evasión rodeando el obstáculo en sentido horario (*Clockwise*) y antihorario (*Counter-Clockwise*), utilizando la distancia euclídea para seleccionar matemáticamente el camino más corto.

- **Crear herramientas de análisis visual:** Dibujar en pantalla las líneas de construcción, los puntos de intersección y las zonas de maniobra para facilitar la comprensión del algoritmo.

- **Mejorar la experiencia de usuario (UX):** Incorporar funcionalidades como una cámara inteligente con auto-centrado, controles de zoom y un sistema de "Restaurar" (*Snapshots*) que guarde y recupere las posiciones de los experimentos para facilitar la repetición de pruebas.

1.3. Estructura del documento

Este trabajo se organiza en los siguientes capítulos para detallar el proceso de investigación y desarrollo:

En primer lugar, en el **Capítulo 2 - Marco teórico** se explican los conceptos físicos y matemáticos en los que se apoya el simulador. Se abordan los fundamentos de la navegación robótica, los *Velocity Obstacles*, el cálculo geométrico de tangentes e intersecciones, así como las tecnologías web utilizadas (Next.js, TypeScript y Canvas API).

A continuación, en el **Capítulo 3 - Desarrollo y Algoritmia** se detalla la implementación técnica del motor de simulación. Se desglosa el código responsable de la clasificación de

escenarios, la lógica de los casos *Easy*, *Nested* y *Multi-Obstruction*, la evaluación de rutas (*Clockwise* vs. *Counter-Clockwise*) y el bucle de renderizado y físicas.

Seguidamente, en el **Capítulo 4 – Arquitectura e implementación computacional** se describen la arquitectura del proyecto, el diseño de la interfaz de usuario y la forma en la que se ha estructurado el estado de la aplicación para manejar la interactividad en tiempo real.

Por último, en el **Capítulo 5 - Conclusiones y Trabajo Futuro**, se evalúan los objetivos alcanzados, se reflexiona sobre la eficiencia matemática del simulador y se proponen posibles mejoras, como la escalabilidad a escenarios con más de dos robots o la exportación de trayectorias a sistemas robóticos reales.

Capítulo 2. Marco Teórico

Este capítulo establece las bases conceptuales y matemáticas necesarias para el desarrollo del simulador de movimiento coordinado. Se comienza con una revisión de las técnicas clásicas de evasión de obstáculos, para posteriormente profundizar en la formulación geométrica en la que se basa el algoritmo central de este trabajo.

2.1. Estado del arte en evasión de colisiones

En el ámbito de la robótica móvil, la navegación segura en entornos dinámicos y multi-robot es un problema ampliamente estudiado (Choset et al., 2005). Las estrategias para coordinar el movimiento entre múltiples agentes independientes han evolucionado desde los primeros enfoques geométricos (Sharir & Sifrony, 1991) hasta dividirse históricamente en dos grandes categorías:

- Aproximaciones basadas en el espacio de configuración (Campos Potenciales): En estos métodos, el robot es atraído hacia su destino por una fuerza atractiva y repelido de los obstáculos por fuerzas de repulsión (Khatib, 1986). Aunque son computacionalmente eficientes, sufren de problemas graves como los mínimos locales, donde el robot puede quedar atrapado antes de llegar a su meta.

- Aproximaciones basadas en la velocidad (*Velocity Obstacles*): A diferencia de los campos potenciales, estas técnicas operan en el espacio de velocidades. El concepto de *Velocity Obstacle* (VO) introducido por Fiorini y Shiller (1998) supuso un cambio de paradigma, permitiendo predecir y evitar colisiones futuras calculando qué rangos de velocidad conducirán a un impacto dadas las trayectorias actuales de los agentes.

El presente trabajo se fundamenta en los modelos geométricos derivados de la teoría de obstáculos de velocidad, adaptándolos para calcular trayectorias analíticas exactas en tiempo real.

2.2. Fundamentos geométricos del Velocity Obstacle

El concepto de *Velocity Obstacle* define el conjunto de todas las velocidades relativas que resultarán en una colisión entre dos objetos en algún momento futuro.

Para el cálculo de colisiones entre dos robots circulares, A y B, con radios r_A y r_B respectivamente, es posible simplificar el espacio de configuración (*C-Space*) mediante la suma de Minkowski, un enfoque introducido clásicamente en la robótica por Lozano-Pérez (1983), tal y como se ilustra en la Figura 2.1. Este método consiste en reducir el robot A a un único punto, mientras que el robot B se expande geométricamente mediante una suma de Minkowski con el radio de A, definiendo un radio de seguridad $R_{safe} = r_A + r_B$. De esta forma, el problema original de colisión de dos cuerpos se transforma en un problema equivalente donde un punto debe evitar un obstáculo expandido.

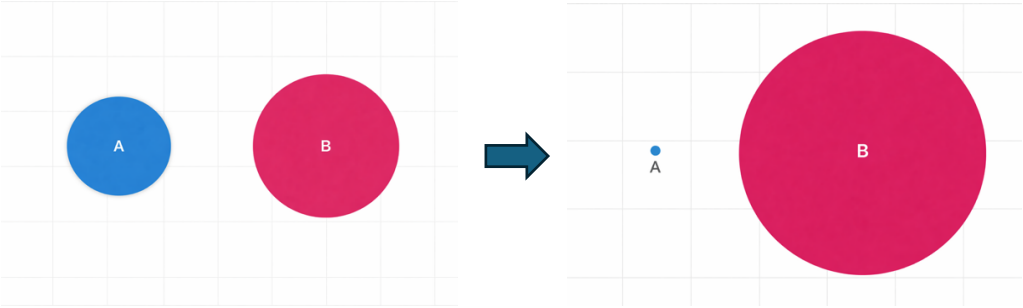


Figura 2.1: Simplificación del espacio de configuración mediante la suma de Minkowski.

Matemáticamente, el *Velocity Obstacle* que el robot B representa para el robot A se define como:

$$VO_{A|B} = \{v \mid \exists t > 0, (p_A + v * t) \in Disk(p_B, r_A + r_B)\}$$

donde $Disk(p_B,\ r_A + r_B)$ representa un disco centrado en p_B con el radio de seguridad previamente definido. Visualmente, este conjunto forma un cono (el cono de colisión) cuyo vértice se sitúa en p_A y cuyas aristas son tangentes al disco de seguridad de B.

Cualquier vector de velocidad que caiga dentro de este cono implicará una colisión inminente. Por tanto, el objetivo del sistema de control es seleccionar una velocidad (o en

nuestro caso, proyectar una trayectoria espacial) que bordee este cono por el camino más corto posible.

2.3. Clasificación de escenarios de conflicto

Para resolver la evasión de colisiones de forma exacta, el enfoque utilizado en este trabajo deja de lado los métodos tradicionales que calculan velocidades de forma probabilística. En su lugar, analiza directamente la geometría de los caminos que van a seguir los robots.

El primer paso del algoritmo consiste en evaluar la posición de los robots en el espacio. Para ello definimos el *corridor* como la franja de espacio geométrico barrida por un robot a lo largo de su trayectoria, considerando su radio de seguridad (R_{safe}), visualizado en la Figura 2.2.

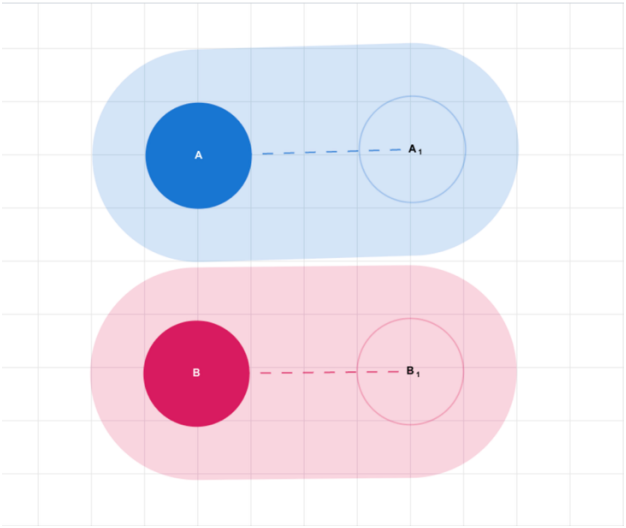


Figura 2.2: Representación de los *corridors* de los robots A y B.

Dependiendo de si un robot tiene su punto de inicio o de destino metido dentro del *corridor* por el que va a circular el otro robot, el problema se clasifica en tres escenarios diferentes:

- **Caso Fácil (Easy):** Ocurre cuando ninguna de las posiciones de los robots se encuentra dentro del *corridor* del otro robot, o cuando el cese temporal del

movimiento de uno permite el paso libre del otro. La resolución implica asignar prioridades simples (quién se mueve primero y quién espera).

- **Caso Anidado (Nested):** Se detecta cuando tanto la posición de inicio como la de fin de un robot se encuentra dentro del *corridor* del otro robot. Requiere que un robot actúe como agente dinámico principal, debiendo rodear al robot obstáculo (anidado) mediante trayectorias tangenciales. En este escenario, el algoritmo debe calcular un punto de intersección (P_{int}), el cual representa la coordenada en el espacio donde el robot anidado deberá detenerse provisionalmente para que se pueda realizar la maniobra de evasión de forma segura.

- **Caso Multi Obstruyente (Multi-Obstruction):** Ocurre cuando se producen cruces donde ambos robots invaden mutuamente el espacio del otro. Representa el escenario más complejo debido a la topología del espacio a evaluar: el *corridor* básico se sustituye por conos de velocidad, exigiendo una partición espacial significativamente más elaborada que el caso Anidado. Esta división geométrica requiere el cálculo de múltiples líneas tangentes y trayectorias curvas, cuyas ecuaciones varían dependiendo de las condiciones específicas del subescenario en el que se encuentren los robots. Además, aunque la resolución del conflicto en la mayoría de estos cruces sigue dependiendo de alcanzar un punto de intersección (P_{int}), la propia complejidad del entorno obliga a que esta coordenada se calcule mediante diferentes estrategias.

2.4. Generación de trayectorias y análisis espacial

El núcleo matemático del sistema reside en la generación de trayectorias tangentes y la evaluación de "mundos" simétricos (sentido horario vs. sentido antihorario) para encontrar la ruta de evasión óptima.

2.4.1. Cálculo de tangentes

El cálculo de las líneas tangentes es la base matemática sobre la que funciona el proyecto. Estas líneas no solo sirven para marcar por dónde debe pasar el robot para no chocar

(manteniendo siempre la distancia de seguridad), sino que actúan como líneas de corte que dividen el mapa en distintas zonas lógicas.

En el caso anidado (*Nested*), estas líneas trazadas desde el origen del robot definen los límites exactos de cuatro áreas (Zona 1, Zona 2, Zona 3 y Zona 4). Gracias a esta división, el algoritmo comprueba al instante en qué zona cae el destino. Así sabe si el robot puede ir en línea recta o si tiene que rodear al obstáculo por un lado o por otro.

En el caso Multi Obstruyente (*Multi-Obstruction*), las tangentes también se utilizan para delimitar las áreas de peligro y situar a los robots, aunque aquí hacen falta además otros cálculos para resolver el cruce.

Matemáticamente, para encontrar el ángulo de apertura α de dos líneas tangentes que van desde un punto de origen (O_x, O_y) hacia un círculo de seguridad con centro en (C_x, C_y) y radio r, se usa esta fórmula:

$$\alpha = arctan2(C_y - O_y, C_x - O_x) \pm \arcsin\left(\frac{r}{d(O,C)}\right)$$

Una vez calculado este ángulo, se puede sacar la posición exacta del punto de tangencia P_t en el plano mediante:

$$P_t = O + \sqrt{d(O,C)^2 - r^2} * [\cos(\alpha), \sin(\alpha)]$$

Al usar estas fórmulas, el simulador trabaja con coordenadas reales y precisas. Esto asegura que los caminos para esquivar sean lo más cortos posibles y vayan exactamente rozando la distancia de seguridad, sin dar rodeos innecesarios.

2.4.2. Partición por Zonas y Mundos Paralelos

Una vez establecidas las tangentes, el espacio alrededor de los robots se divide en áreas lógicas o zonas (Z_1, Z_2, Z_3 y Z_4). En ambos casos se calculan estas cuatro regiones, aunque la forma de obtenerlas cambia según el escenario. Estas zonas sirven para definir cómo se comportará el robot durante la maniobra, dependiendo de en qué área caiga exactamente la posición de destino.

La aplicación de herramientas de geometría integral para resolver este tipo de problemas de planificación de movimientos óptimos fue introducida originalmente por Icking et al. (1993). Para garantizar que el simulador no solo encuentre una ruta segura, sino matemáticamente la más corta posible, nos basamos en la relación entre la longitud de una curva convexa y su función de soporte. Según la fórmula del área superficial de Cauchy, la longitud $l(C)$ de una curva cerrada y convexa C puede calcularse mediante la integral de su función de soporte $h_C(\theta)$:

$$l(C) = \int_0^{2\pi} h_C(\theta) d\theta.$$

En el contexto de este simulador, esta propiedad matemática es fundamental. Demuestra de forma analítica que la distancia mínima absoluta para esquivar un obstáculo se consigue minimizando la anchura del espacio barrido por el movimiento, lo cual se logra geométricamente trazando líneas rectas tangentes que enlacen con el perímetro exacto de la zona de seguridad. Es decir, justifica por qué el algoritmo basa sus rutas en tangentes y arcos.

Apoyándose en este fundamento teórico, para asegurar que se elige la ruta más corta, el algoritmo evalúa la geometría en dos "mundos" distintos: el sentido horario (Clockwise) y el sentido antihorario (Counter-Clockwise). Se simula matemáticamente la división del espacio por ambos lados del robot que estorba y se suma la distancia total de los tramos rectos y curvos que habría que recorrer en cada caso. Como parte fundamental de este análisis espacial, el sistema aplica también en ciertos escenarios complejos una técnica de inversión temporal. Este método consiste en calcular la trayectoria de evasión tratando el punto de destino como si fuera el de origen. Esto es especialmente útil cuando el conflicto se produce muy cerca de la meta, permitiendo al algoritmo determinar con mayor precisión qué flanco es más seguro para determinar el recorrido.

Para obtener una medida exacta de estas opciones, evitando cualquier dependencia de la resolución gráfica o aproximaciones basadas en la suma de píxeles, el motor lógico emplea métricas analíticas. En primer lugar, calcula la distancia euclídea para evaluar la longitud de los segmentos rectos de aproximación y salida:

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

donde $(x_1,\ y_1)$ y $(x_2,\ y_2)$ representan las coordenadas cartesianas de los puntos de inicio y fin del segmento evaluado, respectivamente.

En segundo lugar, aplica la fórmula geométrica de la longitud del arco para medir con precisión el recorrido curvo a lo largo de los perímetros de seguridad:

$$L = r \cdot \theta$$

donde r representa el radio de seguridad de la circunferencia evaluada y θ es el ángulo central barrido por el arco en radianes.

Con estos valores analíticos exactos, el sistema suma las distancias de los distintos tramos, compara los resultados de todas las simulaciones paralelas y selecciona automáticamente la opción que ofrece la distancia de evasión mínima para que el recorrido sea el óptimo global.

2.5. Tecnologías de desarrollo web

Para la construcción de este simulador interactivo, se ha optado por un conjunto de tecnologías web modernas que garantizan un alto rendimiento y una arquitectura escalable. El núcleo de la aplicación esta desarrollado utilizando *React* (Meta Platforms, Inc., 2024), una librería enfocada en la creación de interfaces de usuario dinámicas, apoyada sobre el framework *Next.js* (Vercel, Inc., 2024) para la optimización del enrutado y la gestión del estado general.

Todo el desarrollo lógico y matemático se ha programado de forma estricta mediante *TypeScript* (Microsoft Corporation, 2024), lo que ha permitido evitar errores de tipo durante la compilación en cálculos geométricos complejos. Finalmente, para la representación visual y el renderizado gráfico de los robots, las zonas y las trayectorias, se ha utilizado directamente la Canvas API nativa de HTML5 (Mozilla Foundation, 2024), desacoplando así la carga gráfica de la lógica de la interfaz.

2.5.1. Next.js y React

La aplicación y su interfaz de usuario se han desarrollado con *React*, una biblioteca basada en componentes que facilita la gestión del estado dinámico del simulador, como la actualización de las posiciones de los robots o la activación y desactivación de la vista de depuración. Además, el proyecto se apoya en Next.js, que actúa como framework principal, proporcionando una estructura general, el sistema de enrutamiento y las optimizaciones necesarias para su despliegue en entorno web.

2.5.2. TypeScript

El motor lógico del simulador requiere un alto grado de precisión matemática. Por ello, se ha empleado TypeScript como lenguaje de programación principal. Al incorporar un sistema de tipado estático sobre JavaScript, permite definir con mayor rigor las estructuras de datos, como coordenadas, vectores o radios de seguridad, y verificar su coherencia durante el desarrollo, lo que reduce la posibilidad de errores en tiempo de ejecución y aporta mayor fiabilidad en los cálculos.

2.5.3. HTML5 Canvas API

Para la representación gráfica, se ha elegido la API de Canvas nativa de HTML5 sobre otros motores más pesados y complejos. Esta solución permite el renderizado dinámico de gráficos 2D con aceleración por hardware y resulta especialmente adecuada para este proyecto, ya que proporciona un control preciso a nivel de píxel. Gracias a ello, es posible dibujar con exactitud elementos como los robots, las líneas tangentes, los conos de velocidad o las zonas de decisión, manteniendo además una fluidez constante en la simulación en torno a los 60 fotogramas por segundo.

Capítulo 3 – Desarrollo y Algoritmia

Este capítulo describe cómo se ha implementado el motor de simulación, mostrando cómo hemos convertido las ideas geométricas del marco teórico en algoritmos. A lo largo de las próximas secciones, se explica paso a paso la estructura lógica del sistema, la clasificación de los escenarios de conflicto y las estrategias de resolución aplicadas en cada uno. Asimismo, se profundiza en el sistema de evaluación matemática que optimiza las técnicas de evasión y, finalmente, se expone cómo estas rutas se convierten en secuencias de movimiento dinámico a través del motor de físicas.

3.1. Arquitectura del motor lógico

El motor lógico constituye el núcleo analítico de la aplicación y ha sido diseñado bajo un enfoque predictivo y determinista. Esto implica que, a diferencia de los sistemas de navegación reactiva que modifican su comportamiento en tiempo real ante obstáculos imprevistos, este algoritmo calcula la totalidad de la maniobra de evasión antes de que los agentes comiencen a moverse. Este diseño garantiza que las trayectorias resultantes sean matemáticamente exactas y globalmente óptimas.

El ciclo de ejecución del algoritmo se ejecuta en el instante en que el usuario acciona el botón de inicio. En ese momento, el sistema detiene cualquier actualización interactiva y realiza una captura de los datos, registrando las coordenadas de origen y destino de ambos robots, así como sus respectivos radios de seguridad. A partir de este instante, el flujo de procesamiento se divide en tres grandes etapas secuenciales.

En la primera etapa se realiza una validación básica de seguridad: se verifica que las posiciones iniciales y finales de los robots no se superpongan. Si el código detecta que los robots ya están chocando en su punto de salida o que van a colisionar irremediablemente en su destino, el sistema lanza un mensaje de error emergente (popUp) advirtiendo al usuario y aborta la ejecución de la simulación.

Una vez identificado el problema, se inicia la segunda etapa: analizar las proyecciones de las rutas para clasificar el tipo de conflicto geométrico y el cálculo matemático. Para esto último, el algoritmo traza las tangentes, calcula las intersecciones y construye la

trayectoria de evasión más adecuada. Dado que los cálculos varían en función del tipo de conflicto, esta lógica se desarrollará con mayor detalle en los siguientes apartados.

Por último, en la tercera etapa, las coordenadas obtenidas se organizan en una secuencia estructurada de movimientos. Este "plan de vuelo" se transfiere al motor gráfico basado en Canvas, que se encarga de animar a los agentes y hacer que sigan la trayectoria calculada.

3.2. Clasificación de escenarios

Para la identificación y resolución de los conflictos de trayectoria en el motor de simulación, este trabajo toma como base el marco teórico sobre el movimiento coordinado óptimo establecido por Kirkpatrick & Liu (2016). Las definiciones geométricas y la categorización formal de los escenarios que se detallan a continuación (*Easy, Nested, Multi-Obstruction*) han sido adaptadas a partir de dicho estudio para su implementación práctica en el algoritmo del simulador.

Con este fundamento teórico, para que el software sea capaz de tomar decisiones automatizadas, el sistema debe evaluar las colisiones mediante la proyección de las trayectorias originales de los agentes (es decir, la ruta teórica en línea recta desde su inicio hasta su meta, previa a cualquier cálculo de evasión).

Como se definió en la sección 2.3, el sistema utiliza el concepto de *corridor* para determinar la viabilidad de los movimientos. El clasificador central, que hemos denominado *isRobotinCorridor*, opera de forma analítica comprobando si las coordenadas de origen o destino de un robot invaden el *corridor* del otro robot. A partir de estas comprobaciones, se realizan cuatro verificaciones booleanas fundamentales:

1. ¿Está el inicio de A en el *corridor* de B? (A0_in_B)

2. ¿Está el destino de A en el *corridor* de B? (A1_in_B)

3. ¿Está el inicio de B en el *corridor* de A? (B0_in_A)

4. ¿Está el destino de B en el *corridor* de A? (B1_in_A)

Al cruzar estos cuatro valores booleanos (verdadero o falso), el motor lógico obtiene una tabla de verdad, estructurada como se muestra en la Tabla 3.1, que permite clasificar e identificar el tipo de escenario ante el que nos encontramos.

	$A_0 \notin Corr_{\mathbb{B}}$ $A_1 \notin Corr_{\mathbb{B}}$	$A_0 \notin Corr_{\mathbb{B}}$ $A_1 \in Corr_{\mathbb{B}}$	$A_0 \in Corr_{\mathbb{B}}$ $A_1 \notin Corr_{\mathbb{B}}$	$A_0 \in Corr_{\mathbb{B}}$ $A_1 \in Corr_{\mathbb{B}}$
$B_0 \notin Corr_{\mathbb{A}}$ $B_1 \notin Corr_{\mathbb{A}}$	Easy	Easy	Easy	Nested
$B_0 \notin Corr_{\mathbb{A}}$ $B_1 \in Corr_{\mathbb{A}}$	Easy	Multi	Easy	Multi
$B_0 \in Corr_{\mathbb{A}}$ $B_1 \notin Corr_{\mathbb{A}}$	Easy	Easy	Multi	Multi
$B_0 \in Corr_{\mathbb{A}}$ $B_1 \in Corr_{\mathbb{A}}$	Nested	Multi	Multi	Multi

Tabla 3.1: Tabla de verdad para la clasificación analítica de escenarios basada en la invasión de *corridors*.

En lugar de evaluar las dieciséis combinaciones posibles de esta tabla de forma individual, el algoritmo agrupa los resultados basándose en tres reglas geométricas principales. La primera de ellas identifica el caso Anidado (*Nested*). Este escenario se activa matemáticamente cuando los dos puntos de un robot, tanto su posición de inicio como la de destino, arrojan un valor verdadero al comprobar si están dentro del *corridor* del otro agente. Físicamente, esto significa que uno de los robots va a realizar prácticamente todo su recorrido encapsulado en la zona de peligro del otro, lo que le obliga a asumir el rol de evasor y maniobrar a su alrededor. En ciertos casos el conflicto no puede resolverse con un simple rodeo y requiere el cálculo analítico de un punto de espera.

La segunda regla determina el caso Multi-Obstruction. El clasificador lo detecta cuando la tabla de verdad muestra una invasión mutua; es decir, cuando al menos un punto del robot A invade el *corridor* de B y, simultáneamente, al menos un punto del robot B invade el *corridor* de A. Esta condición de simetría refleja cruces de trayectorias severos, donde los caminos de ambos agentes se intersecan de forma transversal. Al igual que ocurre en el caso Anidado, la complejidad de estos cruces hace necesario también el cálculo de un punto de espera.

Por último, el sistema define el caso Fácil (*Easy*) por descarte de los escenarios complejos. Si la tabla de verdad revela que solo una posición de inicio o de destino invade al otro agente, sin que exista un cruce mutuo ni un encapsulamiento total; o que no se produce la invasión de ningún *corridor*, el clasificador concluye que la geometría de los caminos originales es viable. En esta situación, comprende que el conflicto puede resolverse sin alterar las rutas, gestionando únicamente el tiempo y la prioridad de paso.

3.3. Algoritmia del caso Fácil (Easy)

Este escenario se da cuando la geometría de las trayectorias no obliga a los robots a modificar su ruta original. En estos casos, el algoritmo determina que la mejor solución es gestionar el cruce en el tiempo, estableciendo un orden claro de prioridad de paso entre los robots.

Para determinar el orden de movimiento, el sistema evalúa qué agente actúa como obstáculo en las fases críticas del desplazamiento. La regla principal establece que, si la posición inicial de un robot invade el *corridor* del otro, este debe moverse primero para despejar el camino lo antes posible. Por el contrario, si es la posición final de un robot la que estorba en el pasillo de su compañero, la prioridad recaerá sobre el segundo, permitiéndole atravesar el área libremente antes de que el primero llegue a su destino y bloquee la intersección. Aplicando esta lógica a las variables extraídas de la tabla de verdad, si el destino del robot A está en el *corridor* de B, o si el inicio del robot B está en el *corridor* de A, el sistema decreta que el robot B tiene la prioridad de marcha. En cualquier otro caso, la prioridad recae por defecto sobre el robot A.

Una vez establecida esta jerarquía, el movimiento se ejecuta en dos turnos estrictamente secuenciales. El robot con prioridad avanza en línea recta hacia su meta mientras el segundo permanece detenido en su posición de salida. Cuando el primer agente completa su recorrido y libera el cruce, el motor autoriza el avance del segundo, generando una secuencia de movimiento como la que se ilustra en la Figura 3.2.

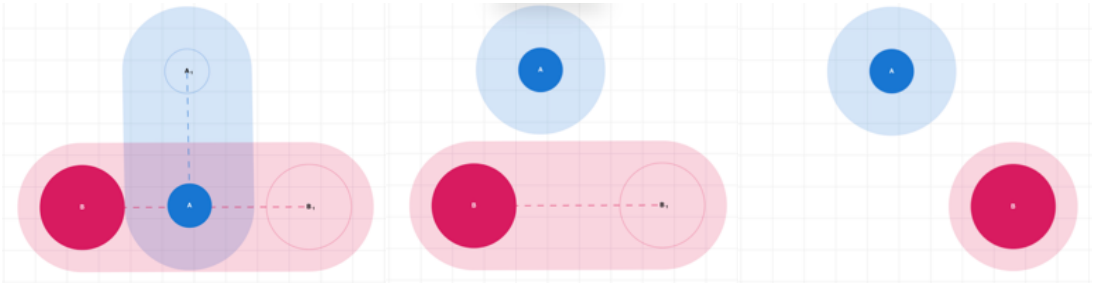


Figura 3.2: Secuencia de resolución temporal en un escenario Easy.

Esta estrategia de "esperar y pasar" resulta muy eficiente para conflictos leves, ya que evita rodeos innecesarios, reduce drásticamente la carga computacional y garantiza en todo momento la distancia de seguridad. Resulta evidente que esta solución es óptima, dado que ambos robots se mueven en línea recta.

Es importante destacar que esta categoría también engloba los escenarios de trayectoria limpia, donde ningún robot invade el pasillo del otro. En estos casos, al no existir riesgo físico de colisión, el orden de salida resulta indiferente. Por ello, tanto en ausencia de invasiones como en cualquier otra situación que no cumpla estrictamente las condiciones de bloqueo mencionadas anteriormente, el sistema otorga el primer turno de paso por defecto al robot A.

3.4. Algoritmia del caso Anidado (Nested)

El escenario Anidado representa un grado de complejidad geométrica intermedio y se activa cuando el clasificador detecta que tanto la posición inicial como la final de un robot se encuentran dentro del *corridor* del otro. Físicamente, esta condición implica que un agente está atrapado en la trayectoria directa del otro robot, por lo que una simple detención temporal resultaría ineficaz. Para resolver este bloqueo de forma segura y coordinada, el sistema recurre al cálculo de un punto de intersección, denominado P_{int}.

El primer paso en la resolución matemática de este caso consiste en asignar los roles de agente anidado y no anidado. Una vez definidos, el algoritmo procede a la división espacial del entorno mediante el cálculo de dos líneas tangentes principales. Estas rectas se trazan siempre desde la posición inicial del robot anidado hacia el perímetro de seguridad ampliado del robot no anidado: una apuntando a su posición inicial y otra a su posición final. La elección de qué lado del obstáculo se utiliza para trazar esas tangentes

(es decir, utilizar la tangente de la "derecha" o la "izquierda", tomando siempre como referencia relativa la línea que une la posición inicial y final del robot no anidado) depende directamente de si el sistema está calculando el rodeo en sentido horario (Clockwise) o antihorario (Counter-Clockwise). Para garantizar que el robot realiza el recorrido más corto, el algoritmo prueba y evalúa en paralelo las distintas trayectorias posibles. Las reglas exactas que usa el sistema para comparar estas opciones y elegir la mejor se explicarán en detalle al final de este capítulo, ya que son una lógica común que el simulador aplica para resolver cualquier escenario.

Las dos rectas tangentes mencionadas actúan como fronteras que dividen el *corridor* en cuatro regiones o zonas lógicas bien diferenciadas (véase la Figura 3.3).

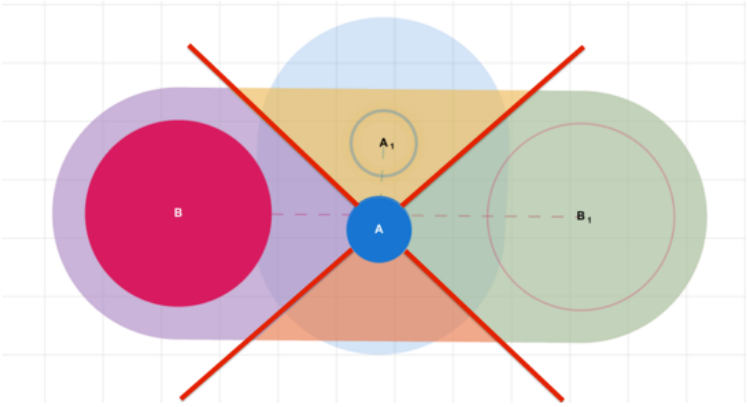


Figura 3.3: División del *corridor* en cuatro zonas lógicas a partir de tangentes.

Dependiendo de la zona específica en la que se ubique el punto de destino del agente anidado, el sistema calcula la posición del punto P_{int}. La maniobra sigue un patrón base de tres turnos: primero el robot anidado avanza desde su posición inicial hasta P_{int}; a continuación, el robot no anidado realiza su trayecto hacia su destino esquivando al agente detenido; por último, el robot anidado concluye su ruta desde P_{int} hasta su meta. Lo que varía fundamentalmente según las zonas es el cálculo matemático de este punto intermedio:

- **Zona 1:** La delimitación exacta de esta región depende directamente del sentido de evasión que esté evaluando el algoritmo en ese momento. En el sentido antihorario (*Counter-Clockwise*), el área queda delimitada a la derecha de la tangente principal dirigida a la posición inicial del robot no anidado y a la

izquierda de la tangente dirigida a su posición final. En cambio, en el sentido horario (Clockwise), al invertirse por completo las referencias laterales del cono, la zona pasa a estar delimitada a la izquierda de la tangente dirigida a la posición inicial y a la derecha de la tangente dirigida a la posición final. Si el destino del robot anidado se encuentra en esta región, el sistema establece que P_{int} coincide directamente con dicho destino. Por lo tanto, la secuencia se simplifica: el robot anidado se desplaza a su posición final sin interrupciones, y seguidamente el robot no anidado avanza hacia su propia meta, realizando la maniobra de evasión, un comportamiento que queda esquematizado en la Figura 3.4.

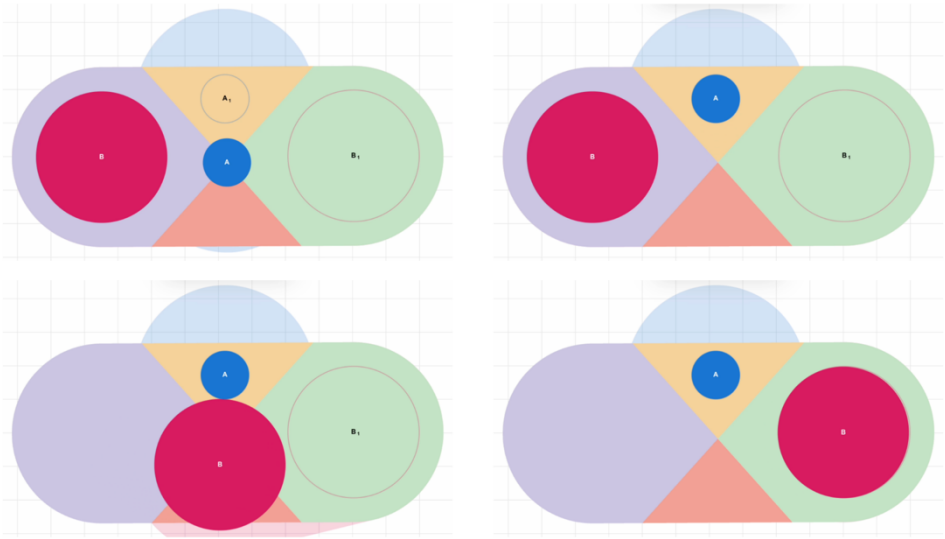


Figura 3.4: Resolución de un escenario Nested con destino en la Zona 1.

- **Zona 2:** Corresponde a la región geométricamente opuesta a la Zona 1. Su delimitación aplica las condiciones laterales contrarias a las descritas en la zona anterior, dependiendo igualmente de si el destino de evasión es horario o antihorario. Si el destino del robot anidado recae aquí, P_{int} asume las coordenadas de la propia posición inicial del robot anidado. En consecuencia, el orden de los turnos se altera: el robot no anidado avanza primero hacia su destino realizando la maniobra de evasión, y solo después el robot anidado se desplaza en línea recta hasta su meta final. La Figura 3.5 recrea visualmente el resultado de esta secuencia.

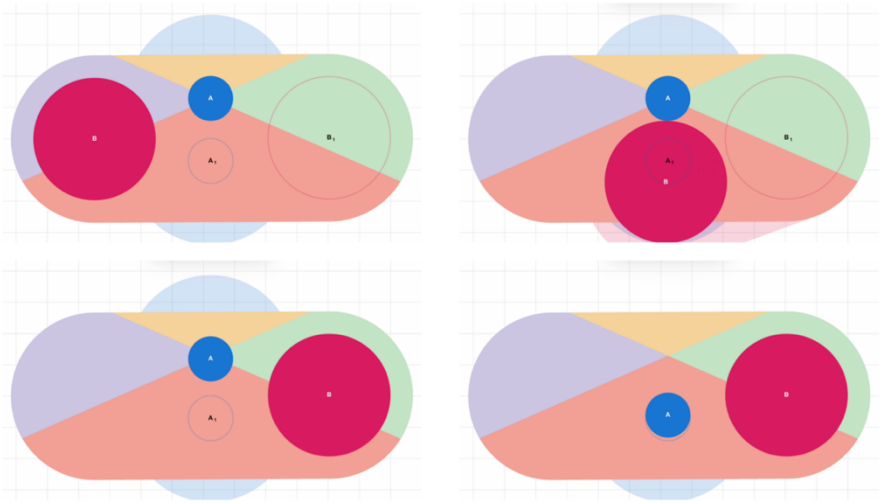


Figura 3.5: Resolución de un escenario Nested con destino en la Zona 2.

- **Zona 3:** Esta región se sitúa en el flanco exterior respecto a ambas tangentes principales (a la derecha en evaluación *Counter-Clockwise* y a la izquierda en *Clockwise*). Esta región se sitúa a la derecha de ambas tangentes principales. Cuando el destino del robot anidado cae en esta zona, el sistema calcula P_{int} hallando la intersección entre la primera tangente principal (la que va hacia la posición final del no anidado) y una nueva recta tangente trazada desde la posición final del robot anidado hacia la inicial del no anidado. Respecto al movimiento, se ejecuta el patrón dinámico de tres fases: primero, el robot anidado avanza hasta el punto de espera P_{int} y se detiene; a continuación, el agente principal completa su recorrido evadiendo al obstáculo; y finalmente, el robot anidado reanuda su marcha desde P_{int} hasta su meta definitiva (véase la Figura 3.6).

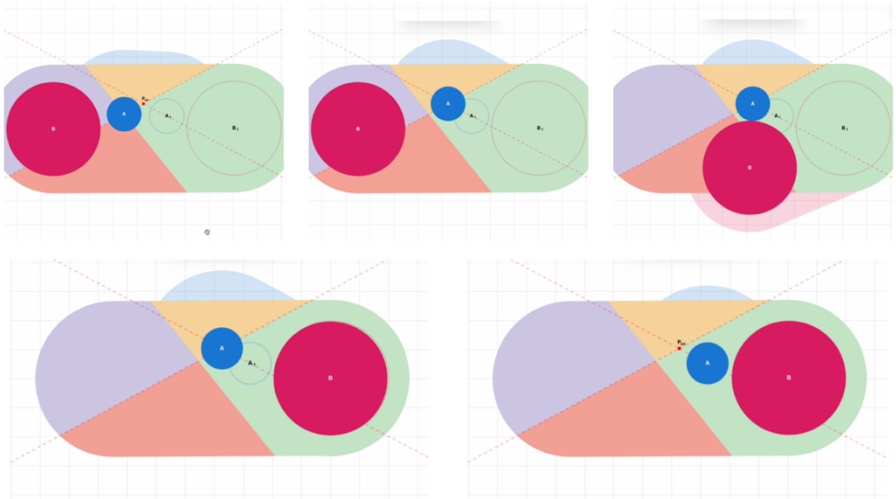


Figura 3.6: Resolución de un escenario Nested con destino en la Zona 3.

- **Zona 4:** Se localiza en el flanco exterior opuesto a la zona 3. En este caso, P_{int} se obtiene calculando la intersección entre la segunda tangente principal (la dirigida hacia la posición inicial del no anidado) y una nueva recta trazada desde la posición final del robot anidado hacia la final del no anidado. El desplazamiento se resuelve mediante los mismos tres turnos secuenciales de la zona anterior. Un ejemplo del trazado final de esta resolución puede examinarse en la Figura 3.7.

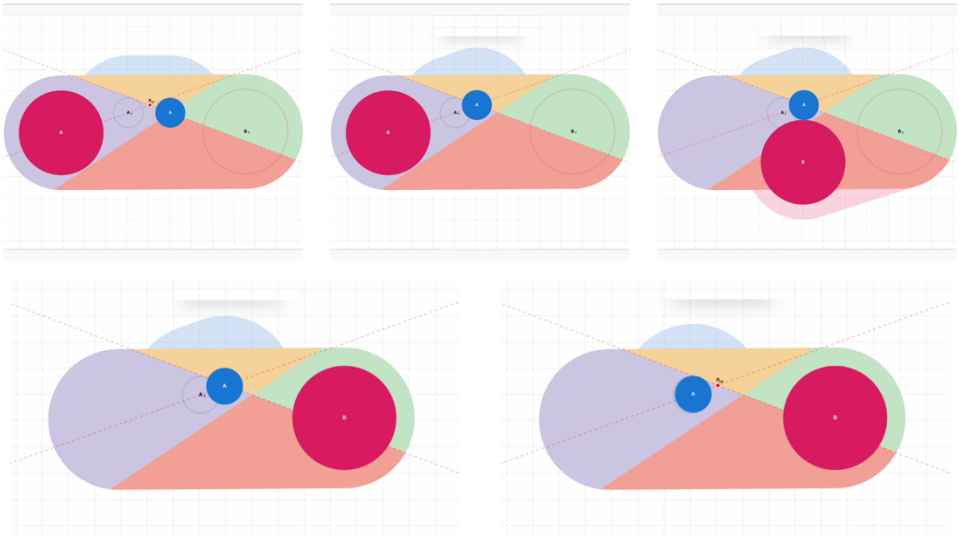


Figura 3.7: Resolución de un escenario Nested con destino en la Zona 4.

3.5. Algoritmia del caso Multi Obstruyente (Multi-Obstruction)

El escenario múltiple representa el mayor nivel de dificultad del simulador y se activa cuando el clasificador detecta una invasión mutua; es decir, cuando al menos una posición del robot A se encuentra dentro del *corridor* del robot B y, simultáneamente, una posición del robot B invade el *corridor* del robot A. Físicamente, esta condición describe un cruce de trayectorias severo, donde ninguno de los agentes está completamente anidado dentro del otro, pero sus caminos colisionan de forma inminente. Al igual que en el caso anterior, una detención temporal resultaría ineficaz o provocaría un bloqueo mutuo, por lo que el sistema requiere calcular un punto de desvío y espera (P_{int}), alejado del epicentro del cruce.

El primer paso en la resolución matemática de este conflicto consiste en asignar los roles operativos y evaluar el sentido de evasión óptimo. Decidir qué robot debe esquivar y por qué lado hacerlo es especialmente complejo en este escenario. Para resolverlo, el

simulador hace que todas las opciones válidas compitan entre sí para quedarse con la ruta más rápida. El funcionamiento paso a paso de esta decisión final se detalla en las secciones 3.6 y 3.7 de este capítulo.

El cálculo de las zonas en este escenario difiere significativamente del caso Anidado. En esta ocasión, no se trazan las rectas tangentes desde la posición inicial del robot anidado a las posiciones inicial y final del no anidado; sino que se toman ambas rectas tangentes, (la "derecha" y la "izquierda", tomando siempre como referencia relativa la línea que une la posición inicial y final del robot no anidado) que hay entre las posiciones iniciales de ambos robots. Estas dos rectas, que en adelante denominaremos L1 (tangente derecha, que se corresponde exactamente con la tangente principal calculada previamente para la división de zonas en el caso Anidado) y L5 (tangente izquierda), tomando siempre como referencia relativa la línea que une la posición inicial y final del robot no anidado, proyectan un área en forma de cono que se expande más allá de su *corridor* original, tal y como se ilustra en la Figura 3.8. Es este cono de exclusión el que se somete a la subdivisión en cuatro zonas.

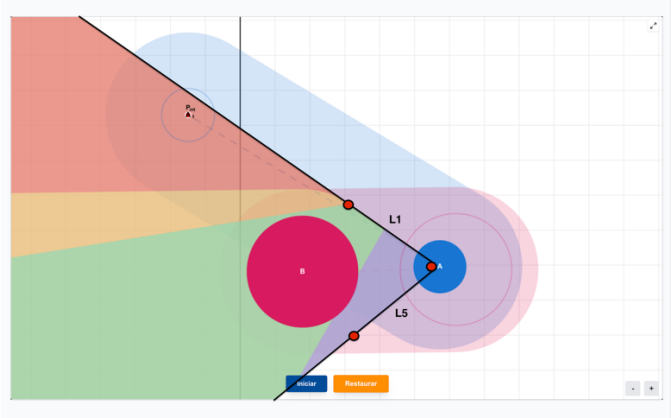


Figura 3.8: Representación de las rectas que delimitan el cono de exclusión (L1 y L5).

Para estructurar estas 4 regiones, el algoritmo calcula 3 líneas divisorias y un arco de circunferencia. La delimitación precisa de cada zona se establece de la siguiente manera:

- **Zona 1**: Se sitúa en el flanco derecho del cono. Queda delimitada por su parte más externa mediante la recta L1 definida anteriormente. Su límite interno (izquierdo) está compuesto por dos elementos: una recta que une los puntos más “a la derecha” de las posiciones inicial y final del robot no anidado (L2), y el perfil

derecho de su posición inicial extendida (su radio de seguridad ampliado) (C1). La formación de esta primera región puede observarse en detalle en la Figura 3.9.

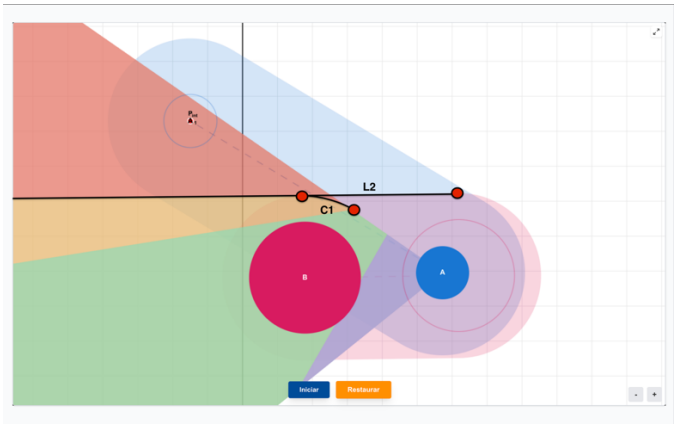


Figura 3.9: Representación de las rectas que forman el límite interno de la Zona 1 (L2 y C1).

- **Zona 2:** Se encuentra inmediatamente a la "izquierda" de la zona 1. Su límite derecho son exactamente la recta y la curva que acabamos de mencionar. Su límite izquierdo lo marca una nueva recta (L3), trazada desde el punto más "a la derecha" de la posición final del robot no anidado hasta el punto de tangencia de la tangente principal "derecha" del cono. Esta delimitación queda representada en la Figura 3.10.

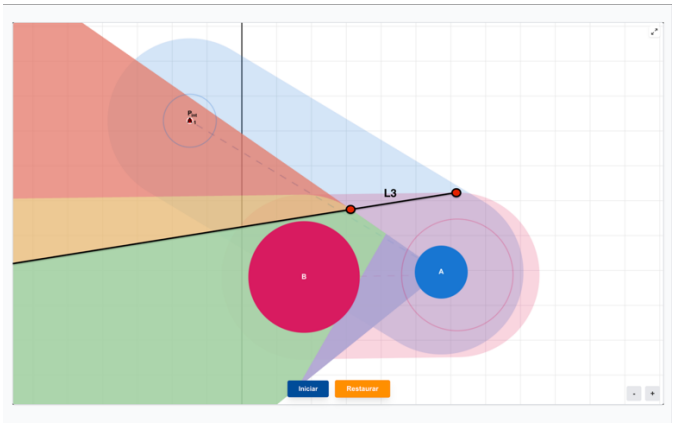


Figura 3.10: Representación de la recta que forma el límite interno de la Zona 2 (L3).

- **Zonas 3 y 4:** Para definir la frontera que separa estas dos regiones correspondientes al flanco izquierdo del cono, el algoritmo requiere calcular previamente un punto de referencia dinámico denominado T. Las coordenadas de este punto dependen de la disposición espacial de los agentes y se evalúan mediante tres condiciones:

- El sistema comprueba primero si los círculos de seguridad extendidos de las posiciones inicial y final del robot no anidado se intersecan. Si es así, dicha intersección define directamente el punto T (véase la Figura 3.11).

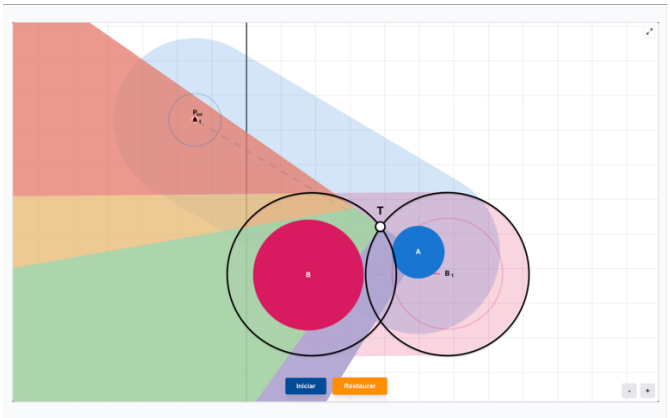


Figura 3.11: Representación del punto T cuando los radios de seguridad se intersecan.

- Si la condición anterior no se cumple, se evalúa la posición inicial del robot anidado. Si esta se encuentra fuera del área de destino ampliada del robot no anidado, T asume las coordenadas de dicha posición inicial (como se aprecia en la Figura 3.12).

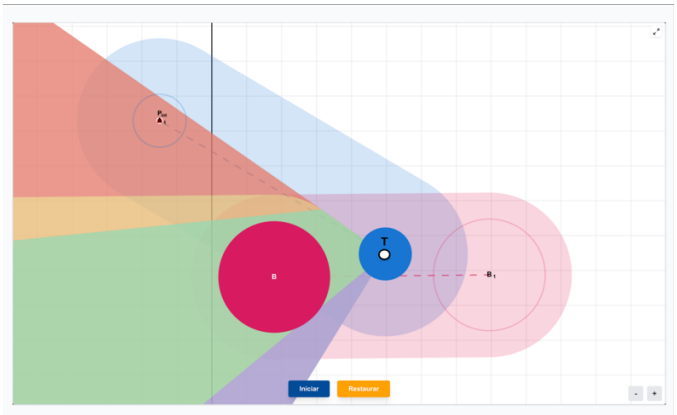


Figura 3.12: Representación del punto T igual a la posición inicial del robot anidado.

- Si, por el contrario, la posición inicial del robot anidado invade el área de destino del no anidado, T se calcula como el punto de intersección analítico entre la tangente maestra "derecha" del cono y el círculo extendido de la posición final del robot no anidado (apreciado en la Figura 3.13).

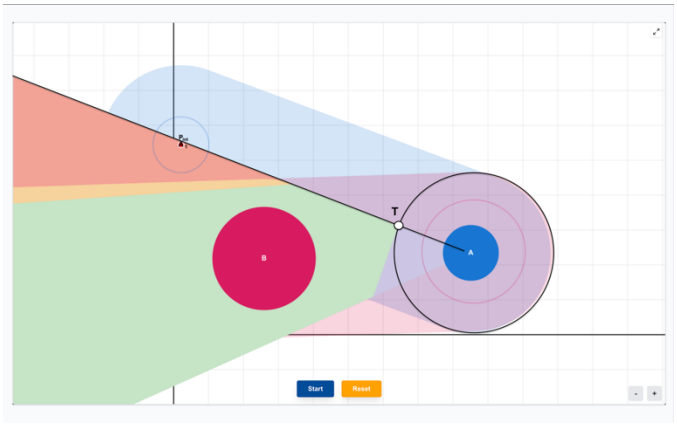


Figura 3.13: Representación del punto T cuando la posición final del robot no anidado contiene la posición inicial del robot anidado.

Una vez que el motor lógico ha determinado la posición exacta del punto T mediante una de estas tres vías, la línea divisoria definitiva entre las Zonas 3 y 4 se construye trazando una recta tangente desde dicho punto T hasta el extremo derecho del perímetro ampliado de la posición final del robot no anidado. Establecida esta línea divisoria, el espacio restante del cono se asigna lógicamente separando la Zona 3 de la 4, partición geométrica que se muestra en la Figura 3.14.

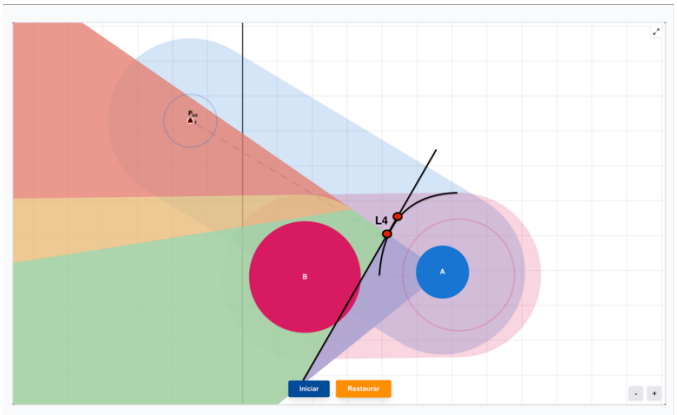


Figura 3.14: Representación de la recta que separa las Zonas 3 y 4 (L4).

Resulta fundamental destacar una particularidad geométrica de este modelo: dependiendo de la disposición de los robots en el espacio, la línea que separa ambas zonas puede proyectarse fuera de los límites del cono principal. En este caso, la Zona 4 no existe lógicamente en la evaluación, por lo que todo el espacio restante del flanco izquierdo recae íntegramente sobre la Zona 3, como ocurre en la Figura 3.15.

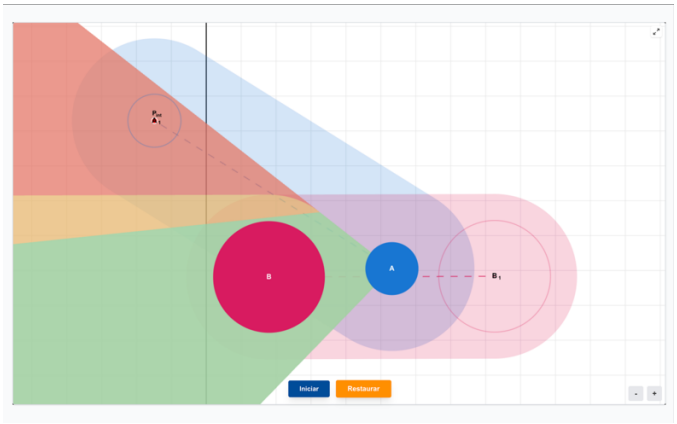


Figura 3.15: Caso en el que la línea L4 se encuentra fuera del cono.

Una vez definidas las zonas que estructuran el cono, la lógica de simulación determina el comportamiento de los agentes en función de la zona específica en la que se ubique la posición inicial del robot anidado.

La mecánica general de movimiento sigue la misma secuencia de tres turnos que el escenario Anidado: primero, el robot anidado avanza desde su posición inicial hasta P_{int}; a continuación, el robot no anidado realiza su trayecto hacia su destino; y, por último, el robot anidado concluye su ruta desde P_{int} hasta su meta. Sin embargo, dada la naturaleza de invasión mutua característica del caso Multi Obstruyente, durante esta secuencia, cualquiera de los dos robots puede verse obligado a ejecutar las maniobras de evasión para esquivar al otro robot. Por lo tanto, el comportamiento dinámico del sistema se define como el diferente cálculo del punto P_{int} en cada situación:

- **Zona 1:** Si el destino del robot anidado se ubica en esta región, el sistema establece que P_{int} equivale directamente a la propia posición inicial del robot anidado. En consecuencia, la secuencia de turnos se simplifica: el robot no anidado avanza primero hacia su destino de forma directa y sin interrupciones. Una vez despejada el área principal, el robot anidado emprende su marcha hacia su meta final, realizando la maniobra de evasión pertinente alrededor del otro robot. Esta dinámica se encuentra representada en la Figura 3.16.

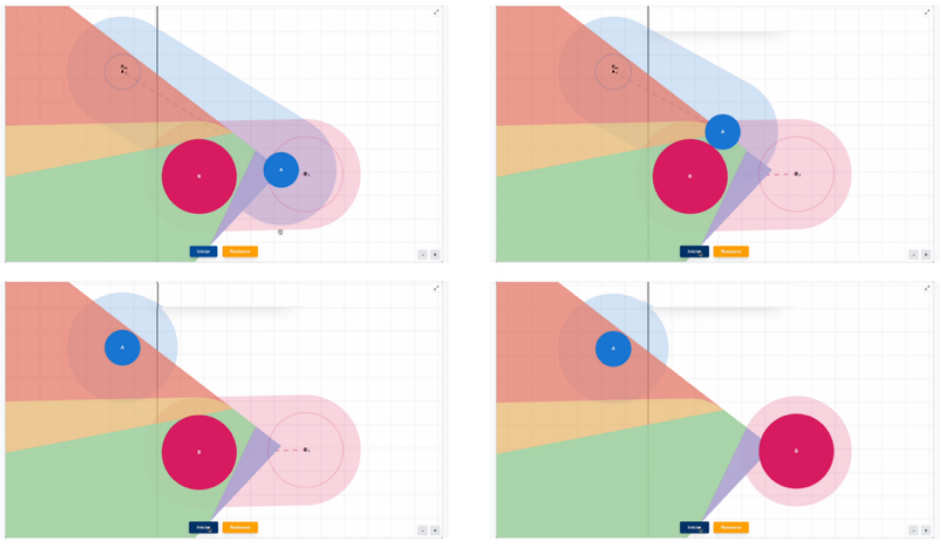


Figura 3.16: Resolución de un escenario Multi-Obstruction con destino en la Zona 1.

- **Zona 2:** Cuando el destino recae en esta área, P_{int} se define como la intersección más a la derecha resultante entre dos geometrías: la recta tangente trazada entre las posiciones finales de ambos robots, y el círculo de seguridad ampliado correspondiente a la posición inicial del robot no anidado (véase Figura 3.17).

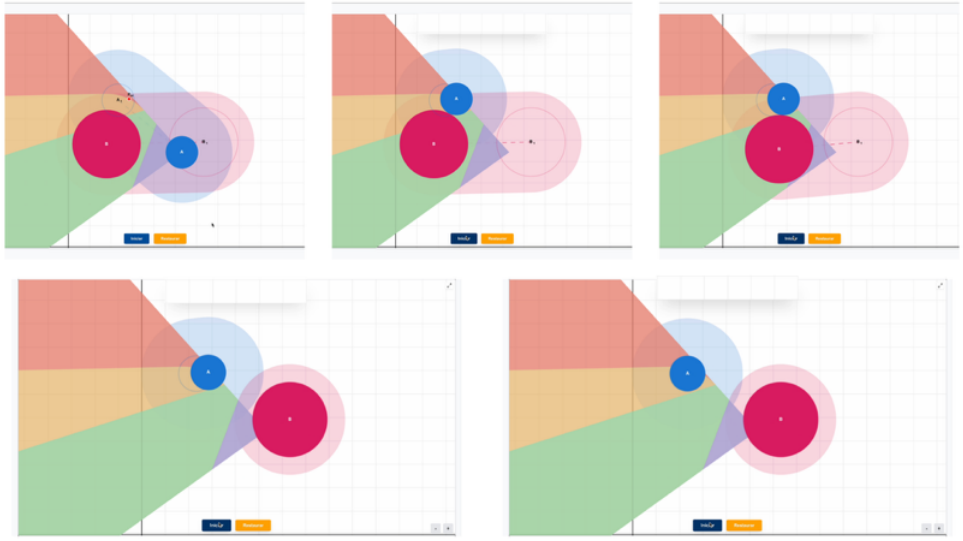


Figura 3.17: Resolución de un escenario Multi-Obstruction con destino en la Zona 2.

- **Zona 3:** Si la posición final del robot anidado cae dentro de esta zona, el sistema calcula P_{int} hallando la intersección de las rectas tangentes proyectadas entre las posiciones iniciales y finales de ambos agentes. Dicha disposición espacial se puede apreciar en la Figura 3.18 junto con una secuencia de ejemplo de este caso.

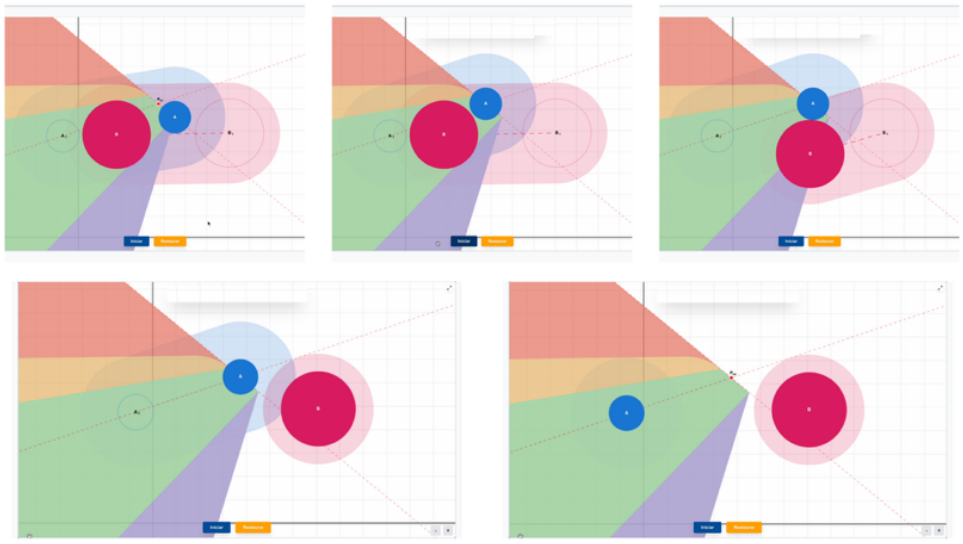


Figura 3.18: Resolución de un escenario Multi-Obstruction con destino en la Zona 3.

- **Zona 4:** Finalmente, en el caso de que el destino se encuentre en esta última región, el punto de espera P_{int} asume exactamente las coordenadas del punto de referencia T calculado anteriormente en este apartado. Esto se visualiza en la Figura 3.19.

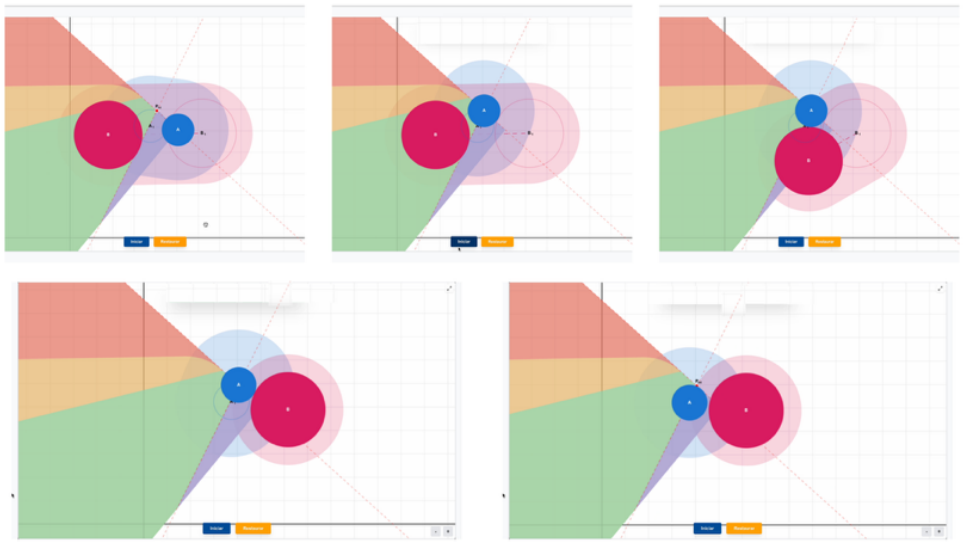


Figura 3.19: Resolución de un escenario Multi-Obstruction con destino en la Zona 4.

Toda la lógica geométrica explicada hasta este punto en esta sección asume por defecto que el cono de exclusión y las tangentes divisorias se proyectan partiendo de la posición inicial del robot anidado. Sin embargo, la amplia variedad de cruces posibles en el escenario Multi Obstruyente implica que esta perspectiva no es siempre viable para resolver el conflicto. Existen situaciones particulares donde intentar esquivar proyectando desde el origen resulta ineficaz.

Para enfrentarnos a esta limitación sin necesidad de desarrollar un segundo motor geométrico independiente, el sistema implementa una lógica de evaluación inversa o "modo espejo". Esta estrategia de aprovechar la simetría para simplificar el problema es exactamente la misma que se utiliza en los artículos de referencia de este proyecto; en concreto, es el enfoque que aplican tanto Kirkpatrick & Liu (2016, 2025) como Esteban et al. (2025) en sus respectivos estudios. Apoyándonos en sus trabajos, el algoritmo decide aplicar esta inversión basándose en dos comprobaciones estrictas:

- **Evaluación lógica:** Al analizar la tabla de verdad inicial, el sistema revisa al detalle qué posiciones exactas están invadiendo el espacio del otro robot. Si detecta que el choque se va a producir cerca de sus posiciones de destino (por ejemplo, porque ambos viajan hacia metas muy cercanas entre sí), el algoritmo marca el caso preventivamente como invertido, configuración que se puede observar en la Figura 3.20.

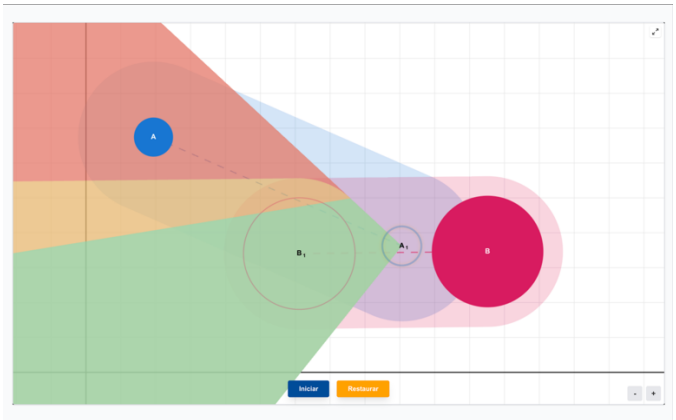


Figura 3.20: Escenario Multi-Obstruction con lógica inversa.

- **Evaluación geométrica:** Aunque la configuración inicial parezca estándar, el sistema lanza una comprobación adicional ("*checkFueraDelCono*"). Si la posición final del robot desplazado se proyecta fuera de los límites del cono, el sistema no intenta forzar un cálculo de trayectoria interior (que no garantiza la optimidad requerida); sino que, bajo criterios de diseño, se opta por aplicar la inversión temporal de roles. Esta estrategia permite asegurar la optimidad de la maniobra de evasión, evitando así cualquier compromiso en la eficiencia de la trayectoria resultante.

Cuando se cumple cualquiera de estas dos condiciones, el algoritmo realiza un intercambio de variables. A nivel lógico, la posición inicial del robot asume el rol de

destino y su posición inicial asume el rol de origen (aplicando este cambio también al robot principal).

Gracias a este cambio, el sistema puede realizar toda la lógica explicada anteriormente (las cuatro zonas, el punto T, el cálculo de P_{int}) adaptándose a que el punto desde el que se proyecta el cono y el robot hacia el que apunta sean ahora las posiciones finales y no las iniciales. Una vez que el algoritmo halla la ruta óptima para este escenario invertido, el simulador simplemente revierte el orden de la trayectoria resultante. De este modo, aunque internamente el código haya calculado el camino como si el robot estuviese yendo del destino al origen, visualmente en la simulación el usuario observará que el robot se mueve con total normalidad, avanzando de forma segura desde su posición inicial hasta su meta, tal y como se aprecia en la Figura 3.21, donde se muestra la ejecución final de esta trayectoria tras la inversión del cálculo.

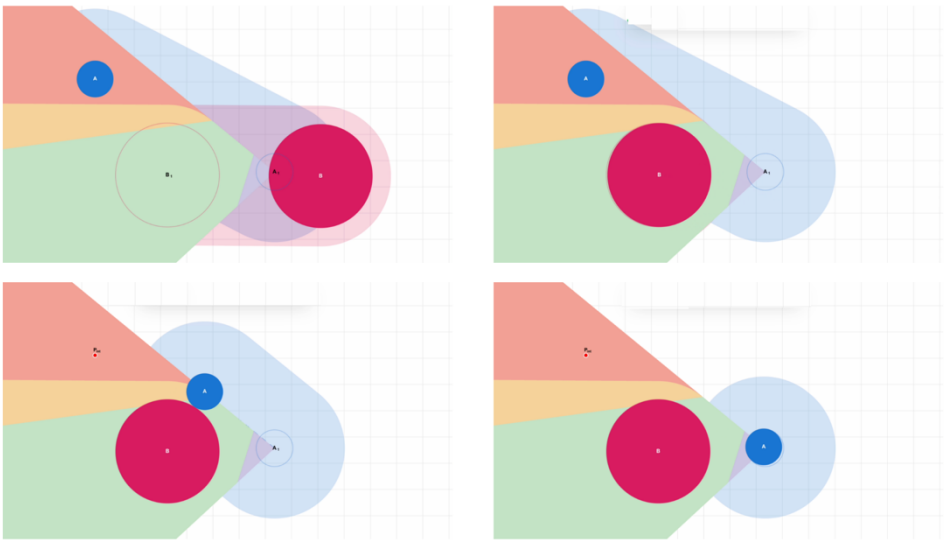


Figura 3.21: Resolución de un escenario Multi-Obstruction con lógica inversa.

3.6. Asignación de roles

Antes de calcular rectas tangentes, zonas espaciales o puntos de espera, el sistema debe establecer una jerarquía clara entre los agentes. Esta fase, que actúa como un árbitro interno, es fundamental para determinar qué robot asume el rol de agente principal (no anidado, con prioridad para mantener su trayectoria base lo máximo posible) y qué robot asume el rol de agente anidado (responsable de ceder el paso y desviarse hacia un punto de espera temporal).

El criterio para asignar estos roles varía en función del escenario detectado previamente por el clasificador:

- **En el escenario Fácil (Easy):** Al no haber bloqueos complejos no hay necesidad de que ninguno de los robots se desplace a ningún punto intermedio, por lo que no existen los roles de agente anidado y no anidado. La jerarquía se establece como ya explicamos en su apartado correspondiente, según quién molesta a quién.

- **En el escenario Anidado (Nested):** El algoritmo verifica si la posición de un robot se encuentra contenida dentro del *corridor* del otro. El agente que cumpla esta condición es designado inmediatamente como el robot anidado, viéndose obligado a maniobrar primero hacia el punto intermedio. Su contraparte asume por defecto el rol de agente principal.

- **En el escenario Multi Obstruyente (Multi-Obstruction):** Al tratarse de una invasión mutua, el sistema emplea una estrategia de competición de candidatos. Se evalúa el escenario por duplicado: primero asumiendo que el robot A es el anidado, y después asumiendo que lo es el robot B. La decisión definitiva de los roles no se toma de forma aislada en este caso, sino que se combina junto con el optimizador de rutas, el cual explicaremos a continuación.

3.7. Evaluación dual y optimización de rutas

Una vez predefinida la jerarquía de los agentes, el motor del simulador se enfrenta a la última gran decisión antes de iniciar el movimiento físico: ¿por qué lado del obstáculo resulta más eficiente realizar el rodeo?

Para resolver esta incógnita, el sistema implementa un motor de evaluación dual. Esto significa que el algoritmo genera dos proyecciones matemáticas completas del escenario: una forzando el trazado en sentido horario (Clockwise) y otra en sentido antihorario (Counter-Clockwise).

Esta dualidad no solo cambia la dirección del movimiento final, sino que condiciona por completo la geometría del problema. En el simulador, la maniobra para que un robot esquive a otro consiste analíticamente en calcular los puntos tangentes al radio de

seguridad ampliado del obstáculo y trazar una ruta que navegue hasta dichos puntos y recorra su arco. El sentido de evaluación dictamina qué conjunto de tangentes se utiliza para todo el proceso:

- En la configuración Counter-Clockwise, el motor lógico establece la regla general de operar sobre el flanco derecho (tomando siempre como referencia relativa el vector de dirección del movimiento implicado). Es importante precisar que el sistema no reutiliza los mismos puntos espaciales, sino que aplica esta misma regla de selección lateral a todos los cálculos independientes del escenario: el criterio de la tangente "derecha" se utiliza tanto en las proyecciones iniciales que estructuran las cuatro zonas espaciales del mapa, como en los nuevos cálculos dinámicos requeridos para obtener los puntos tangenciales de la ruta de evasión del robot.

- Por el contrario, en la configuración Clockwise, el sistema descarta el flanco anterior y aplica exclusivamente el criterio del flanco izquierdo. Someter todos los cálculos geométricos a esta regla opuesta altera por completo la forma de las zonas, la ubicación de las rectas, las coordenadas del punto de intersección (P_{int}) y, en última instancia, el recorrido de evasión final.

El optimizador mide la distancia euclídea de todos los segmentos rectos y arcos trazados en ambos mundos, obteniendo el coste total de recorrido. Se decide entonces ejecutar la partición de zonas con ambas configuraciones e implementar la lógica de las dos para poder compararlas y quedarnos con la que ofrezca un recorrido menor.

La decisión final se aplica según el caso:

- **Optimización en el escenario Anidado (Nested):** Al tener los roles de agente principal y anidado ya fijados en el paso anterior, la competición de rutas es directa. El sistema compara la distancia total de la ruta derecha relativa a la trayectoria (*Counter-Clockwise*) frente a la izquierda (*Clockwise*). La opción de menor longitud es proclamada ganadora y enviada al controlador de animación.

- **Optimización en el escenario Multi Obstruyente (Multi-Obstruction):** Dada la invasión mutua, la complejidad requiere elevar la optimización a una

"competición de candidatos". El motor calcula la evaluación dual asumiendo primero que el robot A cede el paso, y repite el proceso asumiendo que el robot B cede el paso. Al finalizar, el optimizador compara las cuatro distancias totales obtenidas (A-CW, A-CCW, B-CW, B-CCW) y selecciona a un único ganador absoluto, dictaminando simultáneamente el lado del rodeo y qué robot actuará como anidado.

Gracias a esta arquitectura de evaluación paralela, el simulador asegura que la maniobra final ejecutada no solo evite la colisión respetando estrictamente los radios de seguridad, sino que minimice el tiempo de espera y la energía consumida en el desplazamiento.

Capítulo 4. Arquitectura e implementación computacional

Una vez ya se ha definido cómo funciona el algoritmo en el Capítulo 3, este capítulo detalla cómo se ha construido la arquitectura de software para hacerlo realidad. A lo largo de las siguientes secciones se describen el diseño adoptado, la arquitectura general del sistema, la organización de la interfaz del usuario y la gestión del estado, elementos que en conjunto hacen posible la interactividad en tiempo real.

4.1. Arquitectura general del sistema

Para garantizar que el simulador sea rápido, accesible y capaz de recalcular trayectorias al instante, se ha diseñado como una *Single Page Application*, un paradigma de desarrollo donde la interfaz se actualiza dinámicamente sin necesidad de recargar el navegador. Esto significa que toda la carga de trabajo matemático y gráfico se ejecuta directamente en el equipo del usuario, eliminando los tiempos de espera y la latencia que supondría procesar la información mediante peticiones constantes a un servidor externo.

El proyecto está construido sobre la base del framework Next.js y se organiza siguiendo un principio fundamental de la ingeniería de software: la separación de responsabilidades. Para evitar que el código sea difícil de mantener, el sistema se ha dividido en tres grandes capas o bloques funcionales que se comunican entre sí:

1. **Capa de interfaz y controles:** Es la parte visual con la que interactúa el usuario. Su responsabilidad es recoger los parámetros introducidos (como las coordenadas o los radios) y almacenarlos en la memoria central de la aplicación. Gracias al uso de React, detallado en el marco teórico, esta capa gestiona la interactividad de forma ágil, sin interferir en los cálculos matemáticos.

2. **Capa lógica y geometría:** Es el "cerebro" matemático del simulador. Al estar programado con TypeScript, asegura la fiabilidad de las estructuras de datos y opera de forma completamente aislada de la parte visual. Contiene módulos independientes con funciones puras encargadas de calcular distancias, hallar puntos de tangencia, clasificar los escenarios, y determinar la ruta óptima.

3. **Capa de renderizado:** Aprovechando la API nativa de Canvas, este módulo se aísla de la lógica de negocio y se limita a escuchar los resultados para plasmarlos gráficamente en la pantalla (trazando los robots, las zonas y las tangentes). Trabaja en un bucle continuo e independiente que limpia y repinta el escenario constantemente para que su movimiento sea fluido.

Gracias a esta arquitectura, el simulador no solo resuelve el problema geométrico planteado, sino que se convierte en una herramienta escalable a la que le resultaría sencillo añadir nuevas funcionalidades o incluso más robots en el futuro sin tener que reescribir todo el código base.

4.2. Capa de interfaz y controles

El diseño de la interfaz del simulador tiene como objetivo hacer accesible la complejidad matemática del algoritmo mediante un enfoque interactivo, evitando que el usuario tenga que modificar las coordenadas introduciendo texto de forma manual. En la arquitectura del sistema, esta primera capa actúa como el puente periférico: su responsabilidad exclusiva es capturar las interacciones del usuario, normalizar los eventos del navegador y actualizar la memoria central, sin realizar cálculos de evasión geométrica.

Físicamente, la pantalla principal se divide en dos grandes bloques operativos: el espacio visual interactivo y las herramientas de configuración. A continuación, se detallan las funciones, manejadores de eventos (*event handlers*) y componentes lógicos implementados en *TypeScript* que dan soporte a cada uno de estos bloques.

4.2.1. El lienzo interactivo

El lienzo o "Canvas" ocupa la mayor parte de la pantalla y actúa como el entorno físico virtual donde se desarrolla la simulación, cuyo aspecto general se muestra en la Figura 4.1.

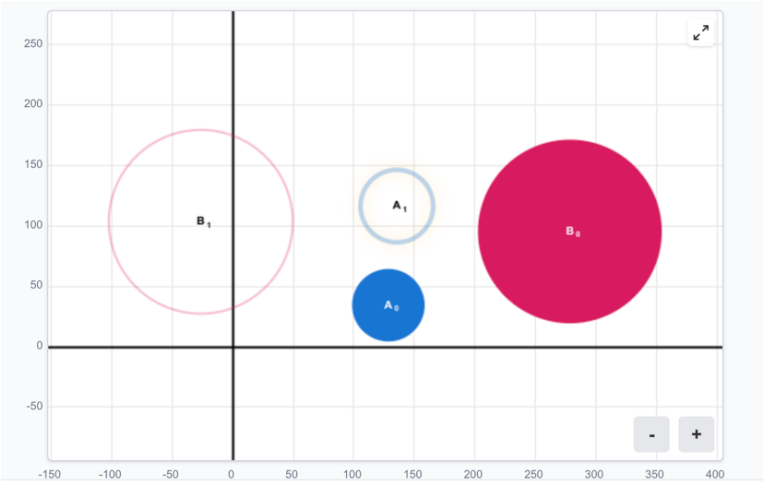


Figura 4.1: Entorno de simulación sobre el lienzo interactivo.

Su diseño está pensado para ofrecer una respuesta visual inmediata mediante manipulación directa (arrastrar y soltar) y navegación fluida por el plano cartesiano. Para lograr esta interactividad, se han implementado las siguientes lógicas y funciones de control:

- **Detección de colisión del cursor (*isRobotHovered*):** Para permitir la manipulación directa, el sistema debe saber en todo momento si el ratón está posicionado encima de un robot. Dado que el lienzo es un mapa de píxeles plano y no contiene elementos gráficos independientes que detecten clics por si solos, la función *isRobotHovered* calcula la distancia euclídea entre las coordenadas cartesianas del cursor y el centro de cada agente. Si esta distancia es menor o igual al radio del robot, la función devuelve su identificador (por ejemplo, 'A_0' o 'B_1'), permitiendo alterar el estilo del cursor, cambiando entre los estados de puntero normal, *grab* (indicando que se detecta un elemento sobre el que se puede hacer clic) o *grabbing* (que indica que se está agarrando un elemento). Todo esto se puede apreciar en la secuencia de imágenes de la Figura 4.2.

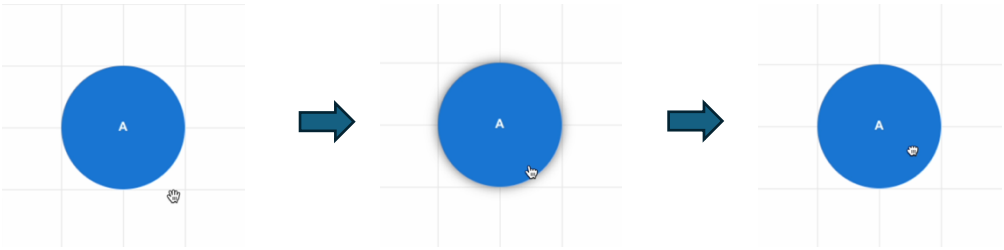


Figura 4.2: Secuencia de estados del cursor según la detección de colisión (*isRobotHovered*).

- **Manejadores de eventos de ratón (*handleMouseDown*, *handleMouseMove*, *handleMouseUp, handleMouseLeave*):** Estas funciones controlan el ciclo de vida de la interacción de escritorio. Cuando el usuario hace clic (*handleMouseDown*), el sistema evalúa si el impacto ha sido sobre un robot (activando el estado de arrastre *dragging*) o sobre el fondo libre del lienzo (activando la bandera de desplazamiento de cámara *isPanning*), tal y como se detalla en el fragmento de código de la Figura 4.3. Mientras el cursor se mueve (*handleMouseMove*), el manejador de alta frecuencia traduce continuamente las coordenadas de la pantalla al espacio lógico de simulación, actualizando en tiempo real la posición del robot o el desfase visual de la cámara. Finalmente, las funciones *handleMouseUp* y *handleMouseLeave* cierran el ciclo. Si el usuario suelta el botón de clic o el cursor abandona accidentalmente los límites del canvas, el sistema neutraliza el arrastre en seco, evitando que el estado quede bloqueado en una acción fantasma.

```
const handleMouseDown = (e: React.MouseEvent<HTMLCanvasElement>) => {
e.preventDefault();
const rect = canvasRef.current!.getBoundingClientRect();
const sx = e.clientX - rect.left;
const sy = e.clientY - rect.top;

const clickedRobotKey = isRobotHovered(sx, sy);
if (clickedRobotKey) {
const r = robotMap[clickedRobotKey];
const worldPos = screenToWorld(sx, sy, canvasSize.width, canvasSize.height);
if (r.x !== null && r.y !== null) {
setDragging({ robot: clickedRobotKey, offsetX: worldPos.x - r.x, offsetY: worldPos.y - r.y
});
if (canvasRef.current) canvasRef.current.style.cursor = "grabbing";
}
} else {
setAutoCenter(false);
isPanning.current = true;
lastMouse.current = { x: e.clientX, y: e.clientY };
if (canvasRef.current) canvasRef.current.style.cursor = "grabbing";
}
};
```

Figura 4.3: Implementación en TypeScript de la función *HandleMouseDown*.

- **Soporte multitáctil nativo (*handleTouchStart*, *handleTouchMove*, *handleTouchEnd*):** Las pantallas de dispositivos móviles registran las interacciones de forma diferente a un ordenador al admitir múltiples puntos de contacto simultáneos. Para adaptar la simulación, estas tres funciones colaboran de forma secuencial abstrayendo los datos de la pantalla a través de una máquina

de estados interactiva. En primer lugar, *handleTouchStart* detecta el contacto inicial y determina la intención del usuario, discriminando si se ha seleccionado un robot para arrastrar o el fondo para mover el lienzo. Seguidamente, *handleTouchMove* actúa como un bucle de alta frecuencia que captura el deslizamiento del dedo, extrayendo únicamente las coordenadas de la primera pulsación física para inyectarlas directamente en el motor lógico que procesa el ratón. Finalmente, *handleTouchEnd* se encarga de limpiar las variables de estado al levantar el dedo, cerrando el ciclo de forma segura para evitar comportamientos erráticos. Esta sincronización garantiza que acciones como arrastrar un agente o desplazar el mapa mantengan la misma fluidez y precisión tanto en un móvil como en un entorno de escritorio.

- **Transformación bidimensional y cámara (*worldToScreen* y *screenToWorld*):** El motor matemático opera en un plano cartesiano teórico donde el eje Y crece hacia arriba, mientras que la pantalla física es un lienzo de píxeles fijos donde el origen (0,0) está en la esquina superior izquierda y el eje Y crece hacia abajo. Para comunicar ambos entornos, estas dos funciones, cuyo código se muestra en la Figura 4.4, actúan como transformadores algebraicos bidireccionales basados en el factor de escala (zoom) y el desplazamiento de la vista (*offsetX*, *offsetY*). Por un lado, *worldToScreen* traduce las coordenadas lógicas de la simulación a píxeles físicos en pantalla, permitiendo al motor gráfico saber exactamente dónde dibujar cada elemento. Por otro lado, *screenToWorld* realiza el proceso inverso, convirtiendo las coordenadas de los píxeles donde el usuario hace clic o toca con el dedo en puntos del mapa virtual, una operación indispensable para que el sistema reconozca qué robots están siendo arrastrados o modificados en el espacio de la simulación.

```typescript
function worldToScreen(x: number, y: number, w: number, h: number) {
  const sx = (x - offsetX) * zoom + w / 2;
  const sy = h / 2 - (y - offsetY) * zoom;
  return { sx, sy };
}

function screenToWorld(sx: number, sy: number, w: number, h: number) {
  const x = (sx - w / 2) / zoom + offsetX;
  const y = (h / 2 - sy) / zoom + offsetY;
  return { x, y };
}
```

Figura 4.4: Implementación en TypeScript de las funciones *worldToScreen* y *screenToWorld*.

- **Algoritmo de cámara inteligente (*Auto-Center*):** Para evitar que el usuario pierda de vista a los robots en trayectorias de larga distancia, se ha implementado un efecto reactivo (*useEffect*) que actúa como cámara automatizada. El algoritmo evalúa constantemente las coordenadas activas, localiza a los agentes situados en los extremos geométricos y dibuja una caja delimitadora (*Bounding Box*) lógica. A partir de los márgenes de esta caja, el sistema calcula de forma determinista el factor de *zoom* y el centro exacto necesarios para encuadrar toda la acción respetando un margen visual de 100 píxeles, tal y como se demuestra en la secuencia de la Figura 4.5.

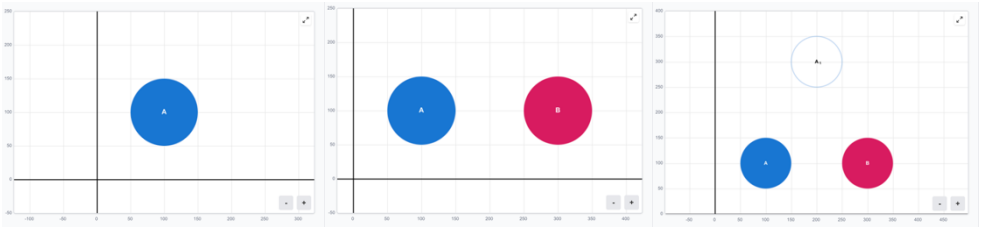


Figura 4.5: Secuencia de adaptación del encuadre gracias a la cámara inteligente.

- **Control manual de perspectiva (*handleZoomIn* y *handleZoomOut*):** Como complemento a la cámara inteligente, el simulador incorpora botones físicos superpuestos al lienzo para manipular la escala de forma interactiva. Al ejecutar estas funciones, el sistema incrementa o reduce progresivamente la variable de *zoom* limitando matemáticamente sus valores máximos y mínimos. Además, cualquier ajuste manual desactiva automáticamente la bandera de centrado (*autoCenter* = false), cediendo el control total al usuario para que pueda inspeccionar cruces de trayectorias concretos sin que el software reencuadre la imagen, tal y como se aprecia en la Figura 4.6, la cual ilustra el mismo escenario final de la Figura 4.5 tras aplicar una reducción de escala visual (*zoom out*).

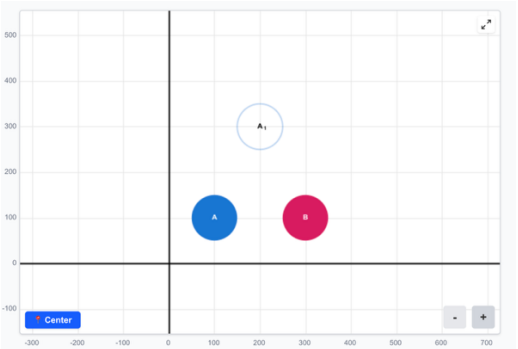


Figura 4.6: Reducción manual de la escala (*zoom out*) mediante los botones del canvas.

- **Redimensionamiento y pantalla completa (*toggleFullscreen* y *ResizeObserver*):** La interfaz del simulador está diseñada para ser completamente adaptable a cualquier tipo de pantalla o resolución. Para garantizar la nitidez de los gráficos, estas dos herramientas cooperan de forma coordinada. Por un lado, la función *toggleFullscreen* se comunica con el navegador para expandir el lienzo interactivo hasta ocupar la totalidad de la pantalla en un modo inmersivo a petición del usuario. Por otro lado, se emplea *ResizeObserver*, una interfaz nativa de la API del navegador en *JavaScript* que monitoriza en segundo plano el espacio asignado al lienzo. En el instante en que este observador detecta que el usuario ha cambiado el tamaño de la ventana del navegador o ha rotado la pantalla de su dispositivo móvil, recalcula de inmediato las dimensiones internas de renderizado (*canvasSize*). Esta actualización fuerza al lienzo a ajustar su número de píxeles reales al espacio disponible, impidiendo que los gráficos sufran distorsiones, se estiren o pierdan definición visual.

4.2.2. El panel de control

Situado habitualmente en un lateral o debajo del canvas en pantallas pequeñas, el panel de control agrupa las herramientas necesarias para configurar los parámetros de la simulación, cuyos elementos se detallan en la Figura 4.7. Se ha estructurado utilizando componentes visuales intuitivos.

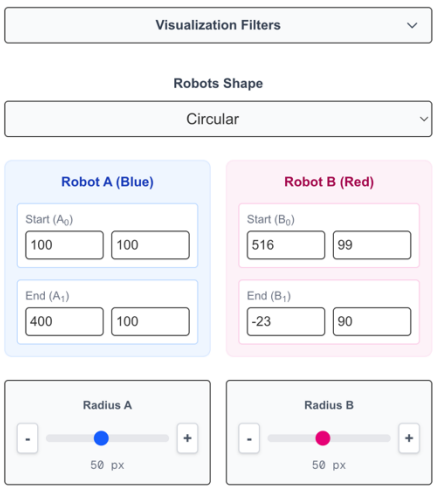


Figura 4.7: Entorno de simulación sobre el lienzo interactivo.

Para su implementación, se han utilizado las siguientes herramientas de *React*:

- **Enlace bidireccional de parámetros y estado reactivo (*useState* y *onChange*):** La interfaz no utiliza formularios web tradicionales en los que el usuario tiene que rellenar datos y pulsar un botón de "Aplicar" para ver los cambios. En su lugar, cada cuadro de texto *(<input type="number">)* y barra deslizante *(<input type="range">)* está conectado a la memoria central del programa mediante el *hook useState* (en variables como A0_x o radioA). Cuando el usuario mueve un control numérico, el evento nativo *onChange* captura inmediatamente ese nuevo valor y ejecuta su función de actualización (por ejemplo, *setRadioA*), tal y como se resalta en la Figura 4.8. Gracias a esto, cualquier cambio que hagamos en el tamaño o posición de los robots se refleja matemáticamente en el lienzo al instante, creando una experiencia mucho más fluida e interactiva.

```
<div className="grid grid-cols-2 gap-4">
<div className="p-3 bg-gray-50 rounded border text-center">
<label className="text-xs font-bold text-gray-600">Radius A</label>
<div className="flex items-center gap-2 mt-2">
<button onClick={() => setRadioA((r) => Math.max(10, r - 1))} className="w-8 h-8
bg-white border border-gray-300 rounded hover:bg-gray-100 flex items-center
justify-center font-bold text-gray-600 shadow-sm">-</button>
<input
type="range"
min="10"
max="100"
value={radioA}
onChange={e => setRadioA(Number(e.target.value))}
className="w-full h-2 bg-gray-200 rounded-lg appearance-none cursor-pointer accent-
blue-600"
/>
<button onClick={() => setRadioA((r) => Math.min(100, r + 1))} className="w-8 h-8
bg-white border border-gray-300 rounded hover:bg-gray-100 flex items-center
justify-center font-bold text-gray-600 shadow-sm">+</button>
</div>
<div className="text-xs text-gray-500 mt-1 font-mono">{radioA} px</div>
</div>
```

Figura 4.8: Implementación en JSX de un control deslizante enlazado al estado de la aplicación mediante el evento *onChange*.

- **Avisos automáticos y limpieza de memoria (*useEffect* y temporizadores):** Para informar al usuario sobre lo que hace el algoritmo (por ejemplo, mostrar qué tipo de escenario matemático ha detectado o lanzar un aviso de colisión en el inicio), se ha creado un sistema de alertas flotantes. Para que estos mensajes no se queden bloqueando la pantalla, desaparecen solos. Esto se ha programado usando un *hook useEffect* que, al detectar que hay un nuevo mensaje que mostrar, arranca una

cuenta atrás con *setTimeout* para ocultarlo automáticamente a los 4 segundos. El detalle técnico más importante de este bloque es que el código termina utilizando la orden *clearTimeout*. Esta instrucción sirve para "limpiar": si el usuario pulsa el botón de inicio cinco veces muy rápido, el sistema destruye el temporizador antiguo antes de crear el nuevo. Esto evita que se acumulen tareas invisibles en segundo plano que podrían terminar saturando el navegador (un problema muy común conocido en programación como *memory leak*) (Meta Platforms, Inc., 2024).

4.3. Capa de lógica y geometría

Esta capa representa el verdadero "cerebro" matemático y predictivo del simulador. A diferencia de la capa de interfaz, que se centra en el usuario, esta sección opera con funciones puras e independientes de la carga gráfica. Su objetivo es doble: por un lado, resolver el problema analítico subyacente (detectar colisiones, calcular tangentes y decidir qué rutas son óptimas) y, por otro, orquestar el flujo lógico de los datos para que el simulador funcione de forma determinista y sin errores de memoria.

A continuación, se desglosan los tres grandes bloques lógicos y las funciones de código que conforman el núcleo computacional de este Trabajo de Fin de Grado:

4.3.1. Funciones de geometría analítica y clasificación

Para mantener el código limpio y estructurado, las operaciones matemáticas fundamentales se han programado como "funciones puras" (módulos aislados que, dados unos mismos datos de entrada, siempre devuelven el mismo resultado sin alterar el resto del programa). Estas funciones constituyen el soporte teórico del simulador:

- **Distancia mínima a una trayectoria (*getDistCenterToSegment*):** Esta función actúa como la herramienta de medición básica de todo el sistema. Su propósito es calcular la distancia más corta en línea recta entre el centro de un robot y el trayecto por el que circulará su compañero. Gracias a este cálculo, el simulador puede comprobar de manera matemática si un agente se encuentra posicionado demasiado cerca o directamente obstruyendo el camino del otro. Si este valor es inferior al margen de seguridad permitido, el software detecta de inmediato el

riesgo de colisión, sirviendo como detonante para activar el resto de las lógicas de evasión (véase Figura 4.9).

```
const getDistCenterToSegment = (px: number, py: number, x1: number, y1: number, x2:
number, y2: number): number => {
const A = px - x1;
const B = py - y1;
const C = x2 - x1;
const D = y2 - y1;

const dot = A * C + B * D;
const len_sq = C * C + D * D;
let param = -1;
if (len_sq !== 0) param = dot / len_sq;

let xx, yy;

if (param < 0) {
xx = x1; yy = y1;
} else if (param > 1) {
xx = x2; yy = y2;
} else {
xx = x1 + param * C;
yy = y1 + param * D;
}

const dx = px - xx;
const dy = py - yy;
return Math.sqrt(dx * dx + dy * dy);
};
```

Figura 4.9: Implementación en TypeScript de la función *getDistCenterToSegment*.

- **Cálculo de orientación espacial (*getSide*):** Esta función implementa una operación geométrica basada en el signo del producto vectorial para determinar la posición relativa de un punto respecto a una línea recta dirigida. Dados dos puntos, A (A_x, A_y) y B (B_x, B_y), que definen una trayectoria, y un punto de prueba P (P_x, P_y), la función calcula el valor numérico de la operación (B_x - A_x) · (P_y - A_y) - (B_y - A_y) · (P_x - A_x). El signo del resultado permite determinar la posición del punto: si es positivo, indica que el punto P se encuentra estrictamente a la izquierda de la trayectoria; si es negativo, a la derecha; y si es cero, significa que los tres puntos están perfectamente alineados. Esta función es el cimiento computacional sobre el que se construyen las decisiones de alto nivel del simulador, permitiendo segmentar el espacio en zonas y discriminar analíticamente entre trayectorias horarias y antihorarias.

- **Validación estática de seguridad (*Prevención de solapamiento*):** Consiste en medir la distancia euclídea entre las coordenadas de origen de ambos robots,

repitiendo el proceso para sus coordenadas de destino. Si esta distancia resulta ser estrictamente menor a la suma de sus radios de seguridad, significa que los agentes colisionan estando parados. En este escenario, la lógica matemática bloquea la ejecución del algoritmo de rutas y emite una señal de error a la capa de interfaz, evitando cálculos innecesarios o comportamientos anómalos.

- **Evaluación de invasión (*isRobotInCorridor*):** Utilizando el cálculo anterior, esta función computa el concepto matemático del *corridor* detallado en los capítulos anteriores. Dado que este ya se modela geométricamente expandido para contemplar la suma de los radios de ambos agentes, la función evalúa si el centro de un robot invade esta área de seguridad proyectada por la trayectoria del otro. Devolverá *true* si detecta esta intrusión de coordenadas, convirtiéndose en el pilar analítico para determinar de forma automatizada quién estorba a quién en el escenario. El código de dicha función se encuentra ilustrado en la Figura 4.10.

```
export const isRobotInCorridor = (
robotPos: Point,
robotRadius: number,
pathStart: Point,
pathEnd: Point,
pathWidthRadius: number
): boolean => {

// Distancia del Centro a la línea central del camino
const distCentro = getDistCenterToSegment(robotPos.x, robotPos.y, pathStart.x,
pathStart.y, pathEnd.x, pathEnd.y);

// Distancia del Borde a la línea central
const distBorde = distCentro - robotRadius;

// ¿Toca la zona de peligro? (Añadimos 0.5 de margen por redondeo de píxeles)
return distBorde < (pathWidthRadius - 0.5);
};
```

Figura 4.10: Implementación en TypeScript de la función *isRobotInCorridor*.

- **Cálculo de rutas de evasión (*getTangentsPointToCircle*):** Cuando un robot necesita esquivar a otro, la ruta más corta matemáticamente implica bordear su radio de seguridad. Este algoritmo geométrico calcula las coordenadas exactas de los dos puntos de tangencia desde la posición del robot que se mueve hacia la circunferencia que rodea al obstáculo.

- **El árbol de decisión lógico (*clasificarEscenario*):** Es la función evaluadora final que da vida a la lógica de clasificación diseñada en el Capítulo 3. Llama a

isRobotInCorridor cuatro veces para construir internamente la tabla de verdad (comprobando si el inicio o el fin de un robot estorban al otro). Dependiendo de las coincidencias booleanas de esta tabla, el algoritmo dictamina y devuelve el tipo exacto de problema a resolver: un caso Easy, Nested o Multi-Obstruction, derivando el control a sus respectivas lógicas de resolución.

4.3.2. Orquestación, evaluación dual y competición de mundos

Una vez que las matemáticas han clasificado el problema, el componente principal debe organizar cómo se van a mover los robots. De esto se encargan las funciones de orquestación:

- **El núcleo coordinador (*handleIniciar*):** Es la función más compleja y extensa del programa. Al pulsar el botón de inicio, congela la interfaz y guarda una "fotografía" (*snapshot*) de las coordenadas exactas. Llama al clasificador y, si el caso es complejo (*Nested* o *Multi*), ejecuta una "competición de mundos". Esto significa que el algoritmo calcula la ruta de evasión completa en sentido horario y simultáneamente en sentido antihorario. Finalmente, compara ambas distancias totales, se queda con la más corta, empaqueta el resultado en una secuencia de turnos ordenada y enciende el motor de la simulación.

- **Cálculo de tangentes (*getTangentPoint*):** Esta función actúa como el motor matemático de bajo nivel del sistema, encargándose de ejecutar la fórmula trigonométrica para obtener las coordenadas de un punto de tangencia individual. Su comportamiento es determinista: se le pide un punto y devuelve únicamente uno, dependiendo de si se le pasa un 1 o un -1 en el parámetro de dirección (*dir*). Es precisamente dentro de esta función donde se implementa el margen de seguridad de 0.01 píxeles para garantizar la robustez del software, evitando que las matemáticas generen divisiones por cero o valores nulos si la distancia es cero, como se aprecia en la Figura 4.11. A diferencia de *getTangentsToCircle* (que funciona como un envoltorio de más alto nivel encargado de llamar a este motor dos veces y usar la lógica de *getSide* para clasificar y devolver un objeto limpio con la tangente izquierda y derecha al esquivar un robot), *getTangentPoint* se invoca de manera directa para resolver la geometría base del simulador, como

calcular las líneas que dividen el *corridor* y separar el cono de visión en distintas zonas espaciales dentro del escenario Multi-Obstruction.

```
  const getTangentPoint = (origin: { x: number, y: number }, target: { x: number,
y: number }, radius: number, dir: number) => {
    const dx = target.x - origin.x;
    const dy = target.y - origin.y;
    const dist = Math.sqrt(dx * dx + dy * dy);

    const mathDist = Math.max(dist, radius + 0.01);

    const angleToCenter = Math.atan2(dy, dx);
    const offset = Math.asin(radius / mathDist);

    const angle = angleToCenter + (dir * offset);
    const L = Math.sqrt(mathDist * mathDist - radius * radius);

    return {
      x: origin.x + L * Math.cos(angle),
      y: origin.y + L * Math.sin(angle)
    };
  };
```

Figura 4.11: Implementación en TypeScript de la función *getTangentPoint*.

- **Controlador de flujo y reinicio (*handleReiniciar*):** Actúa como el freno de emergencia del sistema. Al ejecutarse, frena en seco el bucle de animaciones, vacía la pila de turnos programados y rescata los datos guardados en el snapshot inicial para teletransportar a los robots de vuelta a sus posiciones de origen, dejando el sistema limpio para un nuevo cálculo.

4.3.3. Gestión de estado y sincronización

Para que la simulación se ejecute a 60 fotogramas por segundo (*FPS*) sin que el navegador colapse por falta de memoria, ha sido necesario diseñar una arquitectura de datos que evite los cuellos de botella habituales del *framework React*:

- **Motor de físicas aislado (*simulationRef*):** El problema de usar las variables estándar de React (*useState*) es que cada vez que un dato cambia, el sistema se siente obligado a redibujar toda la interfaz. Si forzamos este proceso 60 veces por segundo para animar el avance de los robots, el navegador se satura y el movimiento se muestra a tirones. Para evitar este cuello de botella, se utilizó la herramienta *useRef* para crear la estructura *simulationRef*. Esta herramienta funciona como una memoria interna silenciosa: guarda las coordenadas exactas

de los robots, su velocidad y la fase del movimiento en tiempo real, pero al actualizarse no fuerza a la pantalla a repintarse. Esto nos permite separar la matemática de los gráficos: el sistema calcula toda la física a máxima velocidad por detrás, y el lienzo simplemente lee esa información de forma independiente para que la animación fluya perfectamente.

- **Sincronizador de variables (*stateRef* y *useEffect*):** Al separar la física (en el *useRef*) de los controles de la interfaz (en el *useState*), surge un problema de sincronización conocido como "cierres obsoletos", donde el motor gráfico intenta leer variables como el *zoom* y obtiene datos antiguos. Para solucionarlo, se implementó el *Hook useEffect*, una función nativa de React que actúa como un espejo: cada vez que el usuario toca un control en la pantalla, este efecto hace una copia instantánea del dato y la guarda en *stateRef*, garantizando que el bucle de dibujo siempre tenga acceso a la información más reciente sin interrumpir su ejecución.

- **Diccionario unificado de agentes (*robotMap* con *useMemo*):** Para mantener el orden de los datos de los cuatro agentes ('A_0', 'A_1', 'B_0', 'B_1'), se ha creado un objeto que agrupa sus posiciones, radios, colores y funciones asociadas. Al envolverlo en la función *useMemo*, le indicamos al programa que memorice esta estructura y solo la reconstruya si el usuario altera deliberadamente un radio o una posición. Esto optimiza enormemente el consumo de memoria, evitando cálculos redundantes durante los ciclos de inactividad del simulador.

4.4. Capa de renderizado

Esta capa actúa como el motor traductor del simulador y no debe confundirse con la interfaz de controles (sección 4.2). Su función exclusiva es coger las matemáticas abstractas, los vectores y las coordenadas invisibles que calcula la capa de lógica (sección 4.3) y transformarlos en gráficos pixelados sobre el lienzo. Es decir, es la responsable de materializar visualmente las decisiones del algoritmo para que el usuario pueda ver la simulación en tiempo real.

A continuación, se detallan las mecánicas de dibujo, los ciclos de animación y las funciones de infraestructura visual que construyen la interfaz final:

4.4.1. El bucle de animación

Para que el movimiento de los robots no se perciba como saltos bruscos entre coordenadas, el sistema no dibuja imágenes estáticas, sino que funciona como un motor de videojuegos convencional, generando fotogramas continuos.

- **El motor de fotogramas (*loop* con *requestAnimationFrame*):** Esta función es el motor continuo del simulador. En lugar de usar un temporizador web tradicional que obliga al sistema a repintar la pantalla cada ciertos milisegundos (lo que puede causar bloqueos y tirones), utiliza la herramienta *requestAnimationFrame*. Esta instrucción se sincroniza automáticamente con el monitor del usuario (habitualmente a 60 fotogramas por segundo) para dibujar la siguiente imagen solo cuando la pantalla está lista, evitando la desincronización visual tal y como se ilustra en el esquema de la Figura 4.12. En cada uno de estos fotogramas, el bucle actualiza el reloj de la simulación, mueve la cámara suavemente si el usuario la está arrastrando y redibuja toda la escena.

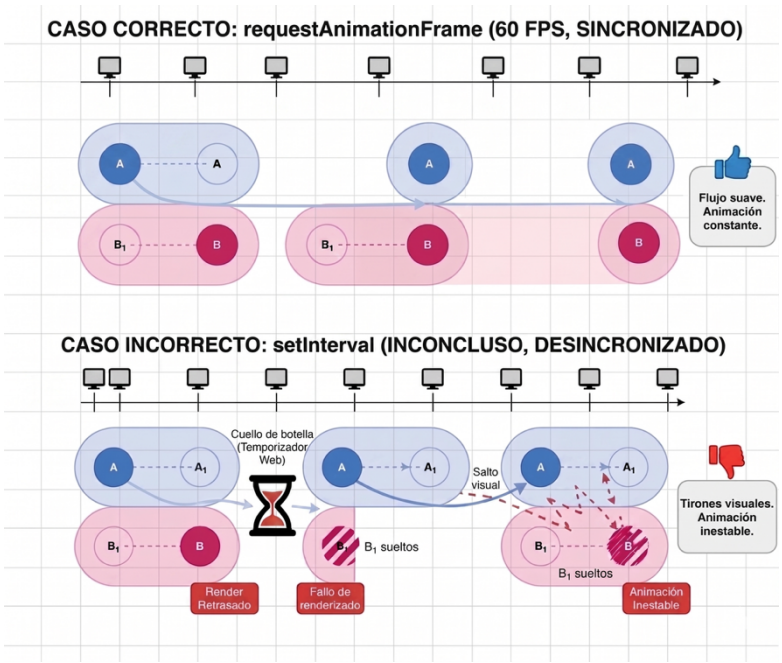


Figura 4.12: Comparativa del ciclo de renderizado: sincronización mediante *requestAnimationFrame* frente a desincronización en temporizadores tradicionales.

- **Interpolación lineal de movimiento (*lerp*):** En la programación gráfica, si animamos un robot pidiéndole que avance un cierto número de píxeles por

fotograma, un ordenador lento haría que el robot tardase más tiempo en llegar a su destino. Para evitar esto, se usa la función *lerp* (Interpolación Lineal), una técnica ampliamente utilizada en gráficos por computador para calcular posiciones intermedias de manera continua y dependiente del tiempo (Foley et al., 1996). En lugar de sumar píxeles, esta fórmula matemática calcula la posición exacta donde debe dibujarse el robot basándose únicamente en el tiempo transcurrido. Por ejemplo, si ha pasado el 50% del tiempo estimado para el viaje, *lerp* dibujará al robot exactamente en la mitad de la recta. Esto garantiza que la simulación sea perfecta y constante, incluso si el navegador sufre una caída de rendimiento temporal (Folley et al., 1996).

4.4.2. Dibujado por capas y componentes gráficos

El dibujo bidimensional se ejecuta mediante un patrón de diseño conocido como "algoritmo del pintor", donde los elementos se dibujan uno encima de otro en un orden estricto, de forma que los objetos renderizados posteriormente ocultan visualmente a los anteriores (Foley et al., 1996).

- **Ciclo de limpieza y dibujado de capas (Sección de dibujo del *loop*):** En cada fotograma, el sistema ejecuta la orden *clearRect* para borrar absolutamente todo el contenido de la pantalla anterior, evitando que los gráficos dejen una "estela". Acto seguido, reconstruye la escena de fondo a frente: primero dibuja la retícula cuadriculada del mapa; luego los *corridors* translúcidos; encima, los polígonos matemáticos de las zonas calculadas y las líneas de intersección discontinuas; y finalmente, en la capa superior, el cuerpo opaco de los agentes (función *drawRobot*) para que nunca queden ocultos bajo otros elementos gráficos.

- **Procesador tipográfico dinámico (*drawSmartText*):** La API nativa del Canvas no entiende de formatos de texto complejos ni subíndices matemáticos. Para solventarlo, se diseñó esta función personalizada. Analiza las etiquetas de texto introducidas (como 'A_1' o 'P_{int}'), aísla los caracteres correspondientes al subíndice numérico o literal, y reduce automáticamente el tamaño de la fuente desplazándola unos píxeles hacia abajo. Esto permite renderizar en el lienzo la notación geométrica formal (A_1, B_0, P_{int}) con un acabado profesional (véase Figura 4.13).

```
  const drawSmartText = (ctx: CanvasRenderingContext2D, label: string, x: number,
y: number, size: number) => {
    ctx.font = `bold ${size}px Arial`;

    if (label.includes('_') || (label.length > 1 && /\d/.test(label))) {
      const parts = label.split('_');
      const main = parts[0].replace(/\d/g, '');
      const sub = parts[1] || label.match(/\d/)?.[0] || "";

      ctx.fillText(main, x, y);
      const mainWidth = ctx.measureText(main).width;
      ctx.font = `bold ${size * 0.7}px Arial`;
      ctx.fillText(sub, x + mainWidth, y + (size * 0.3));
    } else {
      ctx.fillText(label, x, y);
    }
  };
```

Figura 4.13: Implementación en TypeScript de la función *drawSmartText*.

- **Marcadores numéricos de los ejes (*leftTicks* y *bottomTicks* con *useMemo*):** Son los números que se proyectan en los márgenes izquierdo e inferior del *canvas*, funcionando como una regla de medir para que el usuario conozca las coordenadas exactas que está visualizando. Dado que el simulador permite hacer *zoom*, estos números deben adaptarse: si alejamos la cámara, los números se amontonarían. Para evitarlo, estas lógicas calculan dinámicamente el espaciado, multiplicando o dividiendo la distancia entre los números (por ejemplo, saltando de 10 en 10, o de 50 en 50) según el nivel de aumento. Para no saturar la memoria del ordenador calculando esto en cada fotograma, se utiliza la herramienta *useMemo*, que memoriza la regla y solo la recalcula si el usuario cambia el *zoom* o mueve el mapa.

4.4.3. Compatibilidad y soporte del entorno web

El desarrollo de aplicaciones modernas con *frameworks* híbridos como *Next.js* requiere medidas de seguridad específicas para garantizar que el código se ejecute en el lugar correcto (el ordenador del usuario y no el servidor). Para lograrlo, se ha implementado la siguiente estructura de control:

- **Control de hidratación y montaje (*mounted*):** *Next.js* utiliza una tecnología de optimización que intenta renderizar el código en el servidor antes de enviarlo al navegador. Sin embargo, elementos interactivos como el *canvas*, la ventana del navegador o el ratón del usuario no existen en el servidor, lo que provocaría un

error crítico de desincronización (conocido técnicamente como *Hydration Mismatch*). Para solucionar esto, se ha programado una variable booleana *mounted* vinculada a un efecto inicial (*useEffect*). Esta lógica actúa como un semáforo en rojo: obliga al programa a esperar y solo renderiza la interfaz gráfica cuando tiene confirmación absoluta de que la aplicación ya se ha montado de forma segura en el navegador físico del cliente.

Capítulo 5: Conclusiones y Trabajo Futuro

Este último capítulo tiene como objetivo realizar una valoración global del proyecto. A continuación, se presenta una reflexión crítica sobre el proceso de ingeniería llevado a cabo, analizando en primer lugar los obstáculos técnicos y geométricos a los que el sistema ha tenido que sobreponerse. Finalmente, se expondrán las conclusiones definitivas extraídas de la consecución de los objetivos iniciales, así como la proyección de futuras líneas de investigación y mejoras que podrían aplicarse para escalar y perfeccionar la herramienta en el futuro.

5.1. Dificultades y retos superados

- **La optimización de la ruta en el caso Múltiple:** Evaluar las cuatro posibilidades de evasión simultáneas (A-CW, A-CCW, B-CW, B-CCW) supuso una barrera de testeo enorme. Al procesar cada iteración, la estructura geométrica, la delimitación de zonas y los roles de los robots cambiaban por completo. Depurar el código y validar que el simulador era capaz de procesar estos cuatro mundos paralelos y elegir correctamente el camino óptimo fue uno de los mayores retos del proyecto.

- **El cambio a un sistema de referencia relativo:** Inicialmente, a la hora de elegir las tangentes para las rutas Clockwise o Counter-Clockwise, tomábamos la decisión basándonos en si estaban "arriba" o "abajo" en la pantalla. Sin embargo, puesto que el simulador permite cualquier disposición en el lienzo, esta lógica estática fallaba en cuanto el escenario rotaba. El reto consistió en abandonar esa visión absoluta de la pantalla y reconstruir las matemáticas para basar la lógica en si la tangente se ubicaba a la "derecha" o a la "izquierda" del propio vector de movimiento del robot. Gracias a este enfoque relativo, el sistema geométrico selecciona siempre la tangente correcta sin importar la orientación de los agentes.

- **La unificación matemática de las tangentes de evasión:** En relación con el punto anterior, conseguir que el motor de evaluación dual aplicara de forma estricta esta regla de flancos fue otra gran dificultad. El desafío radicaba en programar el algoritmo para que descartara la mitad de la geometría y utilizara de manera unificada un único juego de tangentes (solo las derechas o solo las

izquierdas). Aplicar este mismo criterio tanto para calcular las fronteras de las zonas espaciales como para trazar los puntos exactos de la ruta de evasión final, supuso un proceso laborioso.

- **El enfoque e implementación del "Modo Espejo":** Detectar mediante código cuándo la lógica frontal del cono iba a fallar irremediablemente (al ocurrir la invasión cerca de los destinos o proyectarse fuera del cono) y crear la lógica para invertir los puntos de inicio y fin, engañando al propio motor para que resolviera el conflicto "hacia atrás" y luego revirtiera el movimiento visualmente.

- **Polígonos auto-intersecantes:** A la hora de programar el dibujo de las zonas en el lienzo, la unión secuencial de los vértices mediante *lineTo* (una función de la API nativa de Canvas encargada de conectar las coordenadas actuales del trazado con un nuevo punto mediante una línea recta) presentó un grave problema de renderizado. En situaciones límite, cuando el punto de referencia T y el origen del robot compartían prácticamente las mismas coordenadas, las líneas de las zonas se cruzaban sobre sí mismas antes de cerrar la figura. Esta auto-intersección generaba un fallo en el algoritmo nativo de relleno de color, provocando un desbordamiento gráfico por toda la pantalla. Para solucionarlo, tuvimos que implementar una condición geométrica de seguridad que detecta este colapso espacial y simplifica dinámicamente los vértices del polígono antes de dibujarlo, garantizando una interfaz visual estable.

- **El bucle de físicas en React:** React funciona a base de "estados" (*useState*). Uno de los mayores cuellos de botella técnicos de todo el proyecto surgió al intentar que la simulación se moviera de forma fluida. Dado que la interfaz se diseñó utilizando *React*, inicialmente se emplearon los "estados" del sistema (*useState*) para actualizar la posición de los robots. El grave problema que se detectó fue que, al intentar recalcular y repintar los componentes 60 veces por segundo, la arquitectura de *React* se saturaba, disparaba el consumo de memoria RAM y el movimiento en pantalla resultaba inaceptablemente errático y a tirones. El gran reto consistió en encontrar una forma de "engañar" al framework para separar las matemáticas del dibujado visual. La solución requirió reestructurar el código aislando las variables de movimiento en referencias estáticas (*simulationRef*) e inyectando un bucle nativo del navegador (*requestAnimationFrame*). Además,

hubo que superar un segundo obstáculo: evitar que los robots se desincronizaran y chocaran si el ordenador del usuario sufría una bajada de rendimiento. Para asegurar esta robustez temporal se tuvo que investigar e implementar la interpolación lineal (*lerp*), abandonando la idea de desplazar los robots sumando píxeles fijos y logrando que el sistema calculara sus posiciones basándose estrictamente en el tiempo transcurrido.

- **Mapeo de coordenadas y desarrollo de la cámara inteligente (Auto-Center):** Otro gran desafío a nivel de desarrollo visual fue la desconexión geométrica entre el "Mundo" de simulación y la pantalla física. El motor matemático opera en un plano cartesiano teóricamente infinito donde un robot puede moverse hacia la coordenada (5000, 5000) sin problema, pero el usuario visualiza la acción a través de un rectángulo limitado (el Canvas). Superar esta barrera obligó a programar desde cero un sistema bidireccional de traducción de coordenadas que soportara niveles dinámicos de escala (zoom) y desplazamiento espacial (*panning*) en tiempo real. Sin embargo, el mayor reto de este apartado fue programar el centrado automático continuo. Para evitar que el usuario perdiera de vista a los robots en trayectorias largas, se tuvo que diseñar un algoritmo que evalúa constantemente las posiciones activas, localiza a los agentes situados en los extremos (arriba, abajo, izquierda y derecha) y dibuja una caja delimitadora (*Bounding Box*) invisible que los engloba a todos. Calcular matemáticamente la cantidad exacta de zoom y desplazamiento que la cámara debe aplicar en cada fotograma para que toda esa caja encaje en la pantalla con un margen limpio de 100 píxeles, requirió numerosas horas de ajuste para lograr una experiencia fluida y sin saltos bruscos.

- **La inversión del eje vertical en la pantalla (Canvas):** Un problema constante durante el desarrollo de la lógica geométrica fue la diferencia entre cómo funcionan las matemáticas teóricas y cómo dibuja el navegador de internet. En las matemáticas clásicas que usa el motor del simulador, el eje vertical (Y) crece hacia arriba. Sin embargo, en el Canvas de la pantalla, el punto (0,0) está en la esquina superior izquierda, por lo que el eje Y está invertido y crece hacia abajo. Implementar toda la lógica de conos, tangentes y zonas, teniendo que pensar "al revés" cada vez que mirábamos la pantalla para comprobar si algo funcionaba, complicó mucho la programación. Para solucionar esta confusión visual y no

llenar las fórmulas de cambios de signo, creamos una capa intermedia en el código (las funciones *worldToScreen* y *screenToWorld*). De esta forma, el motor matemático calcula todo en un plano normal y corriente, y solo se le da la vuelta a la coordenada en el último momento, justo antes de pintar los robots en la pantalla.

- **Errores de precisión matemática y bloqueos del simulador (*Fallbacks*):** Otro reto importante fue lidiar con la diferencia entre la geometría teórica y la programación real. Sobre el papel, la fórmula para calcular dónde se cortan dos rectas tangentes es exacta. Sin embargo, en el código, las variables tienen un número limitado de decimales. En ciertos escenarios, cuando las rectas divisorias eran casi paralelas, estos pequeños errores de redondeo computacional hacían que la fórmula fallara y devolviera un valor nulo. Esto supuso una gran dificultad durante las pruebas, ya que, si el sistema no lograba encontrar el punto de intersección exacto, el robot intentaba moverse a una coordenada vacía y el simulador se bloqueaba por completo. Para solucionar esta inestabilidad sin sacrificar el rendimiento, tuvimos que programar mecanismos de seguridad o "*fallbacks*". Si el cálculo matemático principal fracasa, el código lo detecta a tiempo y aplica una solución de emergencia: calcula el punto medio de la trayectoria del robot anidado (entre su propio inicio y destino) y dirige la ruta hacia allí. Gracias a este enfoque, conseguimos que un simple fallo de precisión con los decimales no detenga la ejecución de todo el programa.

5.2. Conclusiones

Tras el diseño y desarrollo de este simulador de movimiento coordinado, se puede concluir que los objetivos planteados al inicio del proyecto se han alcanzado con éxito. Se ha logrado transformar modelos teóricos y matemáticos, que a menudo resultan demasiado abstractos, en una herramienta web interactiva y funcional. Este proyecto demuestra que es posible aplicar geometría pura para lograr que dos robots eviten colisiones de forma exacta, calculando sus rutas al instante y de manera visual.

A diferencia de otros métodos más clásicos, el motor lógico desarrollado destaca por su total autonomía a la hora de tomar decisiones. Gracias al sistema de clasificación mediante los pasillos virtuales, el programa es capaz de identificar por sí mismo si se

enfrenta a un escenario Fácil, Anidado o Multi Obstruyente, aplicando la solución que corresponde en cada caso sin que el usuario tenga que intervenir. Además, no solo se han conseguido evitar los choques manteniendo la distancia de seguridad, sino que el sistema asegura elegir siempre el mejor camino. Al calcular en paralelo el rodeo por un lado y por el otro, el simulador compara las distancias totales y garantiza matemáticamente que la trayectoria elegida sea siempre la más rápida.

Por otro lado, el proyecto ha dejado clara la necesidad de construir un algoritmo robusto y preparado para situaciones extremas. Los cruces más complicados, como aquellos en los que ambos robots se estorban mutuamente cerca de sus destinos, habrían provocado el bloqueo de un sistema sencillo. Sin embargo, la invención de soluciones como el "Modo Espejo", que permite calcular las rutas hacia atrás para luego darles la vuelta, ha dotado al simulador de una gran fiabilidad. A esto hay que sumarle la implementación de mecanismos de seguridad frente a los errores de precisión con los decimales, logrando que el programa no se detenga ante pequeños fallos de cálculo.

Finalmente, desde el punto de vista tecnológico, se ha comprobado que es totalmente viable ejecutar cálculos físicos exigentes directamente en un navegador de internet (disponible en el enlace: https://simulador-robots-tfg-daniel-senespl.vercel.app/) sin perder rendimiento. El acierto de separar la lógica matemática del dibujo en pantalla ha permitido que la animación de los robots se mueva con total fluidez. Todo ello, sumado a una interfaz amigable que incluye una cámara con centrado automático y un modo visual de depuración, convierte a este trabajo en una base muy sólida. El resultado es una plataforma accesible, fácil de comprender y preparada para seguir creciendo y añadiendo nuevas funcionalidades en el futuro.

5.3. Mejoras a futuro

El simulador desarrollado en este proyecto asienta una base matemática sólida y completamente funcional, pero como ocurre con cualquier herramienta de software escalable, presenta diversas líneas de investigación y mejora que podrían explorarse en el futuro. El primer paso para evolucionar el algoritmo consistiría en ampliar la morfología de los agentes. Actualmente, el motor lógico está altamente optimizado para trabajar con radios de seguridad circulares. Una mejora significativa sería extender el motor

geométrico para que sea capaz de procesar robots con diversas morfologías, incluyendo no solo cuadrados y rectángulos (para los cuales la lógica de tangentes actual es ya prácticamente aplicable), sino también triángulos y otras formas geométricas más complejas. El reto en este punto sería adaptar el cálculo de los márgenes de seguridad para que el sistema siga utilizando la misma lógica de zonas y maniobras de evasión independientemente de la figura del robot. Para lograr esta generalización, se podría integrar el cálculo de espacios de configuración mediante técnicas como las sumas de Minkowski, permitiendo que el simulador trabaje con las dimensiones exactas de cualquier vehículo industrial real sin perder la robustez del algoritmo de resolución geométrica original.

Otra posible mejora sería dar el salto del entorno virtual al mundo físico, conectando el algoritmo a plataformas robóticas reales para realizar pruebas de laboratorio. Aunque el simulador funciona en un navegador web, es perfectamente viable utilizarlo como una estación de control centralizada. Para implementar esto, se dotaría a la aplicación de un sistema de comunicación bidireccional en tiempo real, por ejemplo, mediante WebSockets o pasarelas de conexión como ROSBridge (Robot Operating System). De esta forma, el motor web calcularía la ruta óptima de evasión y transmitiría instantáneamente las coordenadas, las velocidades y los turnos de paso a los microcontroladores de los robots físicos, verificando que la fluidez lograda en la pantalla se traduce correctamente al movimiento de los motores en el mundo real.

Finalmente, desde el punto de vista de la complejidad del entorno, el sistema podría expandirse para gestionar escenarios mucho más densos. Por un lado, se podría escalar la algoritmia para coordinar a tres o más robots simultáneamente. Esto supondría un nuevo reto lógico, ya que obligaría a resolver conflictos en cadena y a diseñar mecanismos para evitar bloqueos mutuos o deadlocks cuando varios agentes intentan cederse el paso a la vez. Por otro lado, la inclusión de obstáculos estáticos en el lienzo, como paredes o mobiliario, enriquecería enormemente la simulación. El motor matemático tendría que evolucionar para no solo evitar el choque entre los propios robots, sino para asegurar que las rutas de evasión trazadas no terminen empujando a los agentes contra los límites físicos del entorno.

Bibliografía

Choset, H., Lynch, K. M., Hutchinson, S., Kantor, G., Burgard, W., Kavraki, L. E., & Thrun, S. (2005). Principles of robot motion: Theory, algorithms, and implementation. *MIT Press.*

Esteban, G., Halperin, D., & Silveira, R. I. (2025). Shortest coordinated motion for square robots. *Autonomous Robots, 49*, 14.

Fiorini, P., & Shiller, Z. (1998). Motion planning in dynamic environments using velocity obstacles. *The International Journal of Robotics Research*, 17(7), 760-772.

Foley, J. D., van Dam, A., Feiner, S. K., & Hughes, J. F. (1996). *Computer Graphics: Principles and Practice (2.ª ed.)*. Addison-Wesley Proffessional.

Icking, C., Rote, G., Welzl, E., & Yap, C. (1993). Shortest paths for line segments. *Algorithmica*, 10(2), 182–200.

Khatib, O. (1986). Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research,* 5(1), 90-98.

Kirkpatrick, D. G., & Liu, P. (2016). Characterizing minimum-length coordinated motions for two discs. *Proceedings of the 28th Canadian Conference on Computational Geometry* (pp. 252–259).

Kirkpatrick, D. G. & Liu, P. (2025). Minimum-length coordinated motions for two convex centrally-simetric robots. *arXiv preprint arXiv:2503.02010.*

Lozano-Pérez, T. (1983). *Spatial planning: A configuration space approach. IEEE Transactions on Computers*, 32(2), 108-120.

Meta Platforms, Inc. (2024). *React: The library for web and native user interfaces*. Recuperado de https://react.dev/

Microsoft Corporation. (2024). *TypeScript: JavaScript with syntax for types*. Recuperado de https://www.typescriptlang.org/

Mozilla Foundation. (2024). *Canvas API*. MDN Web Docs. Recuperado de https://developer.mozilla.org/es/docs/Web/API/Canvas_API

Open Robotics. (s.f.). *Gazebo: Robust physics simulation for software developers*. Recuperado de https://gazebosim.org/

Open Robotics. (s.f.). *ROS: Robot Operating System*. Recuperado de https://www.ros.org/

Reeds, J. A., & Shepp, L. A. (1990). Optimal paths for a car that goes both forwards and backwards. *Pacific Journal of Mathematics*, 145(2), 367–393.

Sharir, M., & Sifrony, S. (1991). Coordinated motion planning for two independent robots. *Annals of Mathematics and Artificial Intelligence*, 3(1), 107–130.

Vercel, Inc. (2024). *Next.js: The React Framework for the Web*. Recuperado de https://nextjs.org/

Enlaces de acceso

Con el fin de facilitar la evaluación de la plataforma interactiva en tiempo real y permitir una revisión ágil de los algoritmos sin necesidad de realizar un despliegue o instalación en local, se puede acceder al simulador web directamente a través del siguiente enlace o escaneando el código QR adjunto para su visualización en dispositivos móviles o tabletas:

https://simulador-robots-tfg-daniel-senespl.vercel.app/

Anexo

En este anexo se recoge el código fuente principal correspondiente al desarrollo de nuestro simulador. El código se divide en dos archivos fundamentales: por un lado, *simulationLogic.tsx*, que abstrae la lógica matemática encargada de evaluar la introducción de los agentes en los corridor del otro agente y clasificar el caso geométrico en el que nos encontramos (*Easy*, *Nested* o *Multi-Obstruction*); y por otro lado, el componente principal *page.tsx*, que concentra toda la gestión del estado reactivo de la interfaz, la interactividad del usuario con el ratón, la cámara y el motor continuo de renderizado sobre el lienzo, etc.

simulationLogic.tsx

```
export type Point = { x: number; y: number };

// Funcion auxiliar para calcular la distancia del centro de un robot a una línea
(segmento) definida por dos puntos
const getDistCenterToSegment = (px: number, py: number, x1: number, y1: number, x2:
number, y2: number): number => {
  const A = px - x1;
  const B = py - y1;
  const C = x2 - x1;
  const D = y2 - y1;

  const dot = A * C + B * D;
  const len_sq = C * C + D * D;
  let param = -1;
  if (len_sq !== 0) param = dot / len_sq;

  let xx, yy;

  if (param < 0) {
    xx = x1; yy = y1;
  } else if (param > 1) {
    xx = x2; yy = y2;
  } else {
    xx = x1 + param * C;
    yy = y1 + param * D;
  }

  const dx = px - xx;
  const dy = py - yy;
  return Math.sqrt(dx * dx + dy * dy);
};

// Función para determinar si un robot entra en el "pasillo" definido por el camino
del otro robot
export const isRobotInCorridor = (
  robotPos: Point,
  robotRadius: number,
  pathStart: Point,
  pathEnd: Point,
  pathWidthRadius: number
): boolean => {
```

```
  // Distancia del Centro a la línea central del camino
  const distCentro = getDistCenterToSegment(robotPos.x, robotPos.y, pathStart.x,
pathStart.y, pathEnd.x, pathEnd.y);

  // Distancia del Borde a la línea central
  const distBorde = distCentro - robotRadius;

  // ¿Toca la zona de peligro? (Añadimos 0.5 de margen por redondeo de píxeles)
  return distBorde < (pathWidthRadius - 0.5);
};

export const getTangentsPointToCircle = (
  Px: number, Py: number,
  Cx: number, Cy: number,
  r: number
): Array<{ x: number, y: number }> => {
  const dx = Cx - Px;
  const dy = Cy - Py;
  const distSq = dx * dx + dy * dy;
  const dist = Math.sqrt(distSq);

  if (dist < r) return []; // El punto está dentro

  const angleToCenter = Math.atan2(dy, dx);
  const offset = Math.asin(r / dist);

  // Tangentes
  const t1 = angleToCenter - offset;
  const t2 = angleToCenter + offset;

  // Distancia del punto al punto de tangencia
  const L = Math.sqrt(distSq - r * r);

  return [
    { x: Px + L * Math.cos(t1), y: Py + L * Math.sin(t1) },
    { x: Px + L * Math.cos(t2), y: Py + L * Math.sin(t2) }
  ];
};


// Clasifiador de escenarios basado en la posición de los robots respecto al camino
del otro
export const clasificarEscenario = (
  A0: Point, A1: Point,
  B0: Point, B1: Point,
  radioA: number,
  radioB: number
): string => {

  // A. Analizamos si el Robot A entra en el camino de B
  const A0_in = isRobotInCorridor(A0, radioA, B0, B1, radioB);
  const A1_in = isRobotInCorridor(A1, radioA, B0, B1, radioB);

  // B. Analizamos si el Robot B entra en el camino de A
  const B0_in = isRobotInCorridor(B0, radioB, A0, A1, radioA);
  const B1_in = isRobotInCorridor(B1, radioB, A0, A1, radioA);

  // Logs para depurar
  console.log(`A0 in B? ${A0_in} | A1 in B? ${A1_in}`);
  console.log(`B0 in A? ${B0_in} | B1 in A? ${B1_in}`);
  console.group("🔍 Diagnóstico de Caso");
  console.log(`A0 (Inicio Azul) en Rojo?  ${A0_in ? "SÍ" : "NO"}`);
  console.log(`A1 (Fin Azul)    en Rojo?  ${A1_in ? "SÍ" : "NO"}`);
  console.log(`B0 (Inicio Rojo) en Azul?  ${B0_in ? "SÍ" : "NO"}`);
  console.log(`B1 (Fin Rojo)    en Azul?  ${B1_in ? "SÍ" : "NO"}`);
  console.groupEnd();
```

```
  // Condiciones if basadas en la tabla de verdad de la memoria
  if ((A0_in && B0_in) || (A1_in && B1_in)) {
    return "Multi-Obstruction";
  }
  if ((A0_in && A1_in) || (B0_in && B1_in)) {
    return "Nested";
  }
  return "Easy";
};
```

page.tsx

```
"use client";

import Image from "next/image";
import { useState, useEffect, useRef, useMemo } from "react";
import { clasificarEscenario, isRobotInCorridor, getTangentsPointToCircle } from
"./simulationLogic";

interface DraggingState {
  robot: 'A0' | 'A1' | 'B0' | 'B1';
  offsetX: number;
  offsetY: number;
}

//Estructura que define las propiedades de cada robot
interface RobotData {
  x: number | null;
  y: number | null;
  setX: React.Dispatch<React.SetStateAction<number | null>>;
  setY: React.Dispatch<React.SetStateAction<number | null>>;
  radio: number;
  color: string;
  border: boolean;
  label: string;
}

export default function Home() {
  // Estado para evitar error de hidratación
  const [mounted, setMounted] = useState(false);

  const [debugFilters, setDebugFilters] = useState({
    corridors: true,
    zones: true,
    pint: true,
    trails: true,
    popups: true
  });
  const [isFiltersOpen, setIsFiltersOpen] = useState(false);

  // Estados de configuración
  const [forma, setForma] = useState<string>("circular");
  const [radioA, setRadioA] = useState<number>(50);
  const [radioB, setRadioB] = useState<number>(50);

  // Estados de posición
  const [A0_x, setA0_x] = useState<number | null>(null);
  const [A0_y, setA0_y] = useState<number | null>(null);
  const [A1_x, setA1_x] = useState<number | null>(null);
  const [A1_y, setA1_y] = useState<number | null>(null);
  const [B0_x, setB0_x] = useState<number | null>(null);
  const [B0_y, setB0_y] = useState<number | null>(null);
  const [B1_x, setB1_x] = useState<number | null>(null);
```

```
  const [B1_y, setB1_y] = useState<number | null>(null);

  // Memoria para guardar la foto de las posiciones antes de iniciar la simulación
  const [snapshot, setSnapshot] = useState<{
    A0_x: number | null, A0_y: number | null,
    A1_x: number | null, A1_y: number | null,
    B0_x: number | null, B0_y: number | null,
    B1_x: number | null, B1_y: number | null
  } | null>(null);

  const [showCorridors, setShowCorridors] = useState(false);

  // Gestor del ciclo de vida de la animación para evitar tirones de renderizado
  const simulationRef = useRef({
    active: false,
    phase: 'idle',
    t: 0,
    speed: 0.02,
    turns: [] as { robot: 'A' | 'B', path: { x: number, y: number }[] }[],
    turnIndex: 0,
    currentSegment: 0,
    posA: { x: 0, y: 0 },
    posB: { x: 0, y: 0 },
    trailA: [] as { x: number, y: number }[],
    trailB: [] as { x: number, y: number }[]
  });

  // Función auxiliar de interpolación lineal (LERP)
  const lerp = (start: number, end: number, t: number) => start + (end - start) * t;


  const canvasRef = useRef<HTMLCanvasElement | null>(null);
  const visualizationContainerRef = useRef<HTMLDivElement>(null);
  const [isFullscreen, setIsFullscreen] = useState(false);
  const [canvasSize, setCanvasSize] = useState({ width: 0, height: 0 });
  const viewportMargin = 40;
  const [offsetX, setOffsetX] = useState<number>(300);
  const [offsetY, setOffsetY] = useState<number>(200);
  const [zoom, setZoom] = useState<number>(1);
  const [autoCenter, setAutoCenter] = useState(true);

  // Estado para visualizar las zonas
  const [debugLines, setDebugLines] = useState<Array<{ x1: number, y1: number, x2:
number, y2: number, color: string }>>([]);

  const [zoneInfo, setZoneInfo] = useState<{
    t1: { x: number, y: number },
    t2: { x: number, y: number },
    detected: string
  } | null>(null);

  // Estado para visualizar el Punto de Intersección en Zonas 3 y 4
  const [pInt, setPInt] = useState<{ x: number, y: number } | null>(null);

  // Estado para las líneas de la intersección
  const [pIntLines, setPIntLines] = useState<{ p1: { x: number, y: number }, p2: { x:
number, y: number } }[]>([]);

  // Estados de interacción del usuario
  const isPanning = useRef(false);
  const lastMouse = useRef({ x: 0, y: 0 });
  const [mouseScreenPos, setMouseScreenPos] = useState<{ sx: number, sy: number } |
null>(null);
  const [dragging, setDragging] = useState<DraggingState | null>(null);

  // Popup
```

```
  const [popup, setPopup] = useState<{ show: boolean, message: string, type: 'error'
| 'success' }>({
    show: false,
    message: "",
    type: "error"
  });

  // Efecto de montaje (Para evitar errores de Next.js)
  useEffect(() => {
    setMounted(true);
  }, []);

  //Recolector de basura para el popup)
  useEffect(() => {
    if (popup.show) {
      const timer = setTimeout(() => {
        setPopup(prev => ({ ...prev, show: false }));
      }, 4000);
      return () => clearTimeout(timer);
    }
  }, [popup.show]);

  const stateRef = useRef({
    offsetX, offsetY, zoom, dragging,
    robots: {
      A0: { x: A0_x, y: A0_y, radio: radioA },
      A1: { x: A1_x, y: A1_y, radio: radioA },
      B0: { x: B0_x, y: B0_y, radio: radioB },
      B1: { x: B1_x, y: B1_y, radio: radioB },
    }
  });

  useEffect(() => {
    stateRef.current = {
      offsetX, offsetY, zoom, dragging,
      robots: {
        A0: { x: A0_x, y: A0_y, radio: radioA },
        A1: { x: A1_x, y: A1_y, radio: radioA },
        B0: { x: B0_x, y: B0_y, radio: radioB },
        B1: { x: B1_x, y: B1_y, radio: radioB },
      }
    };
  }, [offsetX, offsetY, zoom, dragging, A0_x, A0_y, A1_x, A1_y, B0_x, B0_y, B1_x,
B1_y, radioA, radioB]);


  const baseSpacing = 50;

  const robotMap = useMemo<Record<'A0' | 'A1' | 'B0' | 'B1', RobotData>>(() => ({

    A0: { x: A0_x, y: A0_y, setX: setA0_x, setY: setA0_y, radio: radioA, color:
"#1976d2", border: false, label: "A0" },
    A1: { x: A1_x, y: A1_y, setX: setA1_x, setY: setA1_y, radio: radioA, color:
"#90caf9", border: true, label: "A1" },
    B0: { x: B0_x, y: B0_y, setX: setB0_x, setY: setB0_y, radio: radioB, color:
"#d81b60", border: false, label: "B0" },
    B1: { x: B1_x, y: B1_y, setX: setB1_x, setY: setB1_y, radio: radioB, color:
"#ff9aa2", border: true, label: "B1" },
  }), [A0_x, A0_y, A1_x, A1_y, B0_x, B0_y, B1_x, B1_y, radioA, radioB]);

  // Cámara Inteligente (centrado automático)
  useEffect(() => {
    // Si el usuario movió el mapa manualmente, no hacemos nada
    if (!autoCenter) return;

    const points: { x: number, y: number, r: number }[] = [];
```

```
    Object.values(robotMap).forEach(r => {
      if (r.x !== null && r.y !== null) points.push({ x: r.x, y: r.y, r: r.radio });
    });
    if (points.length === 0) return;

    let minX = Infinity, maxX = -Infinity, minY = Infinity, maxY = -Infinity;

    points.forEach(p => {
      minX = Math.min(minX, p.x - p.r); maxX = Math.max(maxX, p.x + p.r);
      minY = Math.min(minY, p.y - p.r); maxY = Math.max(maxY, p.y + p.r);
    });

    const centerX = (minX + maxX) / 2;
    const centerY = (minY + maxY) / 2;
    const padding = 100; // Margen en píxeles
    const zoomX = canvasSize.width / (maxX - minX + padding);
    const zoomY = canvasSize.height / (maxY - minY + padding);
    const targetZoom = Math.min(Math.max(Math.min(zoomX, zoomY), 0.2), 1.5);

    setOffsetX(centerX);
    setOffsetY(centerY);
    setZoom(targetZoom);

  }, [A0_x, A0_y, A1_x, A1_y, B0_x, B0_y, B1_x, B1_y, radioA, radioB, autoCenter,
canvasSize, robotMap]);

  function worldToScreen(x: number, y: number, w: number, h: number) {
    const sx = (x - offsetX) * zoom + w / 2;
    const sy = h / 2 - (y - offsetY) * zoom;
    return { sx, sy };
  }

  function screenToWorld(sx: number, sy: number, w: number, h: number) {
    const x = (sx - w / 2) / zoom + offsetX;
    const y = (h / 2 - sy) / zoom + offsetY;
    return { x, y };
  }

  // Pantalla completa
  useEffect(() => {
    const handleFullscreenChange = () => {
      const isFull = !!document.fullscreenElement;
      setIsFullscreen(isFull);
      if (!isFull) {
        setCanvasSize({ width: 600, height: 400 });
      }
    };

    document.addEventListener("fullscreenchange", handleFullscreenChange);

    return () => document.removeEventListener("fullscreenchange",
handleFullscreenChange);
  }, []);

  // Redimensionado y re-centrado
  useEffect(() => {
    const updateSize = () => {
      const container = visualizationContainerRef.current;
      if (!container) return;

      if (isFullscreen) {
        setCanvasSize({ width: window.innerWidth - 40, height: window.innerHeight -
80 });
      } else {
        const rect = container.getBoundingClientRect();
```

```
        // Si es móvil, tomamos el ancho de la ventana, si es escritorio, tomamos el
ancho del contenedor.
        const w = window.innerWidth < 1024 ? (window.innerWidth - 50) : (rect.width -
50);
        const h = window.innerWidth < 1024 ? 350 : 450;

        setCanvasSize({ width: Math.max(300, w), height: h });
      }
    };

    // Lanzar al montar
    updateSize();

    // Lanzar con un ligero delay para asegurar que el navegador ha terminado de
calcular el CSS
    const timer = setTimeout(updateSize, 100);

    window.addEventListener('resize', updateSize);
    return () => {
      clearTimeout(timer);
      window.removeEventListener('resize', updateSize);
    };
  }, [isFullscreen]);

  const toggleFullscreen = () => {
    const container = visualizationContainerRef.current as any;
    const doc = document as any;

    if (!container) return;
    if (!isFullscreen) {
      if (container.requestFullscreen) {
        container.requestFullscreen().catch((err: any) => {
          console.warn("Error al intentar fullscreen nativo:", err);
          setIsFullscreen(true);
        });
      } else if (container.webkitRequestFullscreen) {
        container.webkitRequestFullscreen();
      } else {
        setIsFullscreen(true);
      }
    } else {
      if (doc.exitFullscreen) {
        doc.exitFullscreen().catch(() => setIsFullscreen(false));
      } else if (doc.webkitExitFullscreen) {
        doc.webkitExitFullscreen();
      } else {
        setIsFullscreen(false);
      }
    }
  };

  // Verificamos si el ratón está sobre un robot (priorizando el de menor radio en
caso de solapamiento)
  const isRobotHovered = (sx: number, sy: number): 'A0' | 'A1' | 'B0' | 'B1' | null
=> {
    const keysOrdenadas = (Object.keys(robotMap) as Array<keyof typeof
robotMap>).sort((a, b) => {
      return robotMap[a].radio - robotMap[b].radio;
    });

    return keysOrdenadas.find(k => {
      const r = robotMap[k];
      if (r.x === null || r.y === null) return false;
      const { sx: rsx, sy: rsy } = worldToScreen(r.x, r.y, canvasSize.width,
canvasSize.height);
      const size = r.radio * zoom;
```

```
      const dx = sx - rsx;
      const dy = sy - rsy;
      if (forma === "cuadrada") return Math.abs(dx) <= size && Math.abs(dy) <= size;
      else if (forma === "rectangular") return Math.abs(dx) <= size * 1.5 &&
Math.abs(dy) <= size / 2;
      else return Math.sqrt(dx * dx + dy * dy) <= size;
    }) || null;
  };

  // Cuando el usuario hace clic sobre un robot
  const handleMouseDown = (e: React.MouseEvent<HTMLCanvasElement>) => {
    e.preventDefault();
    const rect = canvasRef.current!.getBoundingClientRect();
    const sx = e.clientX - rect.left;
    const sy = e.clientY - rect.top;

    const clickedRobotKey = isRobotHovered(sx, sy);
    if (clickedRobotKey) {
      const r = robotMap[clickedRobotKey];
      const worldPos = screenToWorld(sx, sy, canvasSize.width, canvasSize.height);
      if (r.x !== null && r.y !== null) {
        setDragging({ robot: clickedRobotKey, offsetX: worldPos.x - r.x, offsetY:
worldPos.y - r.y });
        if (canvasRef.current) canvasRef.current.style.cursor = "grabbing";
      }
    } else {
      setAutoCenter(false);
      isPanning.current = true;
      lastMouse.current = { x: e.clientX, y: e.clientY };
      if (canvasRef.current) canvasRef.current.style.cursor = "grabbing";
    }
  };
  //Cuando el usuario mueve el ratón (para arrastrar robots o hacer pan)
  const handleMouseMove = (e: React.MouseEvent<HTMLCanvasElement>) => {
    const rect = canvasRef.current!.getBoundingClientRect();
    const sx = e.clientX - rect.left;
    const sy = e.clientY - rect.top;
    if (!dragging) setMouseScreenPos({ sx, sy });
    if (dragging) {
      const worldPos = screenToWorld(sx, sy, canvasSize.width, canvasSize.height);
      const r = robotMap[dragging.robot];
      r.setX(Math.round(worldPos.x - dragging.offsetX));
      r.setY(Math.round(worldPos.y - dragging.offsetY));
    } else if (isPanning.current) {
      const dx = e.clientX - lastMouse.current.x;
      const dy = e.clientY - lastMouse.current.y;
      setOffsetX(prev => prev - dx / zoom);
      setOffsetY(prev => prev + dy / zoom);
      lastMouse.current = { x: e.clientX, y: e.clientY };
    }
  };

  // Cuando el usuario suelta el ratón (para soltar robots o terminar pan)
  const handleMouseUp = () => {
    isPanning.current = false;
    setDragging(null);
    if (canvasRef.current) canvasRef.current.style.cursor = "grab";
  };

  // Cuando el usuario saca el ratón del canvas (para soltar robots o terminar pan)
  const handleMouseLeave = () => {
    isPanning.current = false;
    setMouseScreenPos(null);
    setDragging(null);
    if (canvasRef.current) canvasRef.current.style.cursor = "grab";
  };
```

```
  // Funciones de Zoom
  const handleZoomIn = () => { setAutoCenter(false); setZoom(z => Math.min(z * 1.1,
8)) };
  const handleZoomOut = () => { setAutoCenter(false); setZoom(z => Math.max(z / 1.1,
0.1)) };

  // Calculo de tangentes
  const getTangentPoint = (origin: { x: number, y: number }, target: { x: number, y:
number }, radius: number, dir: number) => {
    const dx = target.x - origin.x;
    const dy = target.y - origin.y;
    const dist = Math.sqrt(dx * dx + dy * dy);

    const mathDist = Math.max(dist, radius + 0.01);

    const angleToCenter = Math.atan2(dy, dx);
    const offset = Math.asin(radius / mathDist);

    const angle = angleToCenter + (dir * offset);
    const L = Math.sqrt(mathDist * mathDist - radius * radius);

    return {
      x: origin.x + L * Math.cos(angle),
      y: origin.y + L * Math.sin(angle)
    };
  };

  // Función para iniciar la simulación (con comprobaciones y clasificación de caso)
  const handleIniciar = () => {
    if (
      A0_x === null || A0_y === null ||
      A1_x === null || A1_y === null ||
      B0_x === null || B0_y === null ||
      B1_x === null || B1_y === null
    ) {
      // Si falta alguna coordenada, mostramos un error y no iniciamos la simulación
      setPopup({
        show: true,
        message: "Missing coordinates. Please define the position for all robots.",
        type: "error"
      });
      return;
    }

    // Guardamos la foto de las posiciones exactas antes de calcular nada
    setSnapshot({ A0_x, A0_y, A1_x, A1_y, B0_x, B0_y, B1_x, B1_y });

    // Borramos los dibujos anteriores para evitar confusiones
    setShowCorridors(false);
    setPInt(null);
    setPIntLines([]);
    setDebugLines([]);
    setZoneInfo(null);

    // Función para encontrar dónde se cortan dos rectas
    const getIntersection = (p1: { x: number, y: number }, p2: { x: number, y: number
}, p3: { x: number, y: number }, p4: { x: number, y: number }) => {
      const d = (p1.x - p2.x) * (p3.y - p4.y) - (p1.y - p2.y) * (p3.x - p4.x);
      if (d === 0) return null; // Son paralelas

      const t = ((p1.x - p3.x) * (p3.y - p4.y) - (p1.y - p3.y) * (p3.x - p4.x)) / d;
      return {
        x: p1.x + t * (p2.x - p1.x),
        y: p1.y + t * (p2.y - p1.y)
      };
```

```
    };

    const getDist = (x1: number, y1: number, x2: number, y2: number) => {
      return Math.sqrt((x2 - x1) ** 2 + (y2 - y1) ** 2);
    }

    if (getDist(A0_x, A0_y, B0_x, B0_y) < (radioA + radioB)) {
      setPopup({ show: true, message: "⚠️ Error: Robots collide at START.", type:
"error" });
      return;
    }

    if (getDist(A1_x, A1_y, B1_x, B1_y) < (radioA + radioB)) {
      setPopup({ show: true, message: "⚠️ Error: Robots collide at DESTINATION.",
type: "error" });
      return;
    }

    const casoIdentificado = clasificarEscenario(
      { x: A0_x, y: A0_y },
      { x: A1_x, y: A1_y },
      { x: B0_x, y: B0_y },
      { x: B1_x, y: B1_y },
      radioA,
      radioB
    );

    setPopup({
      show: true,
      message: `Detected case: ${casoIdentificado}`,
      type: "success"
    });

    setShowCorridors(true);

    // Funncion auxiliar para determinar de qué lado de una línea está un punto
(positivo = derecha, negativo = izquierda, 0 = en la línea)
    const getSide = (A: { x: number, y: number }, B: { x: number, y: number }, P: {
x: number, y: number }) => {
      return (B.x - A.x) * (P.y - A.y) - (B.y - A.y) * (P.x - A.x);
    };

    // Función para calcular la mejor ruta de evasión alrededor de un círculo,
probando las 4 combinaciones de tangentes posibles
    const getBestEvasionPath = (start: { x: number, y: number }, end: { x: number, y:
number }, center: { x: number, y: number }, rad: number) => {
      const getT = (origin: { x: number, y: number }) => {
        const t1 = getTangentPoint(origin, center, rad, 1);
        const t2 = getTangentPoint(origin, center, rad, -1);
        const val = getSide(start, end, t1);
        return { derecha: val >= 0 ? t1 : t2, izquierda: val < 0 ? t1 : t2 };
      };

      const tStart = getT(start);
      const tEnd = getT(end);
      const paths: any[] = [];

      // Función para evaluar una combinación de tangentes y calcular la distancia
total del camino resultante
      const evalCombo = (t1: { x: number, y: number }, t2: { x: number, y: number },
isPositiveDir: boolean) => {
        const a1 = Math.atan2(t1.y - center.y, t1.x - center.x);
        let a2 = Math.atan2(t2.y - center.y, t2.x - center.x);
        let diff = a2 - a1;

        if (isPositiveDir) { if (diff < 0) diff += 2 * Math.PI; }
```

```
        else { if (diff > 0) diff -= 2 * Math.PI; }

        const dStart = Math.sqrt((t1.x - start.x) ** 2 + (t1.y - start.y) ** 2);
        const dArc = rad * Math.abs(diff);
        const dEnd = Math.sqrt((end.x - t2.x) ** 2 + (end.y - t2.y) ** 2);

        const numSegments = Math.max(6, Math.ceil(Math.abs(diff) * 5));
        const arcPoints: { x: number, y: number }[] = [];
        for (let i = 1; i < numSegments; i++) {
          const currentAngle = a1 + (diff * (i / numSegments));
          arcPoints.push({ x: center.x + rad * Math.cos(currentAngle), y: center.y +
rad * Math.sin(currentAngle) });
        }
        paths.push({ path: [start, t1, ...arcPoints, t2, end], dist: dStart + dArc +
dEnd });
      };

      evalCombo(tStart.derecha, tEnd.derecha, true);
      evalCombo(tStart.derecha, tEnd.derecha, false);
      evalCombo(tStart.izquierda, tEnd.izquierda, true);
      evalCombo(tStart.izquierda, tEnd.izquierda, false);

      paths.sort((a, b) => a.dist - b.dist);
      return paths[0];
    };

    // Función para obtener las tangentes a un círculo desde un punto de origen, y
determinar cuál es la tangente derecha e izquierda respecto a una línea de referencia
    const getTangentsToCircle = (
      origin: { x: number, y: number },
      target: { x: number, y: number },
      rad: number,
      refStart: { x: number, y: number },
      refEnd: { x: number, y: number }
    ) => {
      const t1 = getTangentPoint(origin, target, rad, 1);
      const t2 = getTangentPoint(origin, target, rad, -1);
      const val1 = getSide(refStart, refEnd, t1);
      return { derecha: val1 > 0 ? t1 : t2, izquierda: val1 < 0 ? t1 : t2 };
    };

    if (casoIdentificado === "Easy") {

      // Calculamos quién estorba a quién
      const A0_in = isRobotInCorridor({ x: A0_x!, y: A0_y! }, radioA, { x: B0_x!, y:
B0_y! }, { x: B1_x!, y: B1_y! }, radioB);
      const A1_in = isRobotInCorridor({ x: A1_x!, y: A1_y! }, radioA, { x: B0_x!, y:
B0_y! }, { x: B1_x!, y: B1_y! }, radioB);
      const B0_in = isRobotInCorridor({ x: B0_x!, y: B0_y! }, radioB, { x: A0_x!, y:
A0_y! }, { x: A1_x!, y: A1_y! }, radioA);

      let primerRobot = 'A'; // Por defecto A se mueve primero

      // Lógica de prioridad: Si A estorba al final de B o B estorba al inicio de A -
> B primero
      if (A1_in || B0_in) {
        primerRobot = 'B';
      }
      console.log(`Estrategia Easy: Mover primero ${primerRobot}`);

      const pathA_Easy = [{ x: A0_x!, y: A0_y! }, { x: A1_x!, y: A1_y! }];
      const pathB_Easy = [{ x: B0_x!, y: B0_y! }, { x: B1_x!, y: B1_y! }];

      simulationRef.current = {
        active: true, phase: `moving${primerRobot}`, t: 0, speed: 0.02,
        turns: [
```

```
          { robot: primerRobot as 'A' | 'B', path: primerRobot === 'A' ? pathA_Easy :
pathB_Easy },
          { robot: (primerRobot === 'A' ? 'B' : 'A') as 'A' | 'B', path: primerRobot
=== 'A' ? pathB_Easy : pathA_Easy }
        ],
        turnIndex: 0, currentSegment: 0,
        posA: { x: A0_x!, y: A0_y! }, posB: { x: B0_x!, y: B0_y! },
        trailA: [], trailB: []
      };

    } else if (casoIdentificado === "Nested") {
      console.log("Iniciando lógica Nested (Dual-World)...");

      let R_pos = { x: A0_x!, y: A0_y! };
      let R_dest = { x: A1_x!, y: A1_y! };
      let R_rad = radioA;

      let O_start = { x: B0_x!, y: B0_y! };
      let O_end = { x: B1_x!, y: B1_y! };
      let O_rad = radioB;

      const B_is_nested = isRobotInCorridor({ x: B0_x!, y: B0_y! }, radioB, { x:
A0_x!, y: A0_y! }, { x: A1_x!, y: A1_y! }, radioA);
      if (B_is_nested) {
        R_pos = { x: B0_x!, y: B0_y! };
        R_dest = { x: B1_x!, y: B1_y! };
        R_rad = radioB;
        O_start = { x: A0_x!, y: A0_y! };
        O_end = { x: A1_x!, y: A1_y! };
        O_rad = radioA;
      }

      const robotMovingId = B_is_nested ? 'B' : 'A';
      const robotObstacleId = B_is_nested ? 'A' : 'B';
      const radioSeguridad = radioA + radioB;

      // Función para evaluar en qué "mundo" estamos (CW o CCW) y determinar la zona
resultante, aplicando las reglas de tangentes y lados
      const evaluarMundo = (esClockwise: boolean) => {
        const tStart_A = getTangentPoint(R_pos, O_start, radioSeguridad, 1);
        const tStart_B = getTangentPoint(R_pos, O_start, radioSeguridad, -1);
        const tEnd_A = getTangentPoint(R_pos, O_end, radioSeguridad, 1);
        const tEnd_B = getTangentPoint(R_pos, O_end, radioSeguridad, -1);

        // Invertimos los signos matemáticos si exploramos el mundo CW
        const val_B0 = getSide(R_pos, O_start, tStart_A);
        const t_B0_chosen = esClockwise ? (val_B0 > 0 ? tStart_A : tStart_B) :
(val_B0 < 0 ? tStart_A : tStart_B);

        const val_B1 = getSide(R_pos, O_end, tEnd_A);
        const t_B1_chosen = esClockwise ? (val_B1 < 0 ? tEnd_A : tEnd_B) : (val_B1 >
0 ? tEnd_A : tEnd_B);

        // Determinar Zona en este mundo
        const valStart = getSide(R_pos, t_B0_chosen, R_dest);
        const valEnd = getSide(R_pos, t_B1_chosen, R_dest);
        const leftStart = valStart > 0;
        const leftEnd = valEnd > 0;

        let zona = "Unknown";

        if (esClockwise) {
          //  MUNDO CW
          if (!leftEnd && leftStart) { zona = "ZONE 1"; }
          else if (leftEnd && !leftStart) { zona = "ZONE 2"; }
          else if (!leftEnd && !leftStart) { zona = "ZONE 4"; }
```

```
          else if (leftEnd && leftStart) { zona = "ZONE 3"; }
        } else {
          //  MUNDO CCW (Las condiciones se invierten)
          if (leftEnd && !leftStart) { zona = "ZONE 1"; }
          else if (!leftEnd && leftStart) { zona = "ZONE 2"; }
          else if (leftEnd && leftStart) { zona = "ZONE 4"; }
          else if (!leftEnd && !leftStart) { zona = "ZONE 3"; }
        }

        let seq: ('A' | 'B')[] = [];
        let pathO: { x: number, y: number }[] = [];
        let pathR: { x: number, y: number }[] = [];
        let pInt = null;
        let evasionResult: any = null;
        let pIntLines: any[] = [];

        if (zona === "ZONE 1") {
          seq = [robotMovingId as 'A' | 'B', robotObstacleId as 'A' | 'B'];
          pathR = [R_pos, R_dest];
          evasionResult = getBestEvasionPath(O_start, O_end, R_dest, radioSeguridad);
          pathO = evasionResult.path;

        } else if (zona === "ZONE 2") {
          seq = [robotObstacleId as 'A' | 'B', robotMovingId as 'A' | 'B'];
          pathR = [R_pos, R_dest];
          evasionResult = getBestEvasionPath(O_start, O_end, R_pos, radioSeguridad);
          pathO = evasionResult.path;

        } else {
          // Si el mundo está invertido (CW), cruzamos usando las tangentes opuestas
          const ladoTangente = esClockwise ? 'izquierda' : 'derecha';
          let l1_p2, l2_p2;

          if (zona === "ZONE 3") {
            const tan_A0_B1 = getTangentsToCircle(R_pos, O_end, radioSeguridad,
O_start, O_end);
            const tan_A1_B0 = getTangentsToCircle(R_dest, O_start, radioSeguridad,
O_start, O_end);
            pInt = getIntersection(R_pos, tan_A0_B1[ladoTangente], R_dest,
tan_A1_B0[ladoTangente]);
            l1_p2 = tan_A0_B1[ladoTangente]; l2_p2 = tan_A1_B0[ladoTangente];
          } else {
            const tan_A0_B0 = getTangentsToCircle(R_pos, O_start, radioSeguridad,
O_start, O_end);
            const tan_A1_B1 = getTangentsToCircle(R_dest, O_end, radioSeguridad,
O_start, O_end);
            pInt = getIntersection(R_pos, tan_A0_B0[ladoTangente], R_dest,
tan_A1_B1[ladoTangente]);
            l1_p2 = tan_A0_B0[ladoTangente]; l2_p2 = tan_A1_B1[ladoTangente];

            // Fallback de seguridad
            if (!pInt) {
              const opuesto = esClockwise ? 'derecha' : 'izquierda';
              pInt = getIntersection(R_pos, tan_A0_B0[opuesto], R_dest,
tan_A1_B1[opuesto]);
              l1_p2 = tan_A0_B0[opuesto]; l2_p2 = tan_A1_B1[opuesto];
            }
          }

          if (!pInt || !l1_p2 || !l2_p2) {
            pInt = { x: (R_pos.x + R_dest.x) / 2, y: (R_pos.y + R_dest.y) / 2 };
            l1_p2 = R_dest; l2_p2 = R_pos;
          }

          pIntLines = [{ p1: R_pos, p2: l1_p2 }, { p1: R_dest, p2: l2_p2 }];
          seq = [robotMovingId as 'A' | 'B', robotObstacleId as 'A' | 'B'];
```

```
          pathR = [R_pos, pInt, R_dest];
          evasionResult = getBestEvasionPath(O_start, O_end, pInt, radioSeguridad);
          pathO = evasionResult.path;
        }

        // Calcular distancia total del movimiento del anidado
        let distR = 0;
        for (let i = 0; i < pathR.length - 1; i++) {
          distR += Math.sqrt((pathR[i + 1].x - pathR[i].x) ** 2 + (pathR[i + 1].y -
pathR[i].y) ** 2);
        }

        // Distancia total de ambos robots (Evasión O + Trayectoria R)
        const totalDistance = evasionResult.dist + distR;

        // Construir los turnos
        let turns: any[] = [];
        if (zona === "ZONE 3" || zona === "ZONE 4") {
          turns = [
            { robot: robotMovingId, path: [R_pos, pInt!] },
            { robot: robotObstacleId, path: pathO },
            { robot: robotMovingId, path: [pInt!, R_dest] }
          ];
        } else {
          turns = [
            { robot: seq[0], path: seq[0] === robotMovingId ? pathR : pathO },
            { robot: seq[1], path: seq[1] === robotMovingId ? pathR : pathO }
          ];
        }

        return {
          esClockwise, zona, totalDistance, turns, pathR, pathO, pInt, pIntLines,
          t_B0_chosen, t_B1_chosen
        };
      };

      // Evaluamos ambos mundos y nos quedamos con el más corto
      const mundoCCW = evaluarMundo(false);
      const mundoCW = evaluarMundo(true);

      console.group("Competición de Mundos (Nested)");
      console.log(`Mundo CCW (${mundoCCW.zona}): ${mundoCCW.totalDistance.toFixed(2)}
px`);
      console.log(`Mundo CW  (${mundoCW.zona}): ${mundoCW.totalDistance.toFixed(2)}
px`);

      const ganador = mundoCW.totalDistance < mundoCCW.totalDistance ? mundoCW :
mundoCCW;

      console.log(`Ganador: Mundo ${ganador.esClockwise ? 'Clockwise' : 'Counter-
Clockwise'} (${ganador.zona})`);
      console.groupEnd();

      // Aplicar el resultado ganador a la simulación
      setPopup({ show: true, message: `Nested Case: ${ganador.zona} (Via
${ganador.esClockwise ? 'CW' : 'CCW'})`, type: "success" });

      if (ganador.pInt) {
        setPInt(ganador.pInt);
        setPIntLines(ganador.pIntLines);
      } else {
        setPInt(null);
        setPIntLines([]);
      }
```

```
      // Variables para dibujar las líneas de tangente (con un largo fijo para
visualización)
      const maxLen = 25000;

      // TANGENTE DE INICIO
      let dx_s = ganador.t_B0_chosen.x - R_pos.x;
      let dy_s = ganador.t_B0_chosen.y - R_pos.y;
      let len_s = Math.sqrt(dx_s * dx_s + dy_s * dy_s);

      // Si la distancia es casi 0 (está en el borde), usamos la tangente
perpendicular
      if (len_s < 0.1) {
        dx_s = -(O_start.y - R_pos.y);
        dy_s = (O_start.x - R_pos.x);
        len_s = Math.sqrt(dx_s * dx_s + dy_s * dy_s) || 1;
      }

      // TANGENTE DE DESTINO
      let dx_e = ganador.t_B1_chosen.x - R_pos.x;
      let dy_e = ganador.t_B1_chosen.y - R_pos.y;
      let len_e = Math.sqrt(dx_e * dx_e + dy_e * dy_e);

      // Si la distancia es casi 0 (está en el borde), usamos la tangente
perpendicular
      if (len_e < 0.1) {
        dx_e = -(O_end.y - R_pos.y);
        dy_e = (O_end.x - R_pos.x);
        len_e = Math.sqrt(dx_e * dx_e + dy_e * dy_e) || 1;
      }

      const vStart_Chosen = {
        dx: (dx_s / len_s) * maxLen,
        dy: (dy_s / len_s) * maxLen
      };
      const vEnd_Chosen = {
        dx: (dx_e / len_e) * maxLen,
        dy: (dy_e / len_e) * maxLen
      };
      setZoneInfo({
        R_pos: R_pos,
        p1: { x: R_pos.x + vStart_Chosen.dx, y: R_pos.y + vStart_Chosen.dy },
        p2: { x: R_pos.x - vStart_Chosen.dx, y: R_pos.y - vStart_Chosen.dy },
        p3: { x: R_pos.x + vEnd_Chosen.dx, y: R_pos.y + vEnd_Chosen.dy },
        p4: { x: R_pos.x - vEnd_Chosen.dx, y: R_pos.y - vEnd_Chosen.dy },
        clip: { start: O_start, end: O_end, width: radioSeguridad }, detected:
ganador.zona
      } as any);

      simulationRef.current = {
        active: true, phase: `moving${ganador.turns[0].robot}`, t: 0, speed: 0.02,
        turns: ganador.turns, turnIndex: 0, currentSegment: 0,
        posA: { x: A0_x!, y: A0_y! }, posB: { x: B0_x!, y: B0_y! },
        trailA: [], trailB: []
      };
    } else if (casoIdentificado === "Multi-Obstruction") {
      console.log("Iniciando lógica Multi...");

      // Evaluamos la tabla de verdad (¿Quién está dentro de quién?)
      const a0_in_b = isRobotInCorridor({ x: A0_x!, y: A0_y! }, radioA, { x: B0_x!,
y: B0_y! }, { x: B1_x!, y: B1_y! }, radioB);
      const a1_in_b = isRobotInCorridor({ x: A1_x!, y: A1_y! }, radioA, { x: B0_x!,
y: B0_y! }, { x: B1_x!, y: B1_y! }, radioB);
      const b0_in_a = isRobotInCorridor({ x: B0_x!, y: B0_y! }, radioB, { x: A0_x!,
y: A0_y! }, { x: A1_x!, y: A1_y! }, radioA);
      const b1_in_a = isRobotInCorridor({ x: B1_x!, y: B1_y! }, radioB, { x: A0_x!,
y: A0_y! }, { x: A1_x!, y: A1_y! }, radioA);
```

```
      const radioSeguridad = radioA + radioB;

      // Lógica para aplicar las reglas de tangentes y lados según si estamos en el
caso normal o el caso invertido,
      const checkNormal = (r0: boolean, r1: boolean, o0: boolean, o1: boolean) => {
        return (r0 && !r1 && o0 && !o1) || (r0 && !r1 && o0 && o1) || (r0 && r1 &&
!o0 && o1) || (r0 && r1 && o0 && o1);
      };

      const checkReversed = (r0: boolean, r1: boolean, o0: boolean, o1: boolean) => {
        return (!r0 && r1 && !o0 && o1) || (!r0 && r1 && o0 && o1) || (r0 && r1 && o0
&& !o1);
      };

      const A_Normal = checkNormal(a0_in_b, a1_in_b, b0_in_a, b1_in_a);
      const A_Reversed = checkReversed(a0_in_b, a1_in_b, b0_in_a, b1_in_a);

      const B_Normal = checkNormal(b0_in_a, b1_in_a, a0_in_b, a1_in_b);
      const B_Reversed = checkReversed(b0_in_a, b1_in_a, a0_in_b, a1_in_b);

      // Funcion para comprobar si el destino del robot anidado está fuera del cono
      const checkFueraDelCono = (origen: { x: number, y: number }, destino: { x:
number, y: number }, obsInicio: { x: number, y: number }, obsFin: { x: number, y:
number }) => {
        const t1 = getTangentPoint(origen, obsInicio, radioSeguridad, 1);
        const t2 = getTangentPoint(origen, obsInicio, radioSeguridad, -1);
        const val_t1 = getSide(obsInicio, obsFin, t1);
        const tRight = val_t1 >= 0 ? t1 : t2;
        const tLeft = val_t1 < 0 ? t1 : t2;

        const sideL1 = getSide(origen, tRight, destino);
        const sideL5 = getSide(origen, tLeft, destino);
        return (sideL1 < 0 || sideL5 >= 0);
      };

      const createRay = (start: { x: number, y: number }, through: { x: number, y:
number }) => {
        const vx = through.x - start.x;
        const vy = through.y - start.y;
        const vlen = Math.sqrt(vx * vx + vy * vy) || 1;
        const maxLen = 100000;
        return { start, end: { x: through.x + (vx / vlen) * maxLen, y: through.y +
(vy / vlen) * maxLen } };
      };


      const getRayCircleIntersection = (start: any, through: any, center: any,
radius: number) => {
        const vx = through.x - start.x, vy = through.y - start.y;
        const vLen = Math.sqrt(vx * vx + vy * vy) || 1;
        const ux = vx / vLen, uy = vy / vLen;
        const wX = start.x - center.x, wY = start.y - center.y;
        const b = 2 * (ux * wX + uy * wY);
        const c = wX * wX + wY * wY - radius * radius;
        const disc = b * b - 4 * c;

        if (disc >= 0) {
          const t_sol1 = (-b - Math.sqrt(disc)) / 2;
          const t_sol2 = (-b + Math.sqrt(disc)) / 2;
          const p1 = { x: start.x + ux * t_sol1, y: start.y + uy * t_sol1 };
          const p2 = { x: start.x + ux * t_sol2, y: start.y + uy * t_sol2 };

          const d1 = (p1.x - start.x) ** 2 + (p1.y - start.y) ** 2;
          const d2 = (p2.x - start.x) ** 2 + (p2.y - start.y) ** 2;
          return d1 < d2 ? p1 : p2;
```

```
        }
        return null;
      };

      const evaluarCurvaReal = (destino: any, origen: any, tInicio: any, tFin: any,
radio: number, pasos = 15) => {
        const a1 = Math.atan2(tInicio.y - origen.y, tInicio.x - origen.x);
        let a2 = Math.atan2(tFin.y - origen.y, tFin.x - origen.x);
        let diff = a2 - a1;
        if (diff > Math.PI) diff -= 2 * Math.PI;
        if (diff < -Math.PI) diff += 2 * Math.PI;

        const puntosCurva = [];
        for (let i = 0; i <= pasos; i++) {
          const ang = a1 + diff * (i / pasos);
          puntosCurva.push({ x: origen.x + radio * Math.cos(ang), y: origen.y + radio
* Math.sin(ang) });
        }

        for (let i = 0; i < puntosCurva.length - 1; i++) {
          const p1 = puntosCurva[i], p2 = puntosCurva[i + 1];
          if (getSide(p1, p2, origen) * getSide(p1, p2, destino) < 0) return "FUERA";
        }
        return "DENTRO";
      };

      // Función principal para evaluar cada candidato (A o B) y determinar la lógica
normal o inversa
      const evaluarCandidato = (candidato: 'A' | 'B', isReversedInit: boolean) => {
        let R_pos = candidato === 'A' ? { x: A0_x!, y: A0_y! } : { x: B0_x!, y: B0_y!
};
        let R_dest = candidato === 'A' ? { x: A1_x!, y: A1_y! } : { x: B1_x!, y:
B1_y! };
        let O_start = candidato === 'A' ? { x: B0_x!, y: B0_y! } : { x: A0_x!, y:
A0_y! };
        let O_end = candidato === 'A' ? { x: B1_x!, y: B1_y! } : { x: A1_x!, y: A1_y!
};

        let isReversed = isReversedInit;
        if (isReversed) {
          const tempR = R_pos; R_pos = R_dest; R_dest = tempR;
          const tempO = O_start; O_start = O_end; O_end = tempO;
        }

        if (!isReversed && checkFueraDelCono(R_pos, R_dest, O_start, O_end)) {
          isReversed = true;
          const tempR = R_pos; R_pos = R_dest; R_dest = tempR;
          const tempO = O_start; O_start = O_end; O_end = tempO;
        }

        const evaluarMundoMulti = (esClockwise: boolean) => {
          const t1 = getTangentPoint(R_pos, O_start, radioSeguridad, 1);
          const t2 = getTangentPoint(R_pos, O_start, radioSeguridad, -1);
          const val_t1 = getSide(O_start, O_end, t1);
          const tRight = val_t1 >= 0 ? t1 : t2;
          const tLeft = val_t1 < 0 ? t1 : t2;

          const tMain = esClockwise ? tLeft : tRight;
          const tOpp = esClockwise ? tRight : tLeft;
          const mult = esClockwise ? -1 : 1;

          const dx = O_end.x - O_start.x;
          const dy = O_end.y - O_start.y;
          const len = Math.sqrt(dx * dx + dy * dy) || 1;
          const n1 = { x: -dy / len, y: dx / len };
          const n2 = { x: dy / len, y: -dx / len };
```

```
          const p_test = { x: O_start.x + n1.x * 10, y: O_start.y + n1.y * 10 };
          const nRight = getSide(O_start, O_end, p_test) > 0 ? n1 : n2;
          const nLeft = getSide(O_start, O_end, p_test) < 0 ? n1 : n2;
          const nChosen = esClockwise ? nLeft : nRight;

          const tB0_chosen = { x: O_start.x + nChosen.x * radioSeguridad, y:
O_start.y + nChosen.y * radioSeguridad };
          const tB1_chosen = { x: O_end.x + nChosen.x * radioSeguridad, y: O_end.y +
nChosen.y * radioSeguridad };

          let ptT = R_pos;
          const distCenters = Math.sqrt((O_end.x - O_start.x) ** 2 + (O_end.y -
O_start.y) ** 2) || 1;
          const distRtoOend = Math.sqrt((R_pos.x - O_end.x) ** 2 + (R_pos.y -
O_end.y) ** 2);
          let foundArcIntersection = false;

          if (distCenters <= 2 * radioSeguridad) {
            const midX = (O_start.x + O_end.x) / 2;
            const midY = (O_start.y + O_end.y) / 2;
            const h = Math.sqrt(Math.max(0, radioSeguridad ** 2 - (distCenters / 2)
** 2));
            const udx = (O_end.x - O_start.x) / distCenters;
            const udy = (O_end.y - O_start.y) / distCenters;

            const i1 = { x: midX - h * udy, y: midY + h * udx };
            const i2 = { x: midX + h * udy, y: midY - h * udx };

            const I_right = getSide(O_start, O_end, i1) >= 0 ? i1 : i2;
            const I_left = getSide(O_start, O_end, i1) < 0 ? i1 : i2;
            const I_chosen = esClockwise ? I_left : I_right;

            const side1 = getSide(O_start, tMain, I_chosen) * getSide(O_start, tMain,
tB0_chosen);
            const side2 = getSide(O_start, tB0_chosen, I_chosen) * getSide(O_start,
tB0_chosen, tMain);

            if (side1 >= 0 && side2 >= 0) {
              ptT = I_chosen;
              foundArcIntersection = true;
            }
          }

          if (!foundArcIntersection && distRtoOend <= radioSeguridad) {
            const vx = tMain.x - R_pos.x;
            const vy = tMain.y - R_pos.y;
            const vLen = Math.sqrt(vx * vx + vy * vy) || 1;
            const ux = vx / vLen, uy = vy / vLen;
            const wX = R_pos.x - O_end.x, wY = R_pos.y - O_end.y;
            const b = 2 * (ux * wX + uy * wY);
            const c = wX * wX + wY * wY - radioSeguridad ** 2;
            const disc = b * b - 4 * c;

            if (disc >= 0) {
              const t_sol = (-b + Math.sqrt(disc)) / 2;
              ptT = { x: R_pos.x + ux * t_sol, y: R_pos.y + uy * t_sol };
            }
          }

          const line1 = createRay(R_pos, tMain);
          const line2 = createRay(tB1_chosen, tB0_chosen);
          const line3 = createRay(tB1_chosen, tMain);
          const line5 = createRay(R_pos, tOpp);

          const dir_L5 = { x: tOpp.x - R_pos.x, y: tOpp.y - R_pos.y };
```

```
          let L4_dir = { x: 0, y: 0 };
          const distTtoEnd = Math.sqrt((ptT.x - O_end.x) ** 2 + (ptT.y - O_end.y) **
2);

          if (distTtoEnd <= radioSeguridad + 0.5) {
            const rx = ptT.x - O_end.x, ry = ptT.y - O_end.y;
            const tx1 = -ry, ty1 = rx;
            const tx2 = ry, ty2 = -rx;
            L4_dir = (tx1 * dir_L5.x + ty1 * dir_L5.y > tx2 * dir_L5.x + ty2 *
dir_L5.y) ? { x: tx1, y: ty1 } : { x: tx2, y: ty2 };
          } else {
            const t1_end = getTangentPoint(ptT, O_end, radioSeguridad, 1);
            const t2_end = getTangentPoint(ptT, O_end, radioSeguridad, -1);
            const val_t1_end = getSide(O_start, O_end, t1_end);
            const tEnd_Right = val_t1_end >= 0 ? t1_end : t2_end;
            const tEnd_Left = val_t1_end < 0 ? t1_end : t2_end;

            const tEnd_chosen = esClockwise ? tEnd_Left : tEnd_Right;
            L4_dir = { x: tEnd_chosen.x - ptT.x, y: tEnd_chosen.y - ptT.y };

            if (L4_dir.x * dir_L5.x + L4_dir.y * dir_L5.y < 0) {
              L4_dir.x = -L4_dir.x; L4_dir.y = -L4_dir.y;
            }

          }

          const lenL4 = Math.sqrt(L4_dir.x ** 2 + L4_dir.y ** 2) || 1;
          const inf_L4 = { x: ptT.x + (L4_dir.x / lenL4) * 100000, y: ptT.y +
(L4_dir.y / lenL4) * 100000 };
          const line4 = { start: ptT, end: inf_L4 };

          const sideL1 = getSide(line1.start, line1.end, R_dest) * mult;
          const sideL2 = getSide(line2.start, line2.end, R_dest) * mult;
          const sideL3 = getSide(line3.start, line3.end, R_dest) * mult;
          const sideL4 = getSide(line4.start, line4.end, R_dest) * mult;
          const sideL5 = getSide(line5.start, line5.end, R_dest) * mult;

          const estadoCurva = evaluarCurvaReal(R_dest, O_start, tMain, tB0_chosen,
radioSeguridad);

          let zonaDetectada: string | null = null;
          const EPS = 1e-8; // Margen de tolerancia para errores de decimales en JS

          // Aplicamos las reglas de tangentes y lados para determinar la zona con
prioridad a la zona 4,
          // luego la zona 2 (con su curva), luego la zona 3 y por descarte la zona 1
          if (sideL1 < -EPS || sideL5 > EPS) {
            zonaDetectada = "NULL (Out of Cone)";// Comprobamos si está fuera del
cono
          }
          else if (sideL4 >= -EPS && sideL5 <= EPS) {
            zonaDetectada = "ZONE 4";
          }
          else if (sideL2 >= -EPS && sideL3 < EPS && estadoCurva === "DENTRO") {
            zonaDetectada = "ZONE 2";
          }
          else if (sideL3 >= -EPS && sideL4 < EPS) {
            zonaDetectada = "ZONE 3";
          }
          else {
            zonaDetectada = "ZONE 1";
          }

          let A_int = { x: R_dest.x, y: R_dest.y };
          let lineasInterseccion: any[] = [];
```

```
          const tan_A1_B1 = getTangentsToCircle(R_dest, O_end, radioSeguridad,
O_start, O_end);
          const t_A1_B1_chosen = esClockwise ? tan_A1_B1.izquierda :
tan_A1_B1.derecha;

          if (zonaDetectada === "ZONE 1" || zonaDetectada === "NULL (Out of Cone)" ||
zonaDetectada === null) {
            A_int = { x: R_dest.x, y: R_dest.y };
          } else if (zonaDetectada === "ZONE 4") {
            A_int = { x: ptT.x, y: ptT.y };

            // L1 infinita en ambos sentidos
            const dx1 = tMain.x - R_pos.x;
            const dy1 = tMain.y - R_pos.y;
            const len1 = Math.sqrt(dx1 * dx1 + dy1 * dy1) || 1;
            const u1x = dx1 / len1; const u1y = dy1 / len1;
            const inf_L1_A = { x: R_pos.x - u1x * 1000000, y: R_pos.y - u1y * 1000000
};
            const inf_L1_B = { x: R_pos.x + u1x * 1000000, y: R_pos.y + u1y * 1000000
};

            // L4 infinita en ambos sentidos
            const len4 = Math.sqrt(L4_dir.x * L4_dir.x + L4_dir.y * L4_dir.y) || 1;
            const u4x = L4_dir.x / len4; const u4y = L4_dir.y / len4;
            const inf_L4_A = { x: ptT.x - u4x * 1000000, y: ptT.y - u4y * 1000000 };
            const inf_L4_B = { x: ptT.x + u4x * 1000000, y: ptT.y + u4y * 1000000 };

          } else if (zonaDetectada === "ZONE 2") {
            const intersection = getRayCircleIntersection(t_A1_B1_chosen, R_dest,
O_start, radioSeguridad);
            if (intersection) A_int = intersection;

            // Tangente infinita en ambos sentidos
            const dx = R_pos.x - t_A1_B1_chosen.x;
            const dy = R_pos.y - t_A1_B1_chosen.y;
            const len = Math.sqrt(dx * dx + dy * dy) || 1;
            const ux = dx / len; const uy = dy / len;
            const inf_2_A = { x: t_A1_B1_chosen.x - ux * 1000000, y: t_A1_B1_chosen.y
- uy * 1000000 };
            const inf_2_B = { x: t_A1_B1_chosen.x + ux * 1000000, y: t_A1_B1_chosen.y
+ uy * 1000000 };

            lineasInterseccion = [{ p1: inf_2_A, p2: inf_2_B }];
          } else if (zonaDetectada === "ZONE 3") {
            const inter = getIntersection(R_pos, tMain, R_dest, t_A1_B1_chosen);
            if (inter) A_int = inter;

            // L1 infinita en ambos sentidos
            const dx1 = tMain.x - R_pos.x;
            const dy1 = tMain.y - R_pos.y;
            const len1 = Math.sqrt(dx1 * dx1 + dy1 * dy1) || 1;
            const u1x = dx1 / len1; const u1y = dy1 / len1;
            const inf_L1_A = { x: R_pos.x - u1x * 1000000, y: R_pos.y - u1y * 1000000
};
            const inf_L1_B = { x: R_pos.x + u1x * 1000000, y: R_pos.y + u1y * 1000000
};

            // Tangente de destino infinita en ambos sentidos
            const dx_dest = t_A1_B1_chosen.x - R_dest.x;
            const dy_dest = t_A1_B1_chosen.y - R_dest.y;
            const len_dest = Math.sqrt(dx_dest * dx_dest + dy_dest * dy_dest) || 1;
            const udx = dx_dest / len_dest; const udy = dy_dest / len_dest;
            const inf_dest_A = { x: R_dest.x - udx * 1000000, y: R_dest.y - udy *
1000000 };
            const inf_dest_B = { x: R_dest.x + udx * 1000000, y: R_dest.y + udy *
1000000 };
```

```
            lineasInterseccion = [
              { p1: inf_L1_A, p2: inf_L1_B },
              { p1: inf_dest_A, p2: inf_dest_B }
            ];
          }


          const pR0 = candidato === 'A' ? { x: A0_x!, y: A0_y! } : { x: B0_x!, y:
B0_y! };
          const pR1 = candidato === 'A' ? { x: A1_x!, y: A1_y! } : { x: B1_x!, y:
B1_y! };
          const pO0 = candidato === 'A' ? { x: B0_x!, y: B0_y! } : { x: A0_x!, y:
A0_y! };
          const pO1 = candidato === 'A' ? { x: B1_x!, y: B1_y! } : { x: A1_x!, y:
A1_y! };
          const id_R = candidato;
          const id_O = candidato === 'A' ? 'B' : 'A';

          let distTramo1 = Math.sqrt((A_int.x - pR0.x) ** 2 + (A_int.y - pR0.y) **
2);
          let pathR_Tramo1 = [pR0, A_int];
          if (isRobotInCorridor(pO0, radioSeguridad - 0.5, pR0, A_int, 0)) {
            const evasion = getBestEvasionPath(pR0, A_int, pO0, radioSeguridad);
            pathR_Tramo1 = evasion.path;
            distTramo1 = evasion.dist;
          }

          let distTramo2 = Math.sqrt((pO1.x - pO0.x) ** 2 + (pO1.y - pO0.y) ** 2);
          let pathO_Tramo2 = [pO0, pO1];
          if (isRobotInCorridor(A_int, radioSeguridad - 0.5, pO0, pO1, 0)) {
            const evasion = getBestEvasionPath(pO0, pO1, A_int, radioSeguridad);
            pathO_Tramo2 = evasion.path;
            distTramo2 = evasion.dist;
          }

          let distTramo3 = Math.sqrt((pR1.x - A_int.x) ** 2 + (pR1.y - A_int.y) **
2);
          let pathR_Tramo3 = [A_int, pR1];
          if (isRobotInCorridor(pO1, radioSeguridad - 0.5, A_int, pR1, 0)) {
            const evasion = getBestEvasionPath(A_int, pR1, pO1, radioSeguridad);
            pathR_Tramo3 = evasion.path;
            distTramo3 = evasion.dist;
          }

          const turns = [
            { robot: id_R, path: pathR_Tramo1 },
            { robot: id_O, path: pathO_Tramo2 },
            { robot: id_R, path: pathR_Tramo3 }
          ];

          const totalDistance = distTramo1 + distTramo2 + distTramo3;

          return {
            esClockwise, zonaDetectada, A_int, lineasInterseccion, turns,
totalDistance,
            tMain, tOpp, tB0_chosen, tB1_chosen, ptT, line1, line2, line3, line4,
line5,
            anidado: candidato, isReversed, R_pos, O_start, O_end
          };
        };

        const mundoCCW = evaluarMundoMulti(false);
        const mundoCW = evaluarMundoMulti(true);
```

```
        console.log(`   -> Robot ${candidato} Anidado (Vía CW)  |
${mundoCW.zonaDetectada} | Dist: ${mundoCW.totalDistance.toFixed(2)} px`);
        console.log(`   -> Robot ${candidato} Anidado (Vía CCW) |
${mundoCCW.zonaDetectada} | Dist: ${mundoCCW.totalDistance.toFixed(2)} px`);

        return mundoCW.totalDistance < mundoCCW.totalDistance ? mundoCW : mundoCCW;
      };

      console.group(" Competición de Candidatos (Multi)");
      let resultados = [];

      if (A_Normal || A_Reversed) {
        const resA = evaluarCandidato('A', A_Reversed);
        console.log(`Candidato A (Vía ${resA.esClockwise ? 'CW' : 'CCW'},
${resA.zonaDetectada}): ${resA.totalDistance.toFixed(2)} px`);
        resultados.push(resA);
      }

      if (B_Normal || B_Reversed) {
        const resB = evaluarCandidato('B', B_Reversed);
        console.log(`Candidato B (Vía ${resB.esClockwise ? 'CW' : 'CCW'},
${resB.zonaDetectada}): ${resB.totalDistance.toFixed(2)} px`);
        resultados.push(resB);
      }

      // Fallback de seguridad (por si el radar principal falla, forzamos A por
defecto)
      if (resultados.length === 0) resultados.push(evaluarCandidato('A', false));

      // Ordenar resultados para sacar el mejor (el de menor distancia)
      resultados.sort((a, b) => a.totalDistance - b.totalDistance);
      const ganador = resultados[0];

      console.log(` Ganador Absoluto: Robot ${ganador.anidado}`);
      console.groupEnd();

      // Variables para el renderizado
      const R_pos = ganador.R_pos;
      const O_start = ganador.O_start;
      const O_end = ganador.O_end;
      const direccionReal = (ganador.esClockwise !== ganador.isReversed) ? 'CW' :
'CCW';


      // Aplicamos el resultado ganador a la interfaz y a la simulación
      setPopup({
        show: true,
        message: `Multi-Obstruction Case: Robot ${ganador.anidado} is nested |
${ganador.zonaDetectada} (Via ${direccionReal})`,
        type: "success"
      });

      setPInt(ganador.A_int);
      setPIntLines(ganador.lineasInterseccion);

      setZoneInfo({
        type: 'multi',
        R_pos, tB0: ganador.tB0_chosen, tB1: ganador.tB1_chosen, p00: ganador.tMain,
p_BotTan: ganador.tOpp,
        ptT: ganador.ptT,
        O_start, O_end, radioSeguridad,
        clip: { start: O_start, end: O_end, width: radioSeguridad },
        inf_L1: ganador.line1.end, inf_L2: ganador.line2.end, inf_L3:
ganador.line3.end,
        inf_L4: ganador.line4.end, inf_L5: ganador.line5.end,
        detected: ganador.zonaDetectada
```

```
      } as any);

      setDebugLines([]);

      // Inicializamos la simulación con los turns ganadores
      simulationRef.current = {
        active: true,
        phase: `moving${ganador.turns[0].robot}`,
        t: 0,
        speed: 0.02,
        turns: ganador.turns as any,
        turnIndex: 0,
        currentSegment: 0,
        posA: { x: A0_x!, y: A0_y! },
        posB: { x: B0_x!, y: B0_y! },
        trailA: [], trailB: []
      };
    }
  };

  // Función para reiniciar la simulación y restaurar las posiciones iniciales
  const handleReiniciar = () => {

    // Restauramos las posiciones si tenemos una foto guardada
    if (snapshot) {
      setA0_x(snapshot.A0_x); setA0_y(snapshot.A0_y);
      setA1_x(snapshot.A1_x); setA1_y(snapshot.A1_y);
      setB0_x(snapshot.B0_x); setB0_y(snapshot.B0_y);
      setB1_x(snapshot.B1_x); setB1_y(snapshot.B1_y);
    }

    // Frenamos el motor de simulación en seco
    simulationRef.current = {
      active: false, phase: 'idle', t: 0, speed: 0.02,
      turns: [], turnIndex: 0, currentSegment: 0,
      posA: { x: 0, y: 0 }, posB: { x: 0, y: 0 },
      trailA: [], trailB: []
    };
  };

  // Función para imprimir los titulos de los Robots y P_int
  const drawSmartText = (ctx: CanvasRenderingContext2D, label: string, x: number, y:
number, size: number) => {
    ctx.font = `bold ${size}px Arial`;

    if (label.includes('_') || (label.length > 1 && /\d/.test(label))) {
      const parts = label.split('_');
      const main = parts[0].replace(/\d/g, '');
      const sub = parts[1] || label.match(/\d/)?.[0] || "";

      ctx.fillText(main, x, y);
      const mainWidth = ctx.measureText(main).width;
      ctx.font = `bold ${size * 0.7}px Arial`;
      ctx.fillText(sub, x + mainWidth, y + (size * 0.3));
    } else {
      ctx.fillText(label, x, y);
    }
  };

  // BUCLE UNIFICADO: FÍSICA + RENDERIZADO
  useEffect(() => {
    let animationFrameId: number;

    const loop = (time: number) => {
      const sim = simulationRef.current;
```

```
      // LÓGICA DE MOVIMIENTO (FÍSICA)
      if (sim.active) {
        const currentTurn = sim.turns[sim.turnIndex];
        const path = currentTurn.path;

        const pStartInfo = path[sim.currentSegment];
        const pEndInfo = path[sim.currentSegment + 1];
        const dist = Math.sqrt((pEndInfo.x - pStartInfo.x) ** 2 + (pEndInfo.y -
pStartInfo.y) ** 2);

        const VELOCIDAD_GLOBAL = 5;
        sim.speed = dist > 0 ? (VELOCIDAD_GLOBAL / dist) : 1;
        sim.t += sim.speed;

        if (sim.t >= 1) {
          sim.t = 0;
          sim.currentSegment++;

          const exactPos = path[sim.currentSegment];
          if (currentTurn.robot === 'A') sim.posA = exactPos;
          else sim.posB = exactPos;

          if (sim.currentSegment >= path.length - 1) {
            sim.currentSegment = 0;
            sim.turnIndex++;

            if (sim.turnIndex >= sim.turns.length) {
              sim.active = false;
              sim.phase = 'idle';
              if (A1_x !== null) setA0_x(A1_x);
              if (A1_y !== null) setA0_y(A1_y);
              if (B1_x !== null) setB0_x(B1_x);
              if (B1_y !== null) setB0_y(B1_y);
            } else {
              sim.phase = `moving${sim.turns[sim.turnIndex].robot}`;
            }
          }
        } else {
          const pStart = path[sim.currentSegment];
          const pEnd = path[sim.currentSegment + 1];
          const lx = lerp(pStart.x, pEnd.x, sim.t);
          const ly = lerp(pStart.y, pEnd.y, sim.t);
          if (currentTurn.robot === 'A') {
            sim.posA = { x: lx, y: ly };
            sim.trailA.push({ x: lx, y: ly });
          } else {
            sim.posB = { x: lx, y: ly };
            sim.trailB.push({ x: lx, y: ly });
          }
        }
      }


      // LÓGICA DE DRAGGING DE CÁMARA
      const currentState = stateRef.current;
      if (currentState.dragging) {
        const { dragging, offsetX: currOffX, offsetY: currOffY, zoom: currZoom,
robots } = currentState;
        const draggingRobot = robots[dragging.robot];
        let activeCount = 0;
        let minX = Infinity, maxX = -Infinity, minY = Infinity, maxY = -Infinity;

        Object.values(robots).forEach(r => {
          if (r.x !== null && r.y !== null) {
            minX = Math.min(minX, r.x - r.radio); maxX = Math.max(maxX, r.x +
r.radio);
```

```
            minY = Math.min(minY, r.y - r.radio); maxY = Math.max(maxY, r.y +
r.radio);
            activeCount++;
          }
        });

        if (activeCount > 0 && draggingRobot.x !== null && draggingRobot.y !== null)
{
          const SMOOTHING_FACTOR = 0.03; const TOLERANCE = 0.5;
          const screenX = (draggingRobot.x - currOffX) * currZoom + canvasSize.width
/ 2;
          const screenY = canvasSize.height / 2 - (draggingRobot.y - currOffY) *
currZoom;
          const draggingSize = draggingRobot.radio * currZoom;
          let targetOffsetX = currOffX; let targetOffsetY = currOffY;

          if (screenX - draggingSize < viewportMargin) {
            const worldLeft = (viewportMargin - canvasSize.width / 2) / currZoom +
currOffX;
            targetOffsetX = currOffX + (draggingRobot.x - (draggingRobot.radio /
currZoom) - worldLeft);
          } else if (screenX + draggingSize > canvasSize.width - viewportMargin) {
            const worldRight = (canvasSize.width - viewportMargin - canvasSize.width
/ 2) / currZoom + currOffX;
            targetOffsetX = currOffX + (draggingRobot.x + (draggingRobot.radio /
currZoom) - worldRight);
          }
          if (screenY - draggingSize < viewportMargin) {
            const worldTop = (canvasSize.height / 2 - viewportMargin) / currZoom +
currOffY;
            targetOffsetY = currOffY + (draggingRobot.y + (draggingRobot.radio /
currZoom) - worldTop);
          } else if (screenY + draggingSize > canvasSize.height - viewportMargin) {
            const worldBottom = (canvasSize.height / 2 - (canvasSize.height -
viewportMargin)) / currZoom + currOffY;
            targetOffsetY = currOffY + (draggingRobot.y - (draggingRobot.radio /
currZoom) - worldBottom);
          }

          if (Math.abs(targetOffsetX - currOffX) > TOLERANCE / currZoom)
setOffsetX(prev => prev + (targetOffsetX - prev) * SMOOTHING_FACTOR);
          if (Math.abs(targetOffsetY - currOffY) > TOLERANCE / currZoom)
setOffsetY(prev => prev + (targetOffsetY - prev) * SMOOTHING_FACTOR);
        }
      }

      // RENDERIZADO DEL CANVAS
      const canvas = canvasRef.current;
      if (canvas) {
        const ctx = canvas.getContext("2d");
        if (ctx) {
          const w = canvasSize.width;
          const h = canvasSize.height;

          ctx.clearRect(0, 0, w, h);

          // Cálculo de posiciones animadas
          let drawAx = A0_x, drawAy = A0_y;
          let drawBx = B0_x, drawBy = B0_y;

          if (sim.active) {
            drawAx = sim.posA.x; drawAy = sim.posA.y;
            drawBx = sim.posB.x; drawBy = sim.posB.y;
          }

          // Grid y Ejes
```

```
          let dynamicSpacing = baseSpacing;
          while (dynamicSpacing * zoom < 40) dynamicSpacing *= 2;

          const startX = offsetX - (w / 2) / zoom;
          const endX = offsetX + (w / 2) / zoom;
          const startY = offsetY - (h / 2) / zoom;
          const endY = offsetY + (h / 2) / zoom;

          ctx.strokeStyle = "#e6e6e6"; ctx.lineWidth = 1;
          let xLine = Math.floor(startX / dynamicSpacing) * dynamicSpacing;
          while (xLine <= endX + dynamicSpacing) {
            const { sx } = worldToScreen(xLine, 0, w, h);
            ctx.beginPath(); ctx.moveTo(Math.round(sx) + 0.5, 0);
ctx.lineTo(Math.round(sx) + 0.5, h); ctx.stroke();
            xLine += dynamicSpacing;
          }
          let yLine = Math.floor(startY / dynamicSpacing) * dynamicSpacing;
          while (yLine <= endY + dynamicSpacing) {
            const { sy } = worldToScreen(0, yLine, w, h);
            ctx.beginPath(); ctx.moveTo(0, Math.round(sy) + 0.5); ctx.lineTo(w,
Math.round(sy) + 0.5); ctx.stroke();
            yLine += dynamicSpacing;
          }

          ctx.strokeStyle = "#000000"; ctx.lineWidth = 2;
          const { sx: axisX } = worldToScreen(0, 0, w, h);
          ctx.beginPath(); ctx.moveTo(Math.round(axisX) + 0.5, 0);
ctx.lineTo(Math.round(axisX) + 0.5, h); ctx.stroke();
          const { sy: axisY } = worldToScreen(0, 0, w, h);
          ctx.beginPath(); ctx.moveTo(0, Math.round(axisY) + 0.5); ctx.lineTo(w,
Math.round(axisY) + 0.5); ctx.stroke();

          // Corredores dinámicos
          if (showCorridors && debugFilters.corridors) {
            ctx.save();
            const safetyWidth = (radioA + radioB) * 2 * zoom;
            ctx.lineCap = "round";
            ctx.lineWidth = safetyWidth;

            if (drawAx !== null && drawAy !== null && A1_x !== null && A1_y !== null)
{
              const start = worldToScreen(drawAx, drawAy, w, h);
              const end = worldToScreen(A1_x, A1_y, w, h);
              ctx.strokeStyle = "rgba(25, 118, 210, 0.2)";
              ctx.beginPath(); ctx.moveTo(start.sx, start.sy); ctx.lineTo(end.sx,
end.sy); ctx.stroke();
            }

            if (drawBx !== null && drawBy !== null && B1_x !== null && B1_y !== null)
{
              const start = worldToScreen(drawBx, drawBy, w, h);
              const end = worldToScreen(B1_x, B1_y, w, h);
              ctx.strokeStyle = "rgba(216, 27, 96, 0.2)";
              ctx.beginPath(); ctx.moveTo(start.sx, start.sy); ctx.lineTo(end.sx,
end.sy); ctx.stroke();
            }
            ctx.restore();

            ctx.save();
            ctx.setLineDash([10 * zoom, 10 * zoom]);
            ctx.lineWidth = 2;

            if (drawAx !== null && drawAy !== null && A1_x !== null && A1_y !== null)
{
              const start = worldToScreen(drawAx, drawAy, w, h);
              const end = worldToScreen(A1_x, A1_y, w, h);
```

```
              ctx.strokeStyle = "#1976d2"; ctx.globalAlpha = 0.8;
              ctx.beginPath(); ctx.moveTo(start.sx, start.sy); ctx.lineTo(end.sx,
end.sy); ctx.stroke();
            }
            if (drawBx !== null && drawBy !== null && B1_x !== null && B1_y !== null)
{
              const start = worldToScreen(drawBx, drawBy, w, h);
              const end = worldToScreen(B1_x, B1_y, w, h);
              ctx.strokeStyle = "#d81b60"; ctx.globalAlpha = 0.8;
              ctx.beginPath(); ctx.moveTo(start.sx, start.sy); ctx.lineTo(end.sx,
end.sy); ctx.stroke();
            }
            ctx.restore();
          }

          // Visualización de zonas
          if (zoneInfo && debugFilters.zones) {
            ctx.save();
            const z = zoneInfo as any;
            const c_x = z.R_pos ? z.R_pos.x : 0;
            const c_y = z.R_pos ? z.R_pos.y : 0;
            const center = worldToScreen(c_x, c_y, w, h);

            if (z.clip && z.type !== 'multi') {
              const cS = worldToScreen(z.clip.start.x, z.clip.start.y, w, h);
              const cE = worldToScreen(z.clip.end.x, z.clip.end.y, w, h);
              const rad = z.clip.width * zoom;

              ctx.beginPath();
              const ang = Math.atan2(cE.sy - cS.sy, cE.sx - cS.sx);
              ctx.arc(cE.sx, cE.sy, rad, ang - Math.PI / 2, ang + Math.PI / 2);
              ctx.arc(cS.sx, cS.sy, rad, ang + Math.PI / 2, ang - Math.PI / 2);
              ctx.closePath();
              ctx.clip();
            }

            if (z.type === 'multi') {
              const sR_pos = worldToScreen(z.R_pos.x, z.R_pos.y, w, h);
              const stB0 = worldToScreen(z.tB0.x, z.tB0.y, w, h);
              const sp00 = worldToScreen(z.p00.x, z.p00.y, w, h);

              const sL1 = worldToScreen(z.inf_L1.x, z.inf_L1.y, w, h);
              const sL2 = worldToScreen(z.inf_L2.x, z.inf_L2.y, w, h);
              const sL3 = worldToScreen(z.inf_L3.x, z.inf_L3.y, w, h);
              const sL4 = worldToScreen(z.inf_L4.x, z.inf_L4.y, w, h);
              const sL5 = worldToScreen(z.inf_L5.x, z.inf_L5.y, w, h);

              const drawPoly = (points: { sx: number, sy: number }[], color: string,
text: string) => {
                ctx.beginPath();
                ctx.moveTo(points[0].sx, points[0].sy);
                for (let i = 1; i < points.length; i++) ctx.lineTo(points[i].sx,
points[i].sy);
                ctx.closePath();
                ctx.fillStyle = color;
                ctx.fill();

                if (text) {
                  ctx.fillStyle = "rgba(0,0,0,0.8)";
                  ctx.font = "bold 14px Arial";
                  ctx.textAlign = "center";
                  let tx = 0, ty = 0;
                  points.forEach(p => { tx += p.sx; ty += p.sy; });
                  ctx.fillText(text, tx / points.length, ty / points.length);
                }
              };
```

```
              ctx.save();
              ctx.beginPath();
              ctx.moveTo(sR_pos.sx, sR_pos.sy);
              ctx.lineTo(sL1.sx, sL1.sy);
              ctx.lineTo(sL5.sx, sL5.sy);
              ctx.closePath();
              ctx.clip();

              const arcPointsZ1: { sx: number, sy: number }[] = [];
              if (z.O_start && z.radioSeguridad) {
                const a1 = Math.atan2(z.p00.y - z.O_start.y, z.p00.x - z.O_start.x);
                let a2 = Math.atan2(z.tB0.y - z.O_start.y, z.tB0.x - z.O_start.x);
                let diff = a2 - a1;
                if (diff > Math.PI) diff -= 2 * Math.PI;
                if (diff < -Math.PI) diff += 2 * Math.PI;

                const steps = 20;
                for (let i = 0; i <= steps; i++) {
                  const ang = a1 + diff * (i / steps);
                  const px = z.O_start.x + z.radioSeguridad * Math.cos(ang);
                  const py = z.O_start.y + z.radioSeguridad * Math.sin(ang);
                  arcPointsZ1.push(worldToScreen(px, py, w, h));
                }
              }

              // Dibujamos primero las zonas 1 y 2
              drawPoly([sL2, sL1, ...arcPointsZ1], "rgb(243, 162, 150)", "");
              drawPoly([sp00, ...arcPointsZ1, sL2, sL3], "rgb(246, 210, 156)", "");

              // Proyectamos el punto T a la pantalla
              const s_ptT = worldToScreen(z.ptT.x, z.ptT.y, w, h);
              const distT_R = Math.sqrt((z.ptT.x - z.R_pos.x) ** 2 + (z.ptT.y -
z.R_pos.y) ** 2);

              // Dibujamos las zonas 3 y 4 (Con colores sólidos para evitar sombras
grises)
              if (distT_R < 0.5) {
                // CASO LÍMITE: T y el origen son el mismo punto.
                drawPoly([sL4, sL3, sp00, sR_pos], "rgb(198, 228, 198)", "");
                drawPoly([sL5, sL4, sR_pos], "rgb(204, 198, 228)", "");
              } else {
                // CASO NORMAL
                drawPoly([sL4, sL3, sp00, s_ptT], "rgb(198, 228, 198)", "");
                drawPoly([sL5, sL4, s_ptT, sR_pos], "rgb(204, 198, 228)", "");
              }

              ctx.restore();

            } else if (z.p1) {
              const p1 = worldToScreen(z.p1.x, z.p1.y, w, h);
              const p2 = worldToScreen(z.p2.x, z.p2.y, w, h);
              const p3 = worldToScreen(z.p3.x, z.p3.y, w, h);
              const p4 = worldToScreen(z.p4.x, z.p4.y, w, h);

              const drawTriangle = (ptA: { sx: number, sy: number }, ptB: { sx:
number, sy: number }, color: string, text: string) => {
                ctx.beginPath();
                ctx.moveTo(center.sx, center.sy);
                ctx.lineTo(ptA.sx, ptA.sy);
                ctx.lineTo(ptB.sx, ptB.sy);
                ctx.closePath();
                ctx.fillStyle = color;
                ctx.fill();

                ctx.fillStyle = "rgba(0,0,0,0.6)";
```

```
                ctx.font = "bold 12px Arial";
                ctx.textAlign = "center";
                const tx = center.sx + (ptA.sx - center.sx + ptB.sx - center.sx) *
0.05;
                const ty = center.sy + (ptA.sy - center.sy + ptB.sy - center.sy) *
0.15;
                ctx.fillText(text, tx, ty);
              };
              drawTriangle(p1, p3, "rgb(246, 210, 156)", ""); // Z2
              drawTriangle(p2, p4, "rgb(243, 162, 150)", ""); // Z1
              drawTriangle(p3, p2, "rgb(198, 228, 198)", ""); // Z3
              drawTriangle(p1, p4, "rgb(204, 198, 228)", ""); // Z4
            }
            ctx.restore();
          }

          // Dibujar líneas de intersección
          if (pIntLines.length === 2 && debugFilters.pint) {
            ctx.save();
            const drawExtLine = (pA: { x: number, y: number }, pB: { x: number, y:
number }) => {
              const dx = pB.x - pA.x; const dy = pB.y - pA.y;
              const len = Math.sqrt(dx * dx + dy * dy);
              if (len === 0) return;
              const ux = dx / len; const uy = dy / len;
              const sA = worldToScreen(pA.x - ux * 2000, pA.y - uy * 2000, w, h);
              const sB = worldToScreen(pB.x + ux * 2000, pB.y + uy * 2000, w, h);
              ctx.beginPath();
              ctx.moveTo(sA.sx, sA.sy);
              ctx.lineTo(sB.sx, sB.sy);
              ctx.stroke();
            };

            ctx.strokeStyle = "rgba(255, 0, 0, 0.4)";
            ctx.lineWidth = 2;
            ctx.setLineDash([8, 8]);

            drawExtLine(pIntLines[0].p1, pIntLines[0].p2);
            drawExtLine(pIntLines[1].p1, pIntLines[1].p2);
            ctx.restore();
          }

          if (pInt && debugFilters.pint) {
            const p = worldToScreen(pInt.x, pInt.y, w, h);
            ctx.beginPath();
            ctx.arc(p.sx, p.sy, 6, 0, Math.PI * 2);
            ctx.fillStyle = "red";
            ctx.fill();
            ctx.lineWidth = 2;
            ctx.strokeStyle = "white";
            ctx.stroke();

            ctx.fillStyle = "black";
            ctx.font = "bold 14px Arial";
            drawSmartText(ctx, "P_int", p.sx, p.sy - 15, 14);
          }


          const drawTrail = (trail: { x: number, y: number }[], color: string) => {
            if (trail.length < 2) return;
            ctx.save();
            ctx.beginPath();
            const first = worldToScreen(trail[0].x, trail[0].y, w, h);
            ctx.moveTo(first.sx, first.sy);
            for (let i = 1; i < trail.length; i++) {
              const pt = worldToScreen(trail[i].x, trail[i].y, w, h);
```

```
            ctx.lineTo(pt.sx, pt.sy);
          }
          ctx.strokeStyle = color;
          ctx.lineWidth = 3 * zoom;
          ctx.lineCap = "round";
          ctx.lineJoin = "round";
          ctx.globalAlpha = 0.5;
          ctx.stroke();
          ctx.restore();
        };

        // Dibujamos los rastros de movimiento de los robots (si la opción está
activada)
        if (debugFilters.trails) {
          drawTrail(sim.trailA, "#1976d2");
          drawTrail(sim.trailB, "#d81b60");
        }

        // Función para dibujar un robot (con su zona de seguridad, borde resaltado
si está activo, y etiqueta)
        const drawRobot = (x: number | null, y: number | null, r: number, color:
string, border = false, label?: string) => {
          if (x == null || y == null) return false;
          const { sx, sy } = worldToScreen(x, y, w, h);
          const size = r * zoom;

          const isDraggingThis = dragging && dragging.robot === label;
          let isHovered = false;
          if (mouseScreenPos && !dragging) {
            const dx = mouseScreenPos.sx - sx; const dy = mouseScreenPos.sy - sy;
            isHovered = Math.sqrt(dx * dx + dy * dy) <= size;
          }
          const isActive = isDraggingThis || isHovered;
          const baseWidth = border ? 2 : 1;

          if (isActive) {
            ctx.save();
            const pulse = 1 + Math.sin(time * 0.005) * 0.3;
            ctx.shadowColor = border ? "rgba(255, 165, 0, 0.8)" : "rgba(0, 0, 0,
0.6)";
            ctx.shadowBlur = 10 * pulse;
            ctx.lineWidth = baseWidth + 2; ctx.strokeStyle = color;
          } else {
            ctx.lineWidth = baseWidth; ctx.strokeStyle = color;
          }
          if (!border) ctx.fillStyle = color; else ctx.fillStyle = "transparent";

          ctx.beginPath();

          // Soporte para diferentes formas de robot (preparado para la futura
mejora de implantar la
          // lógica a robots de diferentes formas, no solo circulares)
          if (forma === "cuadrada") {
            if (!border) ctx.fillRect(sx - size, sy - size, size * 2, size * 2);
            else ctx.strokeRect(sx - size, sy - size, size * 2, size * 2);
          } else if (forma === "rectangular") {
            const wR = size * 1.5; const hR = size;
            if (!border) ctx.fillRect(sx - wR, sy - hR / 2, wR * 2, hR);
            else ctx.strokeRect(sx - wR, sy - hR / 2, wR * 2, hR);
          } else {
            ctx.arc(sx, sy, size, 0, Math.PI * 2);
            if (!border) ctx.fill(); else ctx.stroke();
          }
          if (isActive) ctx.restore();

          if (label) {
```

```
              ctx.fillStyle = border ? "#000" : "#fff";
              ctx.font = `${Math.max(10, Math.floor(10 * zoom))}px sans-serif`;
              ctx.textAlign = "center";
              ctx.textBaseline = "middle";
              drawSmartText(ctx, label, sx, sy, Math.max(10, Math.floor(10 * zoom)));
            }
            return isHovered;
          };

          let cursorPointer = false;

          // Metemos a los robots en una lista con sus datos de dibujo
          const robotsADibujar = [
            { x: A1_x, y: A1_y, r: radioA, color: "rgba(25, 118, 210, 0.3)", border:
true, label: "A1", tipo: 'destino' },
            { x: B1_x, y: B1_y, r: radioB, color: "rgba(216, 27, 96, 0.3)", border:
true, label: "B1", tipo: 'destino' },
            { x: drawAx, y: drawAy, r: radioA, color: "#1976d2", border: false,
label: "A", tipo: 'origen' },
            { x: drawBx, y: drawBy, r: radioB, color: "#d81b60", border: false,
label: "B", tipo: 'origen' }
          ];

          // Los ordenamos de MAYOR a MENOR radio (el más grande se pinta primero al
fondo)
          robotsADibujar.sort((a, b) => {
            if (b.r !== a.r) return b.r - a.r;
            // Si miden lo mismo, pintamos los destinos primero para que los sólidos
queden encima
            return a.tipo === 'destino' ? -1 : 1;
          });

          // Los pintamos en ese orden exacto
          robotsADibujar.forEach(robot => {
            if (robot.x !== null && robot.y !== null) {
              if (drawRobot(robot.x, robot.y, robot.r, robot.color, robot.border,
robot.label)) {
                cursorPointer = true;
              }
            }
          });

          if (!dragging && !isPanning.current && canvasRef.current) {
            canvasRef.current.style.cursor = cursorPointer ? "pointer" : "grab";
          }
        }
      }

      animationFrameId = requestAnimationFrame(loop);
    };

    animationFrameId = requestAnimationFrame(loop);
    return () => cancelAnimationFrame(animationFrameId);
  }, [
    canvasSize, offsetX, offsetY, zoom, forma, radioA, radioB,
    A0_x, A0_y, A1_x, A1_y, B0_x, B0_y, B1_x, B1_y,
    mouseScreenPos, dragging, showCorridors, zoneInfo, pInt, pIntLines, debugLines,
debugFilters
  ]);

  // Calculo de tics
  const { leftTicks, bottomTicks } = useMemo(() => {
    let dynamicSpacing = baseSpacing;
    while (dynamicSpacing * zoom < 40) dynamicSpacing *= 2;
    const leftWorld = offsetX - (canvasSize.width / 2) / zoom;
    const rightWorld = offsetX + (canvasSize.width / 2) / zoom;
```

```
    const topWorld = offsetY + (canvasSize.height / 2) / zoom;
    const bottomWorld = offsetY - (canvasSize.height / 2) / zoom;
    const bottomTicksData = [];
    let xTick = Math.floor(leftWorld / dynamicSpacing) * dynamicSpacing;
    while (xTick <= rightWorld + dynamicSpacing) {
      const { sx } = worldToScreen(xTick, 0, canvasSize.width, canvasSize.height);
      if (sx >= 0 && sx <= canvasSize.width) bottomTicksData.push({ value: xTick, sx
});
      xTick += dynamicSpacing;
    }
    const leftTicksData = [];
    let yTick = Math.floor(bottomWorld / dynamicSpacing) * dynamicSpacing;
    while (yTick <= topWorld + dynamicSpacing) {
      const { sy } = worldToScreen(0, yTick, canvasSize.width, canvasSize.height);
      if (sy >= 0 && sy <= canvasSize.height) leftTicksData.push({ value: yTick, sy
});
      yTick += dynamicSpacing;
    }
    return { leftTicks: leftTicksData, bottomTicks: bottomTicksData };
  }, [offsetX, offsetY, zoom, baseSpacing, canvasSize]);

  // Eventos táctiles
  const handleTouchStart = (e: React.TouchEvent<HTMLCanvasElement>) => {
    if (e.cancelable) e.preventDefault();
    const touch = e.touches[0];
    const rect = canvasRef.current!.getBoundingClientRect();
    const sx = touch.clientX - rect.left;
    const sy = touch.clientY - rect.top;
    const clickedRobotKey = isRobotHovered(sx, sy);
    if (clickedRobotKey) {
      const r = robotMap[clickedRobotKey];
      const worldPos = screenToWorld(sx, sy, canvasSize.width, canvasSize.height);
      if (r.x !== null && r.y !== null) {
        setDragging({ robot: clickedRobotKey, offsetX: worldPos.x - r.x, offsetY:
worldPos.y - r.y });
      }
    } else {
      setAutoCenter(false);
      isPanning.current = true;
      lastMouse.current = { x: touch.clientX, y: touch.clientY };
    }
  };

  const handleTouchMove = (e: React.TouchEvent<HTMLCanvasElement>) => {
    if (e.cancelable) e.preventDefault();
    const touch = e.touches[0];
    const rect = canvasRef.current!.getBoundingClientRect();
    const sx = touch.clientX - rect.left;
    const sy = touch.clientY - rect.top;
    if (dragging) {
      const worldPos = screenToWorld(sx, sy, canvasSize.width, canvasSize.height);
      const r = robotMap[dragging.robot];
      r.setX(Math.round(worldPos.x - dragging.offsetX));
      r.setY(Math.round(worldPos.y - dragging.offsetY));
    } else if (isPanning.current) {
      const dx = touch.clientX - lastMouse.current.x;
      const dy = touch.clientY - lastMouse.current.y;
      setOffsetX(prev => prev - dx / zoom);
      setOffsetY(prev => prev + dy / zoom);
      lastMouse.current = { x: touch.clientX, y: touch.clientY };
    }
  };

  const handleTouchEnd = () => {
    isPanning.current = false;
    setDragging(null);
```

```
  };

  // Escudo anti-hidratación para evitar que el canvas intente renderizar antes de
estar montado y con sus dimensiones correctas
  if (!mounted) {
    return <div className="min-h-screen w-full bg-white"></div>;
  }

  return (
    <main className="flex flex-col min-h-screen w-full bg-white overflow-x-hidden
text-[#333]">

      {/* POPUP DE MENSAJES */}
      {popup.show && (popup.type === 'error' || debugFilters.popups) && (
        <div className={`fixed top-10 left-1/2 transform -translate-x-1/2 z-50 px-6
py-3 rounded-lg shadow-2xl ${popup.type === 'error' ? 'bg-red-500 text-white' : 'bg-
green-500 text-white'}`}>
          <div className="flex items-center gap-2">
            <span className="font-bold">{popup.type === 'error' ? '✕' :
'✓'}</span><span>{popup.message}</span>
          </div>
        </div>
      )}

      {/* CABECERA */}
      <header className="shrink-0 flex items-center justify-start p-4 bg-[#004C99]
text-white shadow-md z-20 relative">
        <div className="shrink-0 z-10">
          <Image src="/icono.svg" alt="Logo" width={60} height={60} className="w-15
h-15" />
        </div>
        <h1 className="text-lg font-semibold ml-[15px] lg:absolute lg:left-1/2
lg:transform lg:-translate-x-1/2 lg:ml-0 lg:w-full lg:text-center">
          Coordinated Motion Simulator
        </h1>
      </header>

      {/* CUERPO PRINCIPAL */}
      <section className="grow flex flex-col lg:flex-row w-full relative">
        {/* COLUMNA IZQUIERDA: CANVAS */}
        <div
          ref={visualizationContainerRef}
          className="w-full lg:w-1/2 flex items-center justify-center bg-gray-50 p-4
pl-5 border-b lg:border-b-0 lg:border-r border-gray-200"
        >
          <div className={`relative transition-all duration-300 ${isFullscreen ?
'fixed inset-0 z-50 bg-black flex items-center justify-center' : ''}`}>

            {/* POPUP DE MENSAJES (SOLO PANTALLA COMPLETA) */}
            {isFullscreen && popup.show && (popup.type === 'error' ||
debugFilters.popups) && (
              <div className={`absolute top-4 left-1/2 transform -translate-x-1/2 z-
100 px-6 py-3 rounded-lg shadow-2xl ${popup.type === 'error' ? 'bg-red-500 text-
white' : 'bg-green-500 text-white'}`}>
                <div className="flex items-center gap-2">
                  <span className="font-bold">{popup.type === 'error' ? '✕' :
'✓'}</span><span>{popup.message}</span>
                </div>
              </div>
            )}

            <div className="absolute left-0 top-0 h-full w-[30px] pointer-events-
none" style={{ transform: 'translateX(-100%)' }}>
              {!isFullscreen && leftTicks.map(({ value, sy }) => (
```

```
                <span key={value} className="absolute right-1 text-[10px] text-gray-
500" style={{ top: sy, transform: 'translateY(-50%)' }}>{value}</span>
              ))}
            </div>

            <canvas
              ref={canvasRef}
              width={canvasSize.width}
              height={canvasSize.height}
              className="bg-white rounded shadow-sm border border-gray-300 cursor-
grab active:cursor-grabbing block touch-none"
              onMouseDown={handleMouseDown}
              onMouseMove={handleMouseMove}
              onMouseUp={handleMouseUp}
              onMouseLeave={handleMouseLeave}
              onTouchStart={handleTouchStart}
              onTouchMove={handleTouchMove}
              onTouchEnd={handleTouchEnd}
            />

            {/* BOTONES FLOTANTES DE INICIAR/REINICIAR (SOLO FULLSCREEN) */}
            {isFullscreen && (
              <div className="absolute bottom-4 left-1/2 transform -translate-x-1/2
flex gap-4 pointer-events-auto">
                <button
                  onClick={handleIniciar}
                  className="px-6 py-2 bg-[#004C99] hover:bg-[#003366] text-white
text-sm font-bold rounded shadow-lg transition transform hover:scale-105"
                >
                  Start
                </button>
                <button
                  onClick={handleReiniciar}
                  className="px-6 py-2 bg-[#FFA000] hover:bg-[#FF8F00] text-white
text-sm font-bold rounded shadow-lg transition transform hover:scale-105"
                >
                  Reset
                </button>
              </div>
            )}

            {/* BOTÓN CENTRAR (Solo si no está en auto) */}
            {!autoCenter && (
              <button
                onClick={() => setAutoCenter(true)}
                className="absolute bottom-2 left-2 px-3 py-1 bg-blue-600 text-white
text-xs font-bold rounded shadow hover:bg-blue-700 transition"
              >
                📍 Center
              </button>
            )}

            <div className="absolute left-0 bottom-0 w-full h-5 pointer-events-none"
style={{ transform: 'translateY(100%)' }}>
              {!isFullscreen && bottomTicks.map(({ value, sx }) => (
                <span key={value} className="absolute top-1 text-[10px] text-gray-
500" style={{ left: sx, transform: 'translateX(-50%)' }}>{value}</span>
              ))}
            </div>

            <div className="absolute top-2 right-2 flex flex-col gap-2">
              <button onClick={toggleFullscreen} className="p-1 bg-white/90 rounded
shadow hover:bg-gray-100" title="Fullscreen">
                <svg width="16" height="16" viewBox="0 0 24 24" fill="none"
stroke="currentColor" strokeWidth="2"><path d="M15 3h6v6M9 21H3v-6M21 3l-7 7M3 21l7-
7" /></svg>
```

```
              </button>
            </div>
            <div className="absolute bottom-2 right-2 flex gap-2">
              <button onClick={handleZoomOut} className="w-8 h-8 bg-gray-200 rounded
hover:bg-gray-300 flex items-center justify-center font-bold">-</button>
              <button onClick={handleZoomIn} className="w-8 h-8 bg-gray-200 rounded
hover:bg-gray-300 flex items-center justify-center font-bold">+</button>
            </div>
          </div>
        </div>

        {/* COLUMNA DERECHA: CONTROLES */}
        <div className="w-full lg:w-1/2 flex items-center justify-center p-6 bg-white
overflow-y-auto">
          <div className="w-full max-w-md flex flex-col gap-6">

            {/* MENÚ ACORDEÓN FLOTANTE DE FILTROS */}
            <div className="relative w-full mb-2">
              <button
                onClick={() => setIsFiltersOpen(!isFiltersOpen)}
                className="relative w-full p-2 border rounded bg-gray-50 hover:bg-
gray-100 transition-colors focus:outline-none flex items-center justify-center"
              >
                <span className="text-sm font-semibold text-gray-700">Visualization
Filters</span>

                <svg
                  className={`absolute right-3 w-4 h-4 text-gray-600 transform
transition-transform duration-200 ${isFiltersOpen ? 'rotate-180' : ''}`}
                  fill="none"
                  stroke="currentColor"
                  viewBox="0 0 24 24"
                >
                  <path strokeLinecap="round" strokeLinejoin="round" strokeWidth="2"
d="M19 9l-7 7-7-7"></path>
                </svg>
              </button>

              {isFiltersOpen && (
                <div className="absolute top-full left-0 w-full p-4 mt-1 flex flex-
col gap-3 border bg-white rounded shadow-lg z-50">
                  <label className="flex items-center gap-3 cursor-pointer">
                    <input type="checkbox" checked={debugFilters.corridors}
onChange={(e) => setDebugFilters(prev => ({ ...prev, corridors: e.target.checked }))}
className="w-4 h-4 text-blue-600 bg-gray-100 border-gray-300 rounded focus:ring-blue-
500" />
                    <span className="text-sm text-gray-700">Safety Corridors</span>
                  </label>
                  <label className="flex items-center gap-3 cursor-pointer">
                    <input type="checkbox" checked={debugFilters.zones} onChange={(e)
=> setDebugFilters(prev => ({ ...prev, zones: e.target.checked }))} className="w-4 h-
4 text-blue-600 bg-gray-100 border-gray-300 rounded focus:ring-blue-500" />
                    <span className="text-sm text-gray-700">Decision Geometry
(Zones)</span>
                  </label>
                  <label className="flex items-center gap-3 cursor-pointer">
                    <input type="checkbox" checked={debugFilters.pint} onChange={(e)
=> setDebugFilters(prev => ({ ...prev, pint: e.target.checked }))} className="w-4 h-4
text-blue-600 bg-gray-100 border-gray-300 rounded focus:ring-blue-500" />
                    <span className="text-sm text-gray-700">Intersection Point
(P_int)</span>
                  </label>
                  <label className="flex items-center gap-3 cursor-pointer">
                    <input type="checkbox" checked={debugFilters.trails}
onChange={(e) => setDebugFilters(prev => ({ ...prev, trails: e.target.checked }))}
```

```
className="w-4 h-4 text-blue-600 bg-gray-100 border-gray-300 rounded focus:ring-blue-
500" />
                    <span className="text-sm text-gray-700">Trajectories</span>
                  </label>
                  <label className="flex items-center gap-3 cursor-pointer">
                    <input type="checkbox" checked={debugFilters.popups}
onChange={(e) => setDebugFilters(prev => ({ ...prev, popups: e.target.checked }))}
className="w-4 h-4 text-blue-600 bg-gray-100 border-gray-300 rounded focus:ring-blue-
500" />
                    <span className="text-sm text-gray-700">System
Notifications</span>
                  </label>
                </div>
              )}
            </div>

            <div className="text-center">
              <label className="block text-sm font-semibold text-gray-700 mb-
2">Robots' Shape</label>
              <select value={forma} onChange={(e) => setForma(e.target.value)}
className="w-full p-2 border rounded bg-gray-50 text-center">
                <option value="circular">Circular</option>
                <option value="cuadrada" disabled>Square</option>
                <option value="rectangular" disabled>Rectangular</option>
              </select>
            </div>

            <div className="grid grid-cols-2 gap-4">
              {/* PANEL ROBOT A */}
              <div className="flex flex-col gap-3 p-3 bg-blue-50 rounded-lg border
border-blue-100">
                <h4 className="font-bold text-blue-800 text-center text-sm">Robot A
(Blue)</h4>
                <div className="bg-white p-2 rounded border border-blue-200">
                  <span className="text-xs text-gray-500 block mb-1">Start
(A<sub>0</sub>)</span>
                  <div className="flex gap-2">
                    <input type="number" placeholder="X" value={A0_x ?? ""}
onChange={e => setA0_x(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                    <input type="number" placeholder="Y" value={A0_y ?? ""}
onChange={e => setA0_y(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                  </div>
                </div>
                <div className="bg-white p-2 rounded border border-blue-200">
                  <span className="text-xs text-gray-500 block mb-1">End
(A<sub>1</sub>)</span>
                  <div className="flex gap-2">
                    <input type="number" placeholder="X" value={A1_x ?? ""}
onChange={e => setA1_x(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                    <input type="number" placeholder="Y" value={A1_y ?? ""}
onChange={e => setA1_y(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                  </div>
                </div>
              </div>

              {/* PANEL ROBOT B */}
              <div className="flex flex-col gap-3 p-3 bg-pink-50 rounded-lg border
border-pink-100">
                <h4 className="font-bold text-pink-800 text-center text-sm">Robot B
(Red)</h4>
                <div className="bg-white p-2 rounded border border-pink-200">
```

```
                  <span className="text-xs text-gray-500 block mb-1">Start
(B<sub>0</sub>)</span>
                  <div className="flex gap-2">
                    <input type="number" placeholder="X" value={B0_x ?? ""}
onChange={e => setB0_x(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                    <input type="number" placeholder="Y" value={B0_y ?? ""}
onChange={e => setB0_y(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                  </div>
                </div>
                <div className="bg-white p-2 rounded border border-pink-200">
                  <span className="text-xs text-gray-500 block mb-1">End
(B<sub>1</sub>)</span>
                  <div className="flex gap-2">
                    <input type="number" placeholder="X" value={B1_x ?? ""}
onChange={e => setB1_x(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                    <input type="number" placeholder="Y" value={B1_y ?? ""}
onChange={e => setB1_y(e.target.value === "" ? null : Number(e.target.value))}
className="w-full p-1 border rounded text-sm" />
                  </div>
                </div>
              </div>
            </div>

            <div className="grid grid-cols-2 gap-4">
              <div className="p-3 bg-gray-50 rounded border text-center">
                <label className="text-xs font-bold text-gray-600">Radius A</label>
                <div className="flex items-center gap-2 mt-2">
                  <button onClick={() => setRadioA((r) => Math.max(10, r - 1))}
className="w-8 h-8 bg-white border border-gray-300 rounded hover:bg-gray-100 flex
items-center justify-center font-bold text-gray-600 shadow-sm">-</button>
                  <input
                    type="range"
                    min="10"
                    max="100"
                    value={radioA}
                    onChange={e => setRadioA(Number(e.target.value))}
                    className="w-full h-2 bg-gray-200 rounded-lg appearance-none
cursor-pointer accent-blue-600"
                  />
                  <button onClick={() => setRadioA((r) => Math.min(100, r + 1))}
className="w-8 h-8 bg-white border border-gray-300 rounded hover:bg-gray-100 flex
items-center justify-center font-bold text-gray-600 shadow-sm">+</button>
                </div>
                <div className="text-xs text-gray-500 mt-1 font-mono">{radioA}
px</div>
              </div>

              <div className="p-3 bg-gray-50 rounded border text-center">
                <label className="text-xs font-bold text-gray-600">Radius B</label>
                <div className="flex items-center gap-2 mt-2">
                  <button onClick={() => setRadioB((r) => Math.max(10, r - 1))}
className="w-8 h-8 bg-white border border-gray-300 rounded hover:bg-gray-100 flex
items-center justify-center font-bold text-gray-600 shadow-sm">-</button>
                  <input type="range" min="10" max="100" value={radioB} onChange={e
=> setRadioB(Number(e.target.value))} className="w-full h-2 bg-gray-200 rounded-lg
appearance-none cursor-pointer accent-pink-600" />
                  <button onClick={() => setRadioB((r) => Math.min(100, r + 1))}
className="w-8 h-8 bg-white border border-gray-300 rounded hover:bg-gray-100 flex
items-center justify-center font-bold text-gray-600 shadow-sm">+</button>
                </div>
                <div className="text-xs text-gray-500 mt-1 font-mono">{radioB}
px</div>
              </div>
```

```
            </div>



          </div>
        </div>
      </section>

      {/* FOOTER CLÁSICO (SOLO MODO NORMAL) */}
      {!isFullscreen && (
        <footer className="shrink-0 p-4 bg-gray-100 border-t border-gray-200 flex
items-center justify-center gap-6 z-30">
          <button onClick={handleIniciar} className="px-8 py-3 bg-[#004C99] hover:bg-
[#003366] text-white font-bold rounded shadow-lg transition transform hover:scale-
105">
            Start Simulation
          </button>

          <button onClick={handleReiniciar} className="px-8 py-3 bg-[#FFA000]
hover:bg-[#FF8F00] text-white font-bold rounded shadow-lg transition transform
hover:scale-105">
            Reset
          </button>
        </footer>
      )}
    </main>
  );
}
```

Universidad de Alcalá

2025/2026